



WYDZIAŁ ELEKTRONIKI,
TELEKOMUNIKACJI
I INFORMATYKI

Imię i nazwisko studenta: Jakub Grzybowski

Nr albumu: 184670

Poziom kształcenia: Studia pierwszego stopnia

Forma studiów: stacjonarne

Kierunek studiów: Informatyka

Profil: Teleinformatyka

Imię i nazwisko studenta: Kamil Czepiel

Nr albumu: 184374

Poziom kształcenia: Studia pierwszego stopnia

Forma studiów: stacjonarne

Kierunek studiów: Informatyka

Profil: Teleinformatyka

PROJEKT DYPLOMOWY INŻYNIERSKI

Tytuł projektu w języku polskim: Społecznościowa aplikacja internetowa bazująca na algorytmie BeReal

Tytuł projektu w języku angielskim: Social network web app based on BeReal algorithm

Opiekun pracy: mgr inż. Szymon Olewniczak

Streszczenie

Niniejsza praca poświęcona jest próbie utworzenia rozwiązania dla użytkowników mediów społecznościowych, które pomoże zadbać o ich cyfrowy dobrostan, czyli dobre samopoczucie podczas korzystania ze wszelkiego rodzaju aplikacji. Nadmierne korzystanie z takich portali obecnie prowadzi do uzależnień behawioralnych, co jest szeroko opisywane w publikacjach naukowych. Aplikacja, którą autorzy opracowali na potrzeby tego projektu, umożliwia zachowanie odpowiedniej miary w korzystaniu z mediów społecznościowych, dodatkowo maksymalizując jakość relacji zawiązywanych w Internecie. Jej idea oparta jest na algorytmie innej aplikacji – „BeReal”. Zaprezentowano w niej dotychczas niespotykany mechanizm udostępniania treści w Internecie: użytkownik ma możliwość publikacji wpisu ze zdjęciem tylko raz dziennie. Pozwala to codzienne śledzenie aktywności wszystkich znajomych korzystających z aplikacji, ponieważ dana publikacja znika każdego dnia, a przychodzi powiadomienie o możliwości dodania nowej. Autorzy tej pracy skupili się nad wykorzystaniem tego pomysłu do opracowania aplikacji, której celem jest wprowadzenie nowoczesnego sposobu na łączenie ludzi online, ograniczając tym samym liczbę dziennych interakcji. To rozwiązanie bazuje na aktywnościach i zainteresowaniach użytkowników, które mogą wyrazić na swoim profilu w formie zdjęć raz dziennie. W ramach badań nad przydatnością powstającej aplikacji przeprowadzono analizę dwóch utworzonych algorytmów i porównano ich efekty.

Słowa kluczowe: aplikacje społecznościowe, cyfrowy dobrostan, filtrowanie kolaboracyjne, Java Spring, React JS, mikroserwisy.

Dziedzina nauki i techniki, zgodnie z wymogami OECD: Nauki inżynieryjne i techniczne, inżynieria informatyczna

Abstract

The following work is dedicated to an attempt to create a solution for social media users that will help take care of their digital well-being, meaning their sense of well-being while using various types of applications. Excessive use of such platforms currently leads to behavioral addictions, as widely described in scientific publications. The application developed by the authors for this project aims to maintain an appropriate balance in the use of social media while maximizing the quality of relationships formed online. Its concept is based on the algorithm of another application called "BeReal." A novel content-sharing mechanism on the Internet is presented, where users can only publish one post with a photo per day. This allows for daily tracking of the activities of all friends using the application, as each publication disappears every day, with a notification prompting the addition of a new one. The authors of this work focused on utilizing this idea to develop an application that introduces a modern way of connecting people online while reducing the number of daily interactions. This solution is based on the activities and interests of users, which they can express on their profile in the form of photos once a day. While researching the utility of the emerging application, an analysis of two created algorithms was conducted and their effects were compared.

Keywords: social applications, digital well-being, collaborative filtering, Java Spring, React JS, microservices.

Spis treści

Streszczenie.....	2
Abstract	3
Autorstwo rozdziałów w pracy inżynierskiej	7
Wykaz ważniejszych oznaczeń i skrótów	8
1. Wstęp i cel pracy.....	9
2. Fenomen aplikacji społecznościowych	10
2.1. Funkcyjność ułatwia codzienność.....	10
2.2. Nowe znajomości jeszcze bliżej online	10
2.3. Aplikacje łączące użytkowników	11
2.4. Nowe platformy – nowe możliwości	11
2.5. Bariera w mediach społecznościowych	11
2.6. Problem ukazywania prawdy w Internecie.....	12
3. Cyfrowy dobrostan – wyzwanie XXI wieku	13
3.1. Definicja i zakres problemu	13
3.2. Cyfrowe zagrożenia	13
3.2.1. Algorytmy skupiające uwagę użytkownika	13
3.2.2. Aplikacje zakłamujące rzeczywistość.....	14
3.2.3. Aplikacje powodujące „krótkowzroczność”	15
4. Przegląd dostępnych rozwiązań	17
4.1. Instagram	17
4.1.1. Podsumowanie rozwiązań aplikacji Instagram	19
4.2. Hinge	20
4.2.1. Podsumowanie rozwiązań aplikacji Hinge	23
4.3. BeReal.....	24
4.3.1. Sposób działania BeReala	25
4.3.2. Podsumowanie aplikacji BeReal	26
5. Założenia aplikacji.....	27
5.1. Streszczenie.....	27
5.2. Filary aplikacji.....	27
5.3. Spełnione założenia w wersji deweloperskiej	27

6.	Planowanie i projekt aplikacji	28
6.1.	Planowanie funkcjonalności	28
6.2.	Propozycje mechanizmów łączenia użytkowników	29
7.	Wybór technologii i proces wytwarzania	31
7.1.	Wybór technologii (Kamil Czepiel).....	31
7.2.	Wytwarzanie aplikacji (Jakub Grzybowski).....	32
7.2.1.	Repozytoria Git	32
7.2.2.	Wersja aplikacji na zakończenie prac inżynierskich	35
8.	Architektura i logika aplikacji	37
8.1.	Architektura mikroservisów	37
8.1.1.	Wyznaczone mikroservisy	38
8.1.2.	Omówienie działania aplikacji w utworzonej architekturze	38
8.2.	Uwierzytelnianie i autoryzacja	38
8.2.1.	OAuth2.0	39
8.2.2.	OpenID Connect	40
8.3.	Baza danych	40
8.3.1.	PostgreSQL	40
8.3.2.	Liquibase	41
8.3.3.	Struktura tabel	42
8.4.	Application Programming Interface	43
8.4.1.	REST (Representational state transfer)	43
8.4.2.	GraphQL	44
8.4.3.	Websocket	46
8.5.	Logika mikroservisów w Spring Framework	47
8.5.1.	Maven	47
8.5.2.	Struktura projektu	48
8.5.3.	Techniki wytwarzania oprogramowania w projekcie	48
9.	Zaproponowane mechanizmy łączenia użytkowników	53
9.1.	Model użytkownika	53
9.2.	Algorytmy łączenia.....	54
9.2.1.	Odległość cosinusowa	54

9.2.2.	Algorytm losowy	55
9.2.3.	Algorytm ze wstępnym posortowaniem	56
10.	Analiza opracowanych algorytmów	57
10.1.	Miara skuteczności	57
10.2.	Perfekcyjny przyjaciel	57
10.3.	Wyniki pomiaru algorytmów	58
10.4.	Połączenia zaproponowane przez algorytm	59
10.4.1.	Wybrane połączenia	60
11.	Interfejs graficzny aplikacji	66
11.1.	Budowanie aplikacji w React	66
11.1.1.	Konfiguracja i zależności w package.json	66
11.1.2.	Struktura projektu	67
11.1.3.	Komponenty	68
11.1.4.	Props	69
11.1.5.	Stany oraz useHook	69
11.1.6.	React Router	70
11.2.	Integracja z logiką aplikacji	71
11.2.1.	API	71
11.2.2.	Keycloak	71
11.3.	Graficzny Interfejs Użytkownika	73
11.3.1.	Strona główna i logowanie	73
11.3.2.	Profil użytkownika i aktywności	73
11.3.3.	Połączenia	74
11.3.4.	Podgląd wszystkich profili użytkowników	76
11.3.5.	Chat	76
12.	Konteneryzacja i wdrożenie	77
12.1.	Docker (Kamil Czepiel)	77
12.1.1.	Dockerfile	78
12.2.	Serwer deweloperski aplikacji (Jakub Grzybowski)	79
12.2.1.	Topologia środowiska deweloperskiego	80
13.	Podsumowanie	82

13.1. Wnioski z projektu inżynierskiego.....	82
Wykaz literatury	84
Spis grafik.....	85
Spis listingów	87
Spis tabel	89
Załączniki / dodatki	90

Autorstwo rozdziałów w pracy inżynierskiej

1. Jakub Grzybowski odpowiada za:

- Rozdziały od 1. do 6.
- Fragmenty rozdziału 7.
- Rozdział 11.
- Fragmenty rozdziału 12.
- Rozdział 13.

2. Kamil Czepiel odpowiada za:

- Fragmenty rozdziału 7.
- Rozdziały od 8. do 10.
- Fragmenty rozdziału 12.

Wykaz ważniejszych oznaczeń i skrótów

AI (z ang. *Artificial Intelligence*) – Sztuczna Inteligencja.

API (z ang. *Application Programming Interface*) – Interfejs służący do komunikacji między aplikacją a jej logiką w celu uzyskiwania zasobów.

AWS (z ang. *Amazon Web Services*) – Platforma chmurowa firmy Amazon oferująca różne usługi deweloperskie i wdrożeniowe.

CI/CD (z ang. *Continuous Integration/Continuous Delivery*) – Ciągła integracja aplikacji, ciągłe wdrażanie aplikacji. Techniki wspomagające publikowanie aplikacji.

ERD (z ang. *Entity-Relationship Diagram*) – diagram służący do przedstawienia zależności między encjami w relacyjnej bazie danych.

HTTP (z ang. *Hypertext Transfer Protocol*) – Protokół komunikacyjny wykorzystywany w sieci komputerowej do przesłania zasobów przy zdefiniowaniu tzw. URI.

JPA (z ang. *Java Persistent API*) – Interfejs dla języka Java, który definiuje sposoby zarządzania trwałymi zasobami danych.

JS (z ang. *JavaScript*) – Język programowania przeznaczony do aplikacji przeglądarkowych.

JSON (z ang. *JavaScript Object Notation*) – Lekki format tekstowy do wymiany danych.

REST (z ang. *Representational State Transfer*) – Styl architektoniczny stosowany w projektowaniu aplikacji.

SQL (z ang. *Structured Query Language*) – Język zapytań wykorzystywany do komunikacją z relacyjną bazą danych.

URI (z ang. *Uniform Resource Identifier*) – Identyfikator zasobu, do którego odwołuje się protokół HTTP.

1. Wstęp i cel pracy

Postęp technologii i ciągły rozwój Internetu Rzeczy (po ang. *Internet of Things*, IoT) sprawił, że wiele codziennych aktywności odbywa się na smartfonach, które duża część osób ma przy sobie cały czas. Powoduje to swoistą więź z urządzeniem, które jest w stanie pomóc we wielu problemach oraz dostarczyć cennych informacji. Ciągła zależność od jego funkcji niesie za sobą znamienne konsekwencje: buduje to przywiązane przejawiające się częstej potrzebie sięgania po telefon w chwilach, kiedy nie jest to konieczne [1]. Nie ma potrzeby tak naprawdę korzystać z jego funkcji, a i tak jest używany. Kreowany jest także lęk przed utratą tego kluczowego dla codziennego życia narzędzia. Można zauważyć, że ludzie w dobie wszechobecnych inteligentnych urządzeń coraz rzadziej bazują na swojej wiedzy czy pamięci w rozwiązywaniu drobnych, codziennych spraw, a częściej powierzają te zadania elektronicznym asystentom.

Przedstawione efekty zniekształcają także relacje ludzi, którzy korzystają z aplikacji społecznościowych na tych smartfonach. Ciągła styczność z ekranem, mechaniczne wręcz użytkowanie zawartości, przysłania prawdziwą wartość, która stoi po drugiej stronie – żywą osobę.

W tym projekcie skonfrontowano popularne obecnie rozwiązania w dziedzinie mediów społecznościowych. Jego cel to zaproponowanie sposobu na ograniczenie nadmiernego korzystania z tego typu aktywności. Autorzy pragną zaoferować użytkownikowi aplikację, która dba o jego cyfrowy dobrostan, ale pozwala mu również pozostawać w kontakcie z bliskimi, jak i otwierać na nowe znajomości. W rezultacie daje to szansę na pogłębienie relacji lub zbudowania nowych i wartościowych. Aktualne portale będące środowiskiem interakcji wśród ludzi skupiają się głównie na swoim zysku. Jest to realizowane poprzez reklamy, treści promowane, a sam mechanizm działania tych aplikacji ma na celu maksymalizację czasu spędzonego przed ekranem.

W niniejszej pracy opisano autorskie rozwiązanie, jakim jest aplikacja „Once a Day”. W dalszych częściach rozszerzone zostały rozważania na temat kondycji psychicznej ludzi wobec uzależnienia od smartfonów oraz wprowadzony został termin „cyfrowego dobrostanu” i jak do niego dążyć. Dokonano przeglądu współczesnych aplikacji społecznościowych i przedstawiono wyzwania, z jakimi muszą się zmagać, a następnie zestawiając wymagania i założenia powstałe w wyniku analizy, kompleksowo scharakteryzowano wytwarzaną aplikację. Opisano również przeprowadzone badanie nad zaimplementowanymi w niej algorytmami, które łączą użytkowników w aplikacji na podstawie ich zainteresowań. Jest to jej flagowa funkcjonalność, bez której nie odstawałaby od konkurencji. Na końcu pracy przedstawiono wnioski oraz dołączono kompletne pliki źródłowe projektu.

2. Fenomen aplikacji społecznościowych

Na początku pracy, rozważano nad kwestiami dotyczącymi mediów społecznościowych oraz nakreślono skalę zjawiska. Następnie odnotowano różne funkcje, które są świadczone przez takie portale. Przytoczono również fakty na ich temat, porównano zalety, przyczyniające się społeczeństwu, jak i podkreślono pewne bariery, z którymi się zmagają.

2.1. Funkcyjność ułatwia codzienność

XXI wiek przyniósł niespotykany dotąd rozwój technologii mobilnych. Telefon komórkowy dawno przestał pełnić funkcję jedynie aparatu do rozmów, a stał się jedną z ważniejszych rzeczy osobistych. Jego możliwości są cały czas poszerzane, z poziomu obecnych smartfonów można zamówić zakupy do domu, kupić bilet na przejazd komunikacją publiczną czy wylegitymować się, a nawet załatwić sprawę urzędową. Do tego wszystkiego dochodzi masowa „migracja” relacji międzyludzkich do specjalnie zaprojektowanych aplikacji. Jeśli nastąpi potrzeba zobaczenia swojego znajomego na żywo, to wystarczy smartfon i dostęp do szybkiego Internetu. Można już wtedy połączyć się z nim przy użyciu jednego z internetowych komunikatorów.

Aplikacje społecznościowe umożliwiły ludziom zawiązywanie relacji na odległość, poznawanie się drogą *online* czy wymianę zdań na forach. W szerszej perspektywie, na takich portalach skorzystali również celebryci oraz osoby szcycące się popularnością. Dzięki takim platformom jak Facebook czy Instagram mogli w łatwiejszy sposób trafić do swoich odbiorców poprzez wpisy i odpowiedzi w sekcji komentarzy.

2.2. Nowe znajomości jeszcze bliżej online

Wiele potrzeb zostało zaadresowanych poprzez postęp smartfonów i ich aplikacji. Należy nadmienić, że ludzie coraz częściej za pomocą tych narzędzi znajdują sobie nowych znajomych czy nawet drugie połówki. Na podstawie wyliczeń uniwersytetu Stanford [2], zauważono znaczny wzrost poznawania nowych osób online z 2% w 1995 roku do prawie 40% w 2017. Jest to statystycznie najbardziej popularna opcja. Druga pozycja – poznawanie osób w barze lub restauracji – traci ponad 10 punktów procentowych. W ankiecie wzięło udział ponad 5 tysięcy dorosłych osób mieszkających w Stanach Zjednoczonych [2].

Statystyka ta w obecnych czasach nie dziwi i podkreśla, jaką rolę obecnie odgrywa Internet w komunikacji interpersonalnej i zawiązywaniu nowych znajomości. Jest to najszybszy sposób na porozumienie się z jakąkolwiek osobą na świecie, z którą nas coś łączy. Zapewnia sporą elastyczność, ponieważ udostępniamy tylko te informacje o sobie, które chcemy zaprezentować innym. Tę cechę doceniają osoby nieśmiałe czy o usposobieniu introwertycznym. Dzięki takim udogodnieniom zawsze mogą w swoim tempie i strefie komfortu, rozpoczynać nowe relacje na odległość. Właśnie ta właściwość, jak i prostota korzystania ze wszelkiego rodzaju aplikacji internetowych oraz ich dostępność, pozwoliła na utworzenie portali społecznościowych oraz randkowych. Umożliwiają one użytkownikom na stworzenie swojego profilu, na którym przedstawiają, kim są. Tak jak wspomniano powyżej, trafiają tam tylko te informacje, które chciano pokazać drugiej osobie. Przeglądając inne profile, można dostrzec, czy ktoś dzieli wspólne zainteresowania, czy ma zbliżone poczucie humoru, a nawet podobieństwo wyznawanych wartości.

2.3. Aplikacje łączące użytkowników

Aż 81% osób z grupy, która szuka relacji online, odpowiedziało w ankiecie, że poznaje kogoś zupełnie nowego na własną rękę [2]. Narzędzia pozwalające na budowanie nowych znajomości są w ciągłym rozwoju oraz w przyroście popularności. Nie tylko „klasyczne” media społecznościowe (tj. Facebook¹ czy Instagram²) zyskują, ale również aplikacje randkowe, które przekazały pełną wolność użytkownikom w doborze drugiej połówki.

Firma eMarketer przeprowadziła badania dotyczące użytkowników owych aplikacji [3]. Na podstawie danych, w 2020 roku ich liczba zwiększyła się blisko o 20% w porównaniu do poprzedniego roku i osiągnęła liczbę 26,6 miliona osób. Mogło być to spowodowane pandemią, podczas której całe nasze życie przeniosło się do „chmury” [3]. Wszystkie dotychczasowe obowiązki, wykonywane zawody, jak i aktywności musiały się odbywać w sposób bezkontaktowy, do czego w niektórych przypadkach posłużył Internet. Takie rzeczy, jak potrzeba interakcji z drugą osobą, rozmowy na żywo, codziennego spotkania, musiała zostać zaspokojona w inny sposób. Stąd może wynikać ów przyrost aktywnych użytkowników na portalach randkowych.

W 2021 zanotowano spadek o 8 punktów procentowych [3], żeby w kolejnych dwóch latach ponownie zaliczyć wzrost o 2,5 p.p. Ta liczba waha się w okolicach 25 milionów użytkowników, co zwraca uwagę, jak wiele osób korzysta z tej formy randkowania. Żadna inna opcja nie dostarcza młodym ludziom tak przystępnego i zwinnego sposobu na poznanie rówieśników. Wśród nastolatków i młodych dorosłych te rozwiązania technologiczne przyniosły wiele komfortu i zaadresowało potrzeby występujące w tej grupie wiekowej.

2.4. Nowe platformy – nowe możliwości

Z wyliczeń projektu State of the Young Nation organizacji Savanta [4] można wyczytać, że nowe aplikacje do poznawania ludzi wchodzi na rynek i zajmują jej niemałą część. W 2015 roku Tinder zajmował większość udziałów w rynku aplikacji randkowych. W 2020 nowopowstałe aplikacje Bumble czy Hinge zanotowały dobry start wśród ankietowanych i zyskały na popularności [4]. W te statystyki wchodzi odpowiedzi młodzieży, która jak przytoczono stanowi większość zainteresowanych tym tematem. W raporcie State of the Young Nation można przeczytać:

„Młodzi ludzie coraz częściej korzystają z aplikacji randkowych i stron internetowych w poszukiwaniu związków. W 2018 roku 28% osób w wieku 16-24 lat było zarejestrowanych w co najmniej jednej aplikacji randkowej, co stanowi wzrost z 22% sprzed trzech lat (2015). Podczas pandemii Tinder nadal dominował na rynku. Tinder odnotował wzrost dziennych rozmów o 12% w czasie lockdownu oraz ponad trzy miliardy przesunięć palcem na całym świecie. Jednak, według naszych danych trendowych, korzystanie z tej aplikacji nie zmieniło się znacząco w ciągu ostatnich pięciu lat. Możemy jednak zauważyć nowych uczestników pojawiających się na rynku.” [4]

2.5. Bariera w mediach społecznościowych

Nie da się zaprzeczyć, że proces zawiązywania nowych relacji prawdopodobnie zawsze już będzie związany z aktywnościami prowadzonymi online. Sama inicjacja znajomości może mieć

¹ Popularny serwis społecznościowy, źródło: <https://pl.wikipedia.org/wiki/Facebook>

² Popularny serwis do publikowania zdjęć, źródło: <https://pl.wikipedia.org/wiki/Instagram>

miejsce w świecie rzeczywistym, ale prędzej czy później przechodzi ona do Internetu. W taki sposób najprościej pozostawać w kontakcie przez cały czas. Poznanie się dwójki osób najpierw w rzeczywistości ma dla nich wiele korzyści, jednym z nich jest wzajemna świadomość autentyczności. Taka para ma wiedzę na swój temat z prawdziwego życia, a nie taką wyciągniętą z profilu na jednym z portali. Jest zaznajomiona ze swoimi reakcjami, sposobem bycia, wie, co drugą osobę śmieszy, a co wprowadza ją w zły nastrój. Jest to sytuacja przejrzysta dla tych osób, ogromnym wyzwaniem jest wprowadzenie takiego poziom autentyczności i prawdy do świata wirtualnego.

O ile aplikacje społecznościowe i randkowe wprowadziły niebywałe możliwości, umożliwiając poznanie osób, których prawdopodobnie nie dałoby się spotkać przypadkowo na żywo, to trzeba zwrócić uwagę na to, w jaki sposób kreowana jest każdego internetowa postać. Aplikacje nie wymagają, aby umieszczać tam wyłącznie zweryfikowane informacje, zostawia to pole do swojego wizerunku i osobowości. Na profilu takich aplikacji można udostępniać zdjęcia, które stawiają użytkowników jedynie w dobrym świetle, uwypatniają zalety. Nie odsłaniane jest natomiast to, co złe, nieidealne, niereprezentatywne, ale niemniej wciąż prawdziwe.

W swoich skutkach może to wzbudzić rozczarowanie, w końcu tylko rzeczywistość jest w stanie zweryfikować, czy kreacja utworzona online odpowiada prawdziwej osobie. Nie dotyczy się to wyłącznie udostępniania zdjęć, a więc prezentacji swojego wizerunku, ale poprzez media społecznościowe użytkownicy mogą tworzyć kompletnie nowe osoby i podpisywać je swoim nazwiskiem. Zdarza się, że osoba, z którą przeprowadza się dyskusję w sieci, diametralnie różni się od tej, którą można poznać w rzeczywistości. Maniera wypowiedzi, ludzkie gesty czy drobne szczegóły w zachowaniu budują tożsamość, której nie da się przełożyć do świata cyfrowego.

2.6. Problem ukazywania prawdy w Internecie

Zaadresowanie rozważanych problemów okazuje się nie lada wyzwaniem dla współczesnego świata. Nie można oczekiwać, że użytkownicy zrezygnują z wolności przedstawiania siebie w jakim najlepszym świetle. Wymuszenie weryfikacji publikowanych treści w Internecie pod kątem autentyczności i zgodności z prawdą wpłynęłoby rażąco na społeczność. W takim wypadku trzeba wyjść poza ramy obecnej formy, jaką różne portale wprowadziły w ciągu przestrzeni ostatnich lat. Nowe rozwiązania nie mogą skupiać się na zmaksymalizowaniu zysku z perfekcyjnego profilu. Odbiorcy takich aplikacji muszą czuć, że pokazywanie prawdy o sobie, niekoniecznie tej pięknej i idealnej, może pozytywnie wpłynąć na ich odczucia z użytkowania tej platformy. Znalezienie formuły na sukces o takiej specyfikacji w końcu mogłoby zwiększyć autentyczność ludzi w sieci. Zbudowanie pewności siebie, odwaga wyjścia na zewnątrz ze swoją prawdziwą tożsamością nieodwracalnie zmieniłoby podejście do mediów społecznościowych w dzisiejszych czasach. Jednakże pozostaje pytanie – jak zaprojektować takie rozwiązanie i jak osiągnąć nim sukces?

3. Cyfrowy dobrostan – wyzwanie XXI wieku

W poniższym rozdziale omówiono kluczowy temat w dyskusji o mediach społecznościowych i ich konsekwencji, a mianowicie cyfrowy dobrostan. Przedstawiono zagadnienie oraz podano przykłady ewentualnych zagrożeń, które mogą występować w jego obszarze.

3.1. Definicja i zakres problemu

Cyfrowy dobrostan (po ang. *digital well-being*) to termin coraz częściej pojawiający się przy okazji dyskusji o nowych technologiach. Dotyczy on zachowania odpowiedniej miary w korzystaniu z elektroniki, głównie z urządzeń z dostępem do Internetu [5]. Obecnie konsumpcja cyfrowej treści często prowadzi do uzależnień i złych nawyków. Może powodować problemy w sferze mentalnej, takie jak pogorszenie skupienia człowieka, jego koncentracji i samopoczucia, ale także atakuje jego zdrowie fizyczne [6]. Zbyt częste i intensywne korzystanie z tych rzeczy prowadzi do bezsenności, gorszej wydolności organizmu i generalnego rozdrażnienia. Cyfrowy dobrostan jest więc pożądanym celem, zestawem reguł, które pozwalają uniknąć doświadczenia tych dolegliwości [5]. Jego osiągnięcie wymaga samodyscypliny i zdrowego balansu między tym, na co można sobie pozwolić, a tym, co uważa się jako nadużycie [5]. Problem dotyczy głównie młodych osób, które od wczesnego momentu swojego życia mają styczność z elektroniką i są bardziej podatni na jej ewentualne zagrożenia [5]. Nie pomaga fakt, że współcześnie takie urządzenia jak smartfony czy *smartwatche* (z ang. inteligentne zegarki) potrafią spełnić wiele potrzeb człowieka. Poruszono tę kwestię w podrozdziale 2.1., opisano, że technologia wyręczyła ludzi z wielu obowiązków, lecz trzeba pamiętać o konsekwencjach, które za tym idą. Tak jak podkreślano wcześniej, przywiązanie do smartfona dzięki jego możliwościom ma swoje skutki uboczne.

Ustalona etyka użytkowania nowoczesnych urządzeń, nazywana tutaj jako cyfrowy dobrostan, jest czymś, na czym warto się skupić, projektując nowe rozwiązania technologiczne. Idea inżynierska musi się opierać na tym, aby służyć użytkownikowi i dbać o jego dobro, kiedy ma z nią styczność.

3.2. Cyfrowe zagrożenia

Istnieje wiele przeszkód, które utrudniają osiągnięcie cyfrowego dobrostanu. Poniżej zostaną przedstawione niektóre z nich, głównie napotykane w aplikacjach społecznościowych. Pozwoli to na analizę pod kątem usprawnień, jakie można zawrzeć, projektując autorski projekt, który jest istotą tej pracy.

3.2.1. Algorytmy skupiające uwagę użytkownika

- **Opis:** Technika ta opiera się na analizie zachowania użytkownika, preferencji i interakcji z treściami w aplikacji. W rezultacie mechanizm dostarcza coraz więcej sprofilowanych materiałów, które są bardziej atrakcyjne dla konkretnej osoby. To pozwala ją bardziej zaangażować, przez co czas spędzony w aplikacji się wydłuża.
- **Sposób:** Użytkownik, dodając reakcję, komentarz lub udostępniając wpis, przekazuje informacje do modelu uczenia maszynowego, który przewiduje, z jakiego typu treścią ta osoba jest bardziej skłonna do wejścia w interakcję.

- **Zagrożenie:** Użytkownik, który otrzymuje treści dopasowane do jego upodobań, będzie bardziej skłonny do pozostania dłużej w aplikacji. W efekcie może dojść do wypracowania odruchu, który łączy uruchomienie aplikacji z miłym doświadczeniem [7]. Ogranicza to w dużym stopniu percepcję człowieka na to, co otrzymuje w tych materiałach. Przez to może zostać ofiarą manipulacji w przypadku, gdy zostanie mu dostarczona fałszywa informacja. Budzi to lęk przed wytworzeniem masowego zakłamania prawdy lub nakierowaniem osób na niepoprawną, możliwe też krzywdzącą lub niebezpieczną opinię. Oprócz tego, taki algorytm rozwija w użytkowniku uzależnienie. Spędzanie większej ilości czasu przed ekranem powoduje wiele skutków ubocznych, między innymi zmęczenie czy pogorszenie koncentracji, a w skrajnych przypadkach nawet bezsenność i depresję.
- **Próba rozwiązania problemu:** Twórcy nowopowstałych aplikacji mogą ograniczyć profilowanie danych użytkownika poprzez ponowne zdefiniowanie ich głównego celu i założenia. Przykładowo na początku, strona główna użytkownika na Facebooku i Instagramie była wypełniona w większości wpisami znajomych połączonych w aplikacji, potem jednak pojawiły się reklamy i treści promowane, które zaburzyły tę hierarchię. Zauważa się trend, w którym użytkownicy coraz rzadziej dzielą się informacjami o sobie i swoim życiu w tych aplikacjach. Powrót do pełnoprawnego forum komunikacyjnego, w którym znajomi wymieniają się swoimi spostrzeżeniami, fotografiami czy innego rodzaju mediami mógłby być odświeżeniem na rynku portali społecznościowych.

3.2.2. Aplikacje zakłamujące rzeczywistość

- **Opis:** Zagrożenia w cyberprzestrzeni dotyczą również relacji interpersonalnych. Tak jak przedstawiono wcześniej, poznawanie nowych osób poprzez różne aplikacje społecznościowe stało się najbardziej popularną formą zawiązywania relacji. Mimo ogromu zalet, które ta metoda posiada, wyodrębniły się pewne problemy. Głównym z nich jest bariera, którą stawia między ludzi Internet. Ciężko jest wyrazić siebie tam w taki sposób, aby osoba po drugiej stronie mogła poczuć, że obcuje ze swoją nową znajomością w pełnym wymiarze, tak jakby to robiła, rzeczywiście się spotykając.
- **Zagrożenia:** Użytkownik dodaje swoje wyselekcjonowane zdjęcia – publikuje tylko takie, na których dobrze wygląda. Fotografie mogą być już nieaktualne, ponieważ wygląd takiej osoby mógł się zmienić w czasie. Zakłamuje to rzeczywistość i wpływa na dalsze losy zawiązywanych relacji. Można to odnieść nie tylko do przedstawiania swojego zewnętrznego wizerunku, ale całej „kreacji” na portalach społecznościowych. Pełna dowolność i brak ograniczeń w doborze treści udostępnianych na profilach (poza tymi niespełniającymi regulamin korzystania z aplikacji) pozwala użytkownikom na zaprezentowaniu siebie wręcz w idealnym świetle. Taki manewr balansuje między tym, co prawdziwe, a tym, co w pewien sposób zostało zmodyfikowane, ulepszone i upiękzone. W rezultacie taki obraz człowieka nie jest spójny z prawdą i stanowi realne zagrożenie.
- **Próba rozwiązania problemu:** Aby zaproponować wyjście z tej sytuacji, należy się zastanowić się, w jaki sposób przekonać użytkowników, aby chcieli dzielić się rzetelnymi informacjami o sobie. Przede wszystkim, nikogo nie da się zmusić, aby tak postępował.

Jedyne co można narzucić to formę, pewne wspólne zasady, które dotyczyłyby wszystkich użytkowników takiej aplikacji. Dzięki temu istnieje szansa, że każdy zechciałby wedle takiego schematu operować. Budowana byłaby odwaga i pewność siebie użytkowników, ponieważ pozostałe osoby również godziłyby się na przedstawianie siebie w pełnej okazałości, również od tej nieidealnej strony. Takie podejście spopularyzował BeReal [8]. Aplikacja wypracowała niepisany pakt między użytkownikami, w którym każdy udostępnia swoje zdjęcie i zdjęcie tego, co aktualnie robi albo gdzie się znajduje – o przypadkowej, nieznanego porze dnia. Rozwinięcie tego pomysłu może wyeliminować analizowane zagrożenie.

3.2.3. Aplikacje powodujące „krótkowzroczność”

- **Opis:** Coraz częściej nowoczesne formy poznawania ludzi zawodzą. Internet potrafi być uzależniający, ponieważ treści, które w nim przeglądamy, są dostosowane pod nas, co podkreślono w pierwszym wymienionym zagrożeniu. Taki stan rzeczy sprawia, że widok ekranu smartfona sprawia przyjemność, wprowadza ogromne zapałanie i uczula na elementy, które widzimy.
- **Zagrożenia:** Skupienie uwagi użytkownika na osobie w mediach społecznościowych zaczyna się w większości przypadków od sprawdzenia zdjęcia profilowego i pozostałych zdjęć na profilu. Może to być niekomfortowe, a czasami krzywdzące, że opinię o kimś w Internecie można zbudować tylko po przejrzeniu zdjęć danej osoby. Dlatego, użytkownicy bardzo starają się, aby wypaść jak najlepiej na swojej „wizytówce”, co ponownie przysparza pewne komplikacje i niewydolność poprawnego funkcjonowania w sieci. Powraca tutaj kwestia zakłamania w pewnym stopniu, ponieważ jest wiadome, że ktoś, szukając znajomych, a już w ogóle drugiej połówki będzie kierował się atrakcyjnością drugiej osoby. Powoduje to ciągłą gonitwę w budowaniu estetycznych i nieskazitelnych profili. Tak silne przywiązanie do kwestii wizerunku w Internecie nie tylko zakłamuje faktyczny stan rzeczy, ale wprowadza w kompleksy osoby nieuczestniczące w tego typu ewenementach. Użytkownicy takich aplikacji jak Tinder³ (portal poznawania osób i do randkowania) w celu odnalezienia swojej drugiej połówki przeglądają dziennie setki zdjęć innych osób. Liczba osób zarejestrowanych tam powoduje, że nie można skupić swojej uwagi tylko na kilku profilach i dogłębnego przejrzania czy ma się z kimś coś wspólnego. Takie informacje jak zainteresowania, hobby, ulubiona forma spędzania czasu, cele na przyszłość, wyznawane wartości to zazwyczaj elementy drugorzędne, które nie są aż tak wyeksponowane, jak zdjęcie profilowe. Wówczas wybór rzeczywiście sprowadza się do oceny zewnętrznej, co bardzo stagnuje zdrowe zawiązywanie relacji.
- **Próba rozwiązania problemu:** Widać wyraźnie, że istnieje potrzeba, aby ponownie zachęcić użytkowników do zmiany swoich przyzwyczajeń. W tym przypadku warto by mocniej naświetlić i wykorzystać zainteresowania, aktywności i upodobania użytkowników w celu budowania ich profili. Położenie nacisku na faktyczną wartość człowieka, czyli to,

³ Serwis randkowy, źródło: <https://pl.wikipedia.org/wiki/Tinder>

co go kształtuje i sprawia, że jest, jaki jest, może pokazać ludziom, że to jest prawdziwa istota mediów społecznościowych. Mogłoby to zachęcić, aby przedstawiać czy nawet pochwalić się swoim wnętrzem i dać upust wyobraźni, co identyfikuje człowieka w szerszym spojrzeniu niż jedynie jego wizerunek.

4. Przegląd dostępnych rozwiązań

Pomysł na aplikację narodził się w wyniku analizy istniejących rozwiązań w obszarze mediów społecznościowych oraz funkcji, które oferują. Przeprowadzono szczegółową analizę modeli, na których opierają się te aplikacje, aby zainspirować się najlepszymi funkcjonalnościami we własnym projekcie. Trzeba jednocześnie zidentyfikować konkretne potrzeby, które dotychczasowe produkty nie spełniają i zaprojektować ich usprawnienia. Kluczowe również będą kwestie dotyczącego cyfrowego dobrostanu, trzeba podjąć aktywne działanie w celu ochrony użytkowników przed omówionym we wcześniejszy częściach pracy zagrożeniami.

4.1. Instagram

Początkowo, w 2010 roku, Instagram był platformą umożliwiającą użytkownikom udostępnianie zdjęć, prezentowanych na ich profilu w formie siatki kwadratowych obrazów – nawiązujących do formatu wyjściowego aparatów Polaroid lub aparatów Kodak z serii Instamatic. Aplikacja została stworzona jako odpowiedź na rozwijającą się fotografię mobilną, w odróżnieniu od innych platform społecznościowych, gdzie dodawanie zdjęć do wpisów było opcjonalne. Instagram skupił się na oferowaniu dedykowanych narzędzi do edycji fotografii ze smartfonów, umożliwiając dodawanie filtrów, retuszowanie, modyfikowanie odcieni, jasności oraz kontrastu. Wszystkie te funkcje sprawiły, że proces publikacji zdjęć w aplikacji stał się pewnego rodzaju twórczym doświadczeniem, które zdobyło uznanie użytkowników. Przykłady filtrów nałożonych na zdjęcia zostały pokazane na grafice.



Graf. 4.1: dedykowane filtry dla zdjęć we wczesnych wersjach Instagrama

Przez lata, po włączeniu do platform Meta⁴ (firma stworzona przez właściciela Facebooka – Marka Zuckerberga) Instagram drastycznie się zmienił i wprowadził wiele nowych funkcjonalności. Ostatecznie wprowadzono udostępnianie zdjęć w innych proporcjach niż 1 na 1, wsparto umieszczanie filmów czy dodano opcję wysyłania wiadomości prywatnych do innych użytkowników. Jedną z przełomowych elementów dodanych do aplikacji była funkcja „Stories” (z ang. Opowieści, historie), która umożliwiła dodawanie zdjęć oraz filmów, które znikają po 24 godzinach. Użytkownicy bardzo polubili tę formę aktywności w mediach społecznościowych. „Stories” stało się jednym z głównych narzędzi do komunikowania ostatnich wydarzeń z życia oraz obrało formę swoistego forum, w którym twórcy z dużą liczbą obserwujących mogli w bezpośredni sposób zwrócić się do swoich sympatyków. Warto zwrócić uwagę, że sam pomysł na dodanie „Stories” nie wziął się od twórców Instagrama, lecz został skopiowany od inżynierów odpowiedzialnych za powstawanie innej aplikacji – Snapchat⁵. To ta platforma jako pierwsza wprowadziła koncepcja znikających wiadomości, w tym tych publikowanych dla wszystkich i znikających po jednej dobie. Poniżej widać przykładowe „Stories”.



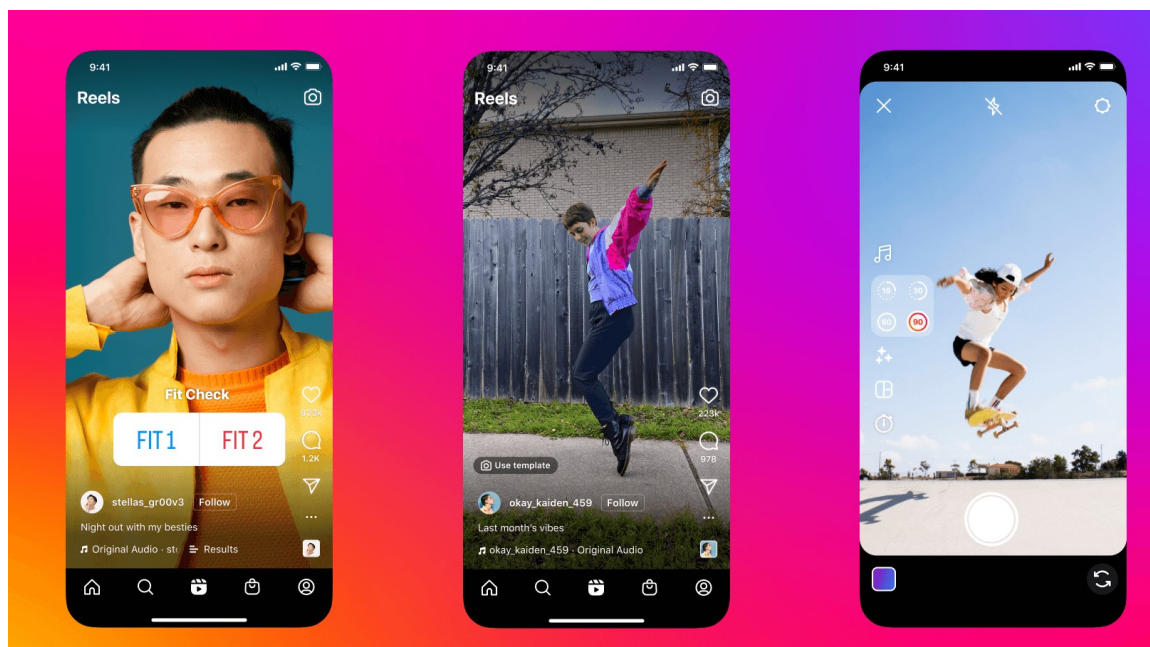
Graf. 4.2: przykład interfejsu Instagram „Stories”

Najnowszym usprawnieniem portalu jest funkcja „Reels” (z ang. Rolki). Są to krótkie, zazwyczaj kilkunastosekundowe filmy (ale istnieje możliwość dodania dłuższego, ponadminutowego nagrania) w orientacji pionowej publikowane przez użytkowników. Ponownie, nie jest to pomysł wywodzący się z Instagrama, lecz z innej platformy, która w ostatnich latach zyskała ogromną popularność – Tik Toka. „Reels” zawierają wszelkiego rodzaju treści, które wykraczają formą poza zwykłe zdjęcia. Społeczność wykorzystuje je do przedstawienia swojego typowego dnia, przepisów kulinarnych czy poradników. Kreatywność użytkowników nie zna granic w tym przekazie w mediach społecznościowych, co powoduje ogromne zainteresowanie i generuje niesamowite zasięgi. Nawet wzrastający twórcy potrafią uzyskać milionowe wyświetlenia na swoich

⁴ Konglomerat technologiczny, który w swoim portfolio posiada m.in. Facebooka czy Instagram, źródło: https://pl.wikipedia.org/wiki/Meta_Platforms

⁵ Aplikacja społecznościowa, która charakteryzuje się znikającymi wiadomościami, źródło: <https://pl.wikipedia.org/wiki/Snapchat>

miniprodukcjach. Stanowi to konkurencję dla takich platform jak YouTube⁶, czyli serwisu wideo, w którym autorzy podobnie umieszczają swoje filmy, jednak w dłuższym formacie. Obie platformy są idealne dla twórców internetowych, nie mają jednak takiego pokrycia w zawiązywaniu znajomości, choć umożliwiają dostrzec utalentowane osoby, które pokazują swoje zdolności w sieci. Fragment interfejsu „Reels” zaprezentowano na grafice.



Graf. 4.3: przykład interfejsu Instagram „Reels”

4.1.1. Podsumowanie rozwiązań aplikacji Instagram

- **Oferowane możliwości:**
 - Instagram przez lata rozbudował swoją ofertę. Użytkownicy mają wiele opcji, aby podzielić się swoimi przeżyciami, historiami czy wspomnieniami. Różnorodność form wpływa na atrakcyjność aplikacji, użytkownicy o różnych charakterystykach odnajdą swoją preferowaną metodę publikowania treści na tej platformie.
- **Spełnione założenia dotyczące autorskiej aplikacji:**
 - Platforma umożliwia dodawanie różnych aktywności, w tym zdjęć, odnośników do piosenek i filmów czy notatek i wpisów. Odbywa się to poprzez „Stories”, ale tego typu elementy można dodać nawet do zdjęć umieszczanych na profilu użytkownika.
 - Można zaobserwować profile znajomych i prowadzić z nimi konwersacje. Nie jest to jednak ograniczone tylko dla obserwujących siebie nawzajem osób. Prywatną wiadomość może rozesłać każdy do dowolnego innego użytkownika, chyba że ten zablokuje tę opcję u siebie.

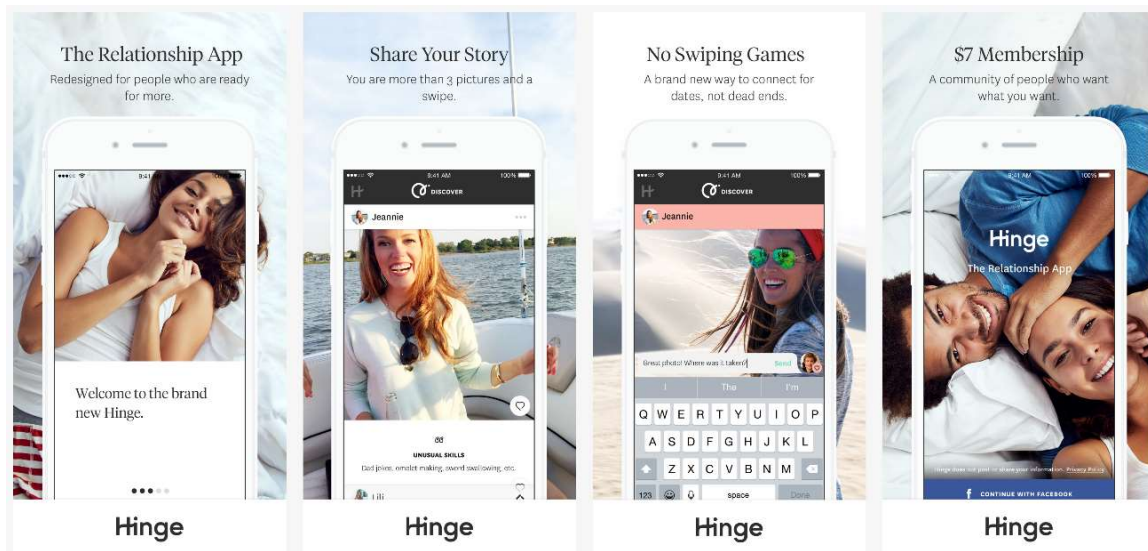
⁶ Serwis do publikowania filmów, źródło: <https://pl.wikipedia.org/wiki/YouTube>

- **Potencjalne zagrożenia i zaniedbane potrzeby**

- Różnorodność i bogactwo treści na tej platformie sprawiło, że użytkownicy spędzają coraz więcej czasu w tej aplikacji. Rozrosło się to już do tego stopnia, że firma Meta, będąca właścicielem Instagrama, otrzymała pozew w kilkudziesięciu stanach USA o negatywne skutki działania ich aplikacji [9]. Prokuratorzy zarzucają firmie przyczynianie się do kryzysu zdrowia psychicznego młodzieży poprzez uzależniający charakter ich platform społecznościowych. Według oskarżycieli firma wielokrotnie wprowadzała użytkowników w błąd co do istotnych zagrożeń związanych z jej platformami, celowo zachęcając dzieci i nastolatków do uzależniającego i kompulsywnego korzystania z mediów społecznościowych [7][9]. Zarzuty związane z wyciągnięciem jak największego zysku kosztem zdrowia użytkowników powoduje, że mimo wielu udogodnień i ciekawych funkcji, jakość aplikacji wyjściowo staje się nie do przyjęcia. W świetle tych faktów nie można uznać Instagrama za odpowiednie miejsce dla cyfrowego dobrostanu i przyjaznego środowiska do poznawania nowych osób.

4.2. Hinge⁷

Hinge to aplikacja randkowa, która umożliwia ludziom zawiązywanie relacji i poznawanie potencjalnych partnerów. Jest znana ze swojego podejścia do połączeń opartego na wspólnych zainteresowaniach i interakcjach. Hinge stara się odróżnić od innych portali, takich jak Tinder⁸, poprzez promowanie prowadzenia dłuższych, bardziej zaangażowanych rozmów co może w rezultacie zaowocować głębszymi relacjami. Autorzy promują swój produkt, mówiąc, że jest to „aplikacja przeznaczona do tego, aby ją usunąć”. Poniżej widać materiały reklamowe tej aplikacji.



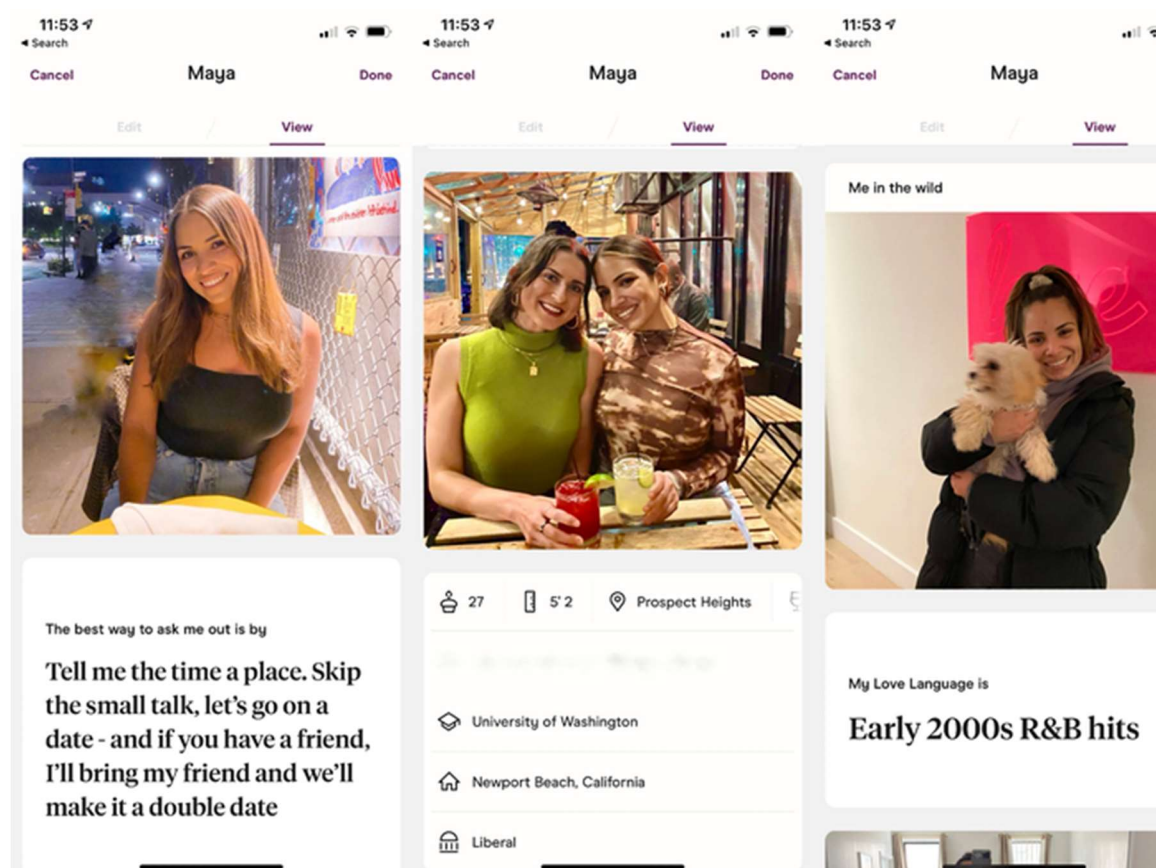
Graf. 4.4: materiał reklamowy Hinge, prezentujący jego funkcje i zalety

Jej działanie polega na tym, aby użytkownicy zareagowali na konkretny element profilu osoby (np. zdjęcie, notatkę głosową czy krótki wpis), co mogą zrobić, odnosząc się do tej

⁷ Aplikacja randkowa, źródło: [https://en.wikipedia.org/wiki/Hinge_\(app\)](https://en.wikipedia.org/wiki/Hinge_(app))

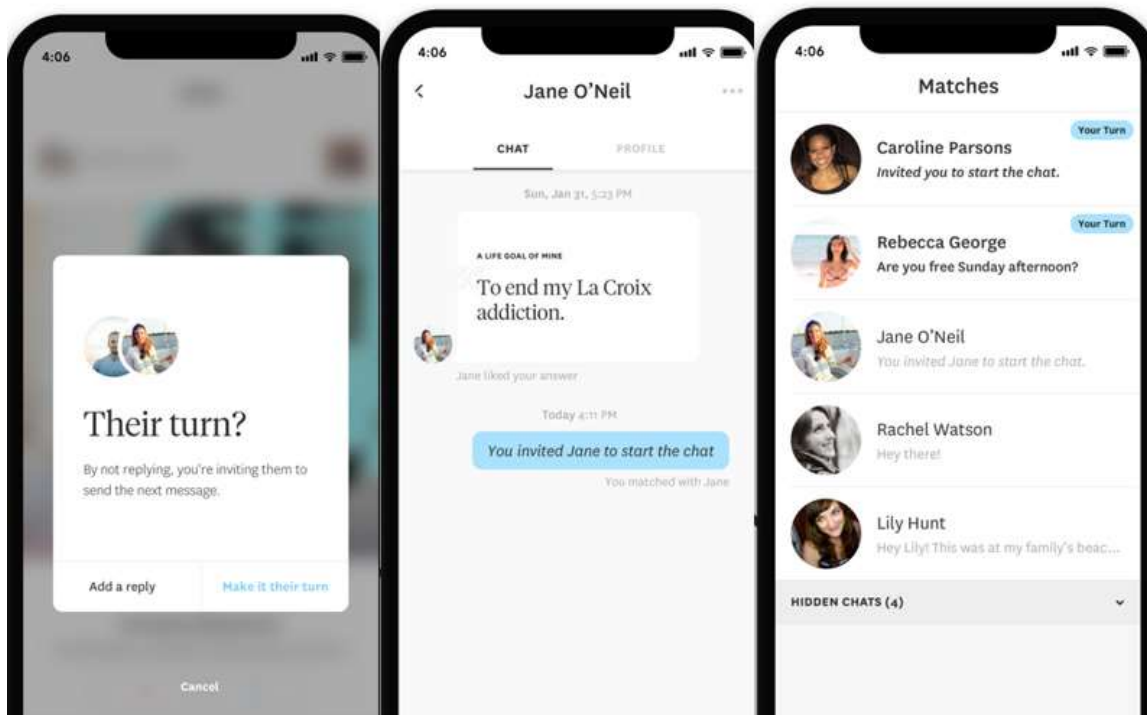
⁸ Aplikacja randkowa, źródło: [https://en.wikipedia.org/wiki/Tinder_\(app\)](https://en.wikipedia.org/wiki/Tinder_(app))

aktywności, klikając na nią i rozpoczynając rozmowę. Oznacza to, że użytkownicy muszą zwrócić uwagę na szczegóły i zastanowić się, co ich przyciąga do danej osoby, zanim podejmą konkretne kroki. Kolejną funkcją jest wybór trzech pytań, na które trzeba odpowiedzieć i które pojawią się na profilu jako ciekawostki o użytkowniku. Jest to kolejna aktywność cechująca osobę w aplikacji, na którą można odpowiedzieć, wysyłając wiadomość. Pomaga to lepiej poznać siebie nawzajem i znaleźć osoby, które mają podobne zainteresowania i wartości. W ten sposób Hinge stara się pomóc użytkownikom znaleźć udane związki, a nie tylko krótkotrwałe i nietrwałe relacje. Poniższa grafika przedstawia fragmenty profili w aplikacji. Widać, że użytkownicy publikują wpisy, informacje o sobie, a także zdjęcia.



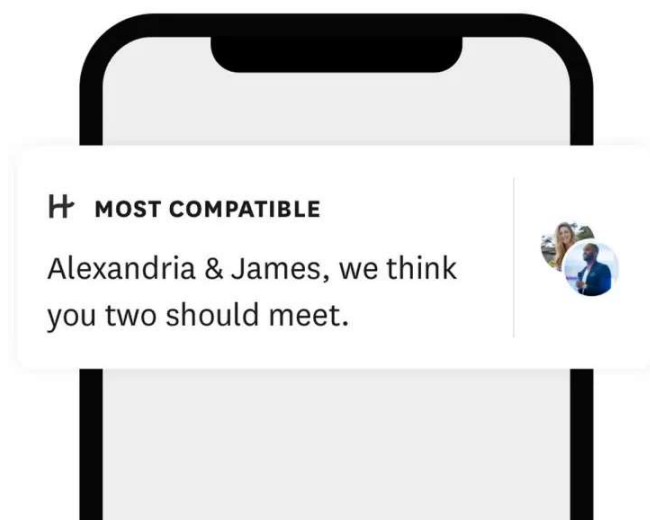
Graf. 4.5: przykładowy interfejs profilu w aplikacji Hinge

Aby uniknąć *ghostingu* (z ang. ignorowania) poprzez użytkowników, co objawia się opuszczeniem i zakończeniem konwersacji przez jedną stronę, czasami celowo, z powodu niskiej jakości rozmowy. Jednakże często użytkownicy zapominają o rozpoczętych rozmowach, głównie przez to, że zapomnieli lub konwersacja zagubiła się wśród innych. Naprzeciw temu problemowi wyszła funkcja „*Your Turn*” (z ang. Twoja Kolej), która polega na powiadomieniu użytkownika, że teraz jest jego kolej, aby wysłać wiadomość. Takie oznaczenie może być wysyłane niezależnie od tego, kto napisał ostatni. Pomaga to wykazać zaangażowanie w rozmowę drugiej strony i chęć do jej kontynuowania. Przykładowy widok w aplikacji z opcją „*Your Turn*” zaprezentowano na grafice.



Graf. 4.6: przykładowy interfejs z funkcją „Your Turn”

Kolejną unikatową cechą Hinge'a jest nowatorski mechanizm rekomendowania użytkowników nazwany „*Most Compatible*” (z ang. Najbardziej Dopasowani), który jest oparty na algorytmie stabilnych połączeń Gale-Shapleya [10]. Polega on na iteracyjnym dobieraniu najlepszego dopasowania dla pary użytkowników. Przy założeniu, że w zbiorze użytkowników jest po równo osób (ich parzysta liczba), algorytm działa tak długo, aż nie zostaną połączeni wszyscy w pary, zapewniając przy tym, że połączenia są stabilne. Stabilność połączenia polega na tym, że algorytm może początkowo dobrać parę użytkowników jako najlepsze dopasowanie, ale w następnej iteracji stwierdzi, że jednak jeden z nich pasuje bardziej do kogoś innego. Odrzuca wówczas tę parę, dobiera lepsze połączenie dla jednego użytkownika, a pozostały bez pary ma dobraną nową, najlepszą dla tej iteracji. Taka wymiana trwa tak długo, jak algorytm nie stwierdzi, że już nie trzeba zmieniać par [10]. Hinge tworzy codziennie takie pary i pokazuje to użytkownikom w aplikacji (co ważne, każda osoba z pary widzi, że zostało znalezione najlepsze dopasowanie dla niej). Wprowadza to kolejne, spore usprawnienie w odniesieniu do jak najbardziej skutecznego i efektywnego łączenia ludzi w Internecie. Sam algorytm uwzględnia o wiele więcej niż użytkownik by mógł, korzystając z aplikacji, gdzie czasami, aby przejrzeć profile proponowanych osób, trzeba poświęcić wiele czasu. Przykład powiadomienia „*Most Compatible*” przedstawiono poniżej.



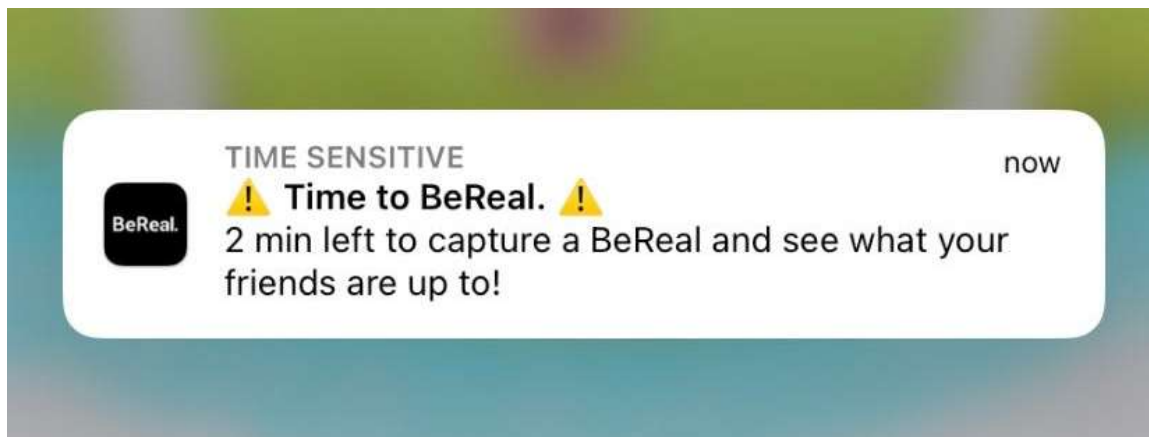
Graf. 4.7: komunikat o Najlepszym Dopasowaniu aplikacji Hinge

4.2.1. Podsumowanie rozwiązań aplikacji Hinge

- **Oferowane możliwości:**
 - Aplikacja zawiera wiele nowatorskich funkcji, które czynią ją niezwykle unikalną na tle konkurencji. Wprowadzono wiele mechanizmów, które angażują użytkowników, pomagają prowadzić rozmowy i dobierają im najlepsze połączenia. Różnorodne aktywności pomagają w bardziej kreatywnym przebiegu poznawania się z innymi. Hinge w sporym stopniu przyczynia się do jakości interakcji ludzi w Internecie, jego celem jest pozostawienie użytkowników z głęboką relacją.
- **Spełnione założenia dotyczące autorskiej aplikacji:**
 - Aplikacja zapewnia łączenie użytkowników w pary i tworzenie przez to połączeń
 - Możliwa jest komunikacja między użytkownikami przez *chat* (z ang. komunikator tekstowy) w celu poszerzania znajomości
 - Platforma umożliwia dodawanie różnych aktywności, w tym zdjęcia, nagrania głosowe, wpisy, odpowiedzi na pytania w formie ciekawostek
- **Elementy, które można zaczerpnąć, tworząc własną aplikację:**
 - Zaimplementowanie algorytmu stabilnych połączeń Gale-Shapleya [10]
 - Wykorzystanie koncepcji „Najlepszego Dopasowania” w celu określenia najlepszej pary
 - Wprowadzenie funkcjonalności podobnej w działaniu do „Twoja Kolej”
- **Potencjalne zagrożenia i zaniedbane potrzeby**
 - Hinge kładzie mocny nacisk na aktywność i zaangażowanie użytkowników. O ile są to wartościowe połączenia oraz ciekawe i inspirujące rozmowy, aplikacja może mieć wpływ na to, ile czasu spędzamy przed telefonem. Zawsze wyświetlane komunikaty bądź skupiające uwagę użytkownika elementy interfejsu są w stanie wprowadzić zachowanie bliskie uzależnieniu. O ile niektóre z funkcji Hinge’a są bardzo dobrze przemyślane, to trzeba mieć na uwadze, jakie skutki to za sobą ciągnie w odniesieniu do cyfrowego dobrostanu.

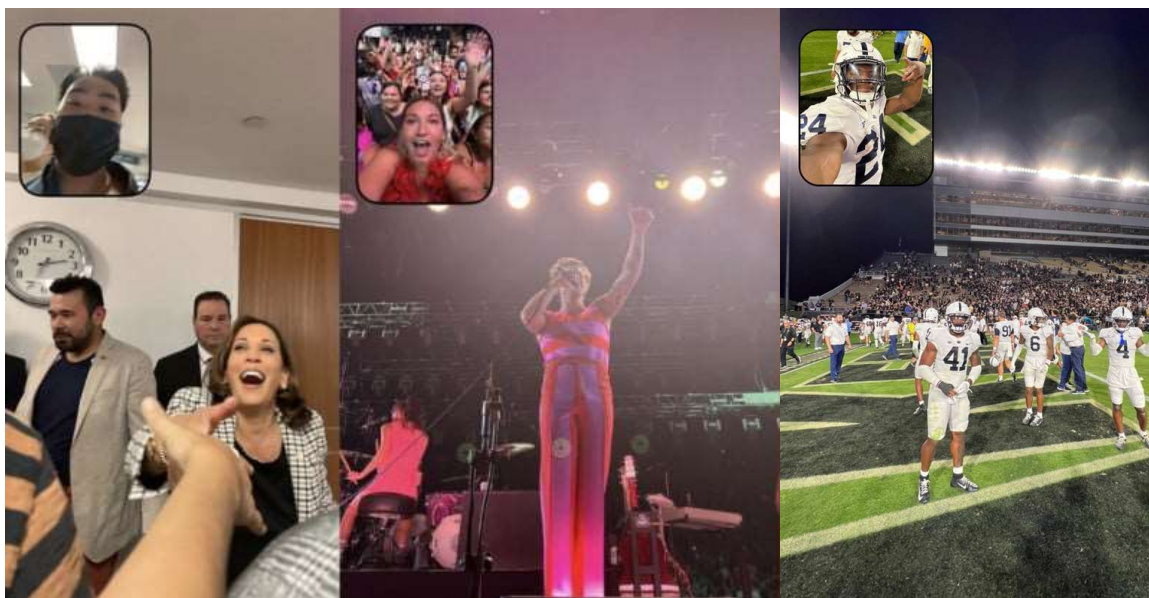
4.3. BeReal

BeReal to aplikacja wydana w 2020 roku, a swoją popularność zyskała na początku 2022. Platforma stawia na autentyczność przekazu i zachęca użytkowników do publikowania zdjęć bez użycia filtrów pokazujących codzienne, zwyczajne chwile. Możliwość na publikowanie treści jest każdego dnia o innej, wylosowanej przez aplikację porze. Każdy użytkownik dostaje raz dziennie powiadomienie „*Time to BeReal.*” (z ang. „Czas być prawdziwym”). Okno powiadomienia poniżej:



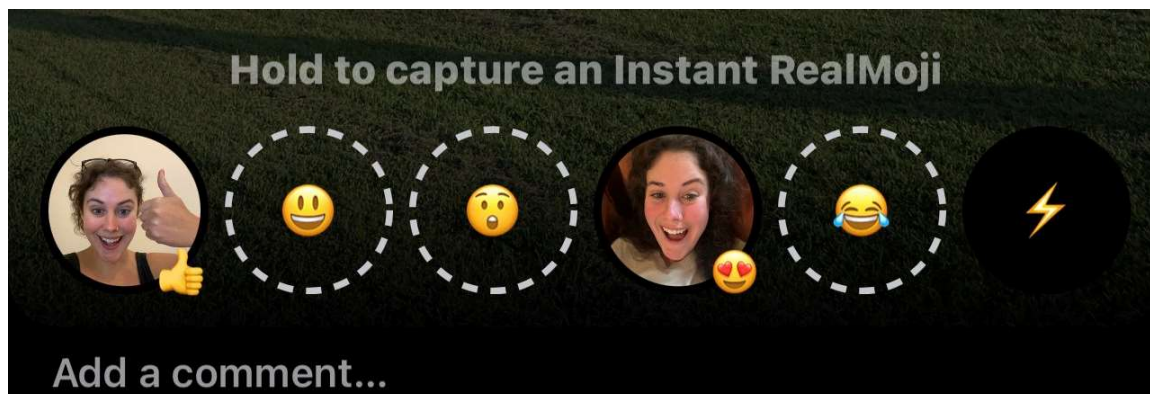
Graf. 4.8: powiadomienie BeReal

W ciągu 2 minut od otrzymania powiadomienia *push* (z ang. powiadomienie pojawiające się na ekranie smartfona) należy zrobić zdjęcie i je udostępnić. Zdjęcia znikają następnego dnia, ponownie o przypadkowej godzinie, a wtedy użytkownik może ponownie opublikować zdjęcia. Aplikacja wykorzystuje przedni, jak i tylny aparat, co pozwala rejestrować wizerunek użytkownika oraz jego otoczenie. Przykładowe wpisy w serwisie wyglądają tak jak na grafice.



Graf. 4.9: przykładowe publikacje zdjęć w aplikacji BeReal

Publikacje innych osób można zobaczyć dopiero po wysłaniu własnego zdjęcia. Oprócz tego w aplikacji można stworzyć swoje „RealMoji”. Są to zdjęcia *selfie* (wykonywane z przedniego aparatu smartfona) nakreślające jaką reakcję wyraził dany użytkownik, widząc zdjęcia innych. Oprócz tego typu reakcji można również zostawiać własne komentarze pod wpisami innych. Przykładowe reakcje w aplikacji przedstawiono na grafice.



Graf. 4.10: reakcje „RealMoji”

Jeśli zdarzy się, że powiadomienie zostanie pominięte, istnieje szansa na opublikowanie zdjęcia z opóźnieniem. Warto jednak pamiętać, że nie można zobaczyć zdjęć innych osób do czasu, kiedy nie opublikuje się swojego wpisu.

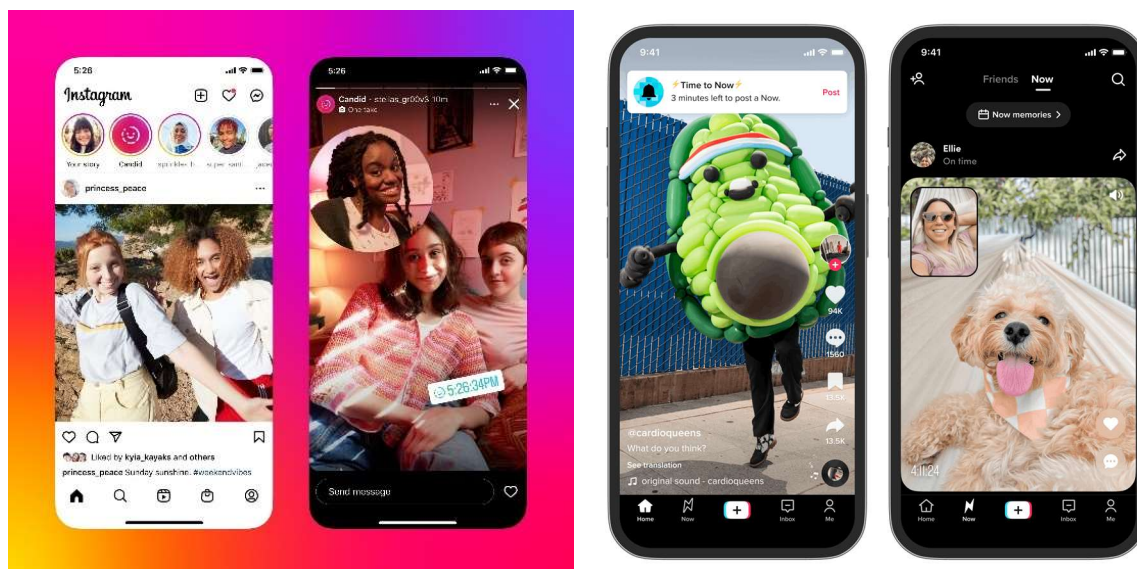
Produkt został opracowany przez dwóch francuskich inżynierów – Alexisa Barreyata (wcześniej pracownika działu produkcji mediów w GoPro) oraz Kévina Perreau, którzy rozpoczęli pracę nad projektem pod koniec 2019 roku. Pojawił się w App Store (sklep z aplikacjami na system iOS) na początku 2020 roku. Minął rok, aż aplikacja została spopularyzowana na francuskich kampusach przez studentów. Na rynek amerykański i brytyjski produkt wszedł dopiero na początku 2022 roku, kiedy to użytkownicy innej platformy – Tik Toka – publikowali materiały o BeRealu na swoich profilach [11].

25 lipca 2022 roku, BeReal pojawił się na topie najczęściej pobieranych aplikacji na App Store i został tam aż do września. Liczba jego użytkowników stale wzrasta i obecnie przekracza 20 milionów aktywnych użytkowników dziennie. Od czerwca 2021 roku, jego wartość przez następne 12 miesięcy wzrosła z 30 milionów dolarów do aż 600 milionów [12].

4.3.1. Sposób działania BeReala

Aplikacja BeReal poprzez swój nietypowy mechanizm umieszczania postów (z ang. wpisów) zaskarbiło sobie sympatię głównie wśród młodych użytkowników. Intuicyjny interfejs, znajome elementy z innych popularnych aplikacji (tj. reakcje na wpisy, komentarze, dodawanie i przeglądanie profilu znajomych) sprawiły, że mimo wysokiego progu wejścia na rynek dla produktów tego typu, BeReal okazał się wielkim sukcesem. Warto wspomnieć, że powstał jako niezależny projekt dwóch osób, nie był zasilony wpływami dużych korporacji, ani żadna nie stała za jego produkcją. Podkreśla to, jak wielkie znaczenie ma dobry pomysł na nowe rozwiązanie. Musi być to coś tak nowatorskiego, że społeczność zainteresuje się tym wyłącznie dzięki samej jakości.

Przytaczana innowacja to mechanizm publikowania postów raz dziennie, o tej samej porze dla wszystkich, jedynie 2 minuty na wykonanie operacji. Po tym czasie aplikacja pokazuje, ile użytkownik spóźnił się ze wstawieniem zdjęcia. Prostota tego rozwiązania, ale też jego pomysłowość uczyniło BeReala unikatowym. W okresie popularności aplikacji inne platformy skopiowały funkcjonalność do siebie. Tak przykładowo powstał „Candid Story” od Instagrama czy Tik Tok⁹ „Now”, których interfejsy dołączono poniżej.



Graf. 4.11 oraz **4.12**: interfejsy „Candid Story” oraz Tik Tok „Now”

4.3.2. Podsumowanie aplikacji BeReal

To, co wprowadził BeReal, można wykorzystać do utworzenia zupełnie nowych mediów społecznościowych, na nowych zasadach i o innej, zgoła odmiennej charakterystyce. Zakończyłoby to erę używania tych portali jako masowe media i obszerne agregatory wiadomości. Niosłoby to za sobą ryzyko utraty bloków reklamowych, ponieważ liczyłoby się tylko to, z czym chcą podzielić się nasi znajomi. Oczywiście nie jest to zwiastun końca dla takich aplikacji jak Facebook, Instagram czy inne, jednakże zmienia to optykę na stan *social media* w trzeciej dekadzie XXI wieku.

Kluczowe korzyści, jakie daje nam BeReal to nie tylko przełamanie bariery autentyczności w Internecie czy bliższy kontakt ze znajomymi, to także mniejsza liczba godzin w ciągu dnia spędzona na *scrollowaniu* (z ang. przesuwaniu treści) różnego rodzaju aplikacji. Wpływa to znacząco na kondycję psychiczną człowieka, pozbawia go przywiązania do codziennej dawki dopaminy, jaką dostarczają w przemyślany sposób twórcy tych platform. Tak szerokie i długie skupienie uwagi użytkownika na ekranie telefonu ma swoje konsekwencje zdrowotne [6]. W dalszej części tej pracy przeprowadzono rozważania na ten temat, ponieważ zachowanie stabilności i równowagi człowieka na tle nowych technologii jest obecnie kluczowym wyzwaniem.

Na podstawie analiz oraz refleksji zawartych w powyższych rozdziałach, staje się pewne, że przekucie innowacji w kolejne pożyteczne rozwiązania, które stawiają czoła współczesnym problemom, jest ogromnym zadaniem dla teraźniejszego społeczeństwa i będzie miało swoje skutki w przyszłości.

⁹ Platforma do umieszczania nagrań wideo, źródło: <https://pl.wikipedia.org/wiki/TikTok>

5. Założenia aplikacji

W tym rozdziale zostanie opisana idea aplikacji, która została przygotowana na potrzeby pracy. Przedstawione poniżej założenia wprowadzone są na kanwie rozwiązań zaprezentowanych w poprzednich częściach pracy. Dokonano analizy pod kątem spełnionych potrzeb i problemów, które trzeba zaadresować w nowej aplikacji, a które nie znajdują pokrycia wśród konkurencji. Oprócz tego zrealizowany zostanie ogólny przegląd funkcji, które autorzy chcieliby zobaczyć w aplikacji.

5.1. Streszczenie

Once a Day – aplikacja przeznaczona do łączenia ludzi otwartych na nowe znajomości, o podobnych cechach charakteru i zainteresowaniach. Skupia się na codziennym udostępnianiu swoich aktywności, które tworzą unikalny profil każdej osoby. Umożliwia ograniczoną ilość interakcji w ciągu dnia, aby ograniczyć “cyfrowy” tryb życia.

5.2. Filary aplikacji

1. RAZ DZIENNIE

Korzystanie z aplikacji w celu dodania aktywności odbywa się raz dziennie. Po zatwierdzeniu aktualizacji profilu nie można go zmienić przez 24 godziny. Pozwala to na utrzymanie cyfrowego dobrostanu oraz naturalność swoich wpisów.

2. AKTYWNOŚCI

Wszystko to, co użytkownik może dodać na swój profil to aktywności. Aplikacja ma umożliwiać dodanie kilku różnorodnych aktywności: zdjęć, odnośników do piosenek czy filmików, notatki (głosowe), opinie, odwiedzane punkty na mapie. Wszystko po to, aby jak najlepiej zdefiniować swoją osobę w Internecie i rzetelnie pokazać swój charakter i zainteresowania.

3. POŁĄCZENIA

Osoby, do których zostaliśmy dobrani przez aplikację to połączenia. Dziennie aplikacja oferuje nam kilka połączeń, z którymi możemy nawiązać kontakt. Liczba ta jest jednak ograniczona z uwagi na motywację utworzenia dłuższej i głębszej konwersacji z mniejszą liczbą osób. Aktywnym połączeniom przypisany jest moduł wymiany wiadomości bezpośrednich w aplikacji.

5.3. Spełnione założenia w wersji deweloperskiej

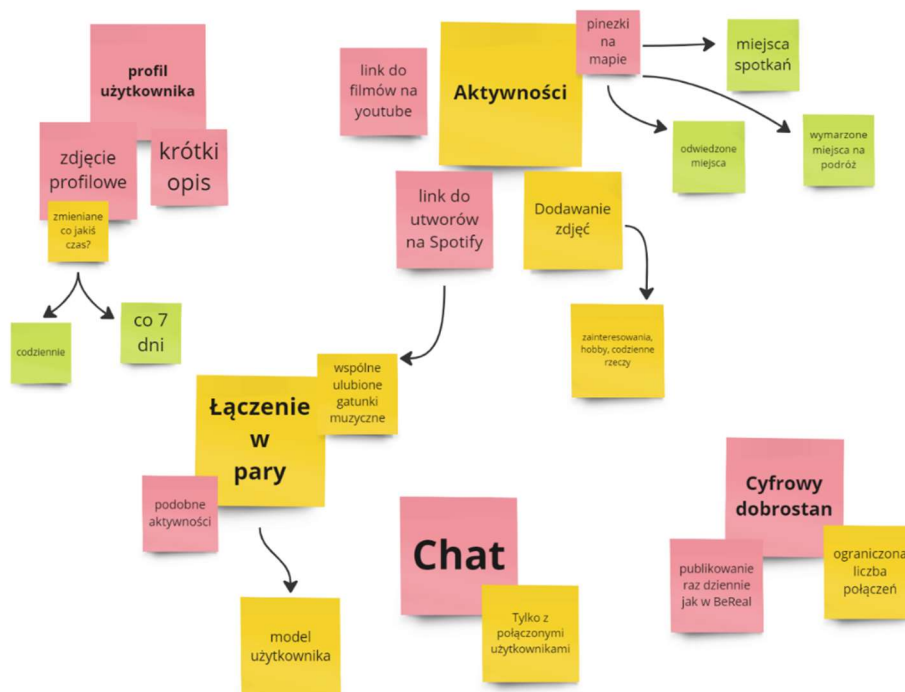
Na zakończenie projektu inżynierskiego, utworzona aplikacja pozostaje w wersji deweloperskiej. Na potrzeby tej pracy zaimplementowano algorytmy łączenia użytkowników na podstawie zdjęć różnych aktywności. Zaprojektowano graficzny interfejs użytkownika, w którym wyświetlane są profile i fotografie użytkowników.

6. Planowanie i projekt aplikacji

W obliczu istniejących rozwiązań i wykrytych potrzeb, z jakimi zmagają się media społecznościowe, zaczęto planowanie własnego rozwiązania. W tym celu przeprowadzono „burzę mózgów”, podczas której zadano kluczowe pytania dotyczące konstrukcji aplikacji i jej założeń. W wyniku tego procesu, autorzy pracy proponowali swoje pomysły, funkcjonalności i ewentualne innowacje. Następnie dobrano konkretne technologie wspomagające wytwarzanie aplikacji, dobrano te, które najlepiej sprawdzą się do takiego projektu, ułatwią spełnić normy i dobre praktyki programowania oraz będą szeroko wspierane przez społeczność. Po zakończeniu wstępnego planowania wiele założeń i wymagań ulegało zmianie w trakcie wytwarzania aplikacji. Brano to pod uwagę, ponieważ był to pierwszy projekt tego typu podjęty przez autorów, dodatkowo jego charakter pozostawał eksperymentalny. Nie przeszkodziło to jednak w realizowaniu założeń i rozwoju pracy. Wiele jej elementów wyklarowało się dopiero po wielu próbach i koncepcjach wprowadzenia konkretnej funkcjonalności. Powstawało wiele idei w odniesieniu do algorytmu łączenia, będącego trzonem tego projektu, ale mimo tego zdecydowano się podjąć kroki, które najpierw go urzeczywistnią, a w dalszej kolejności uczynią go bardziej wydajnego. Autorzy kierowali się bowiem pewną maksymą, wyciągniętą z publikacji matematyka i informatyka Donalda Knutha o tytule *Structured Programming with go to Statements* [13]: „Przedwczesna optymalizacja jest źródłem wszelkiego zła”.

6.1. Planowanie funkcjonalności

W ramach planowania zebrano wszystkie pomysły i umieszczono je na wirtualnej tablicy:



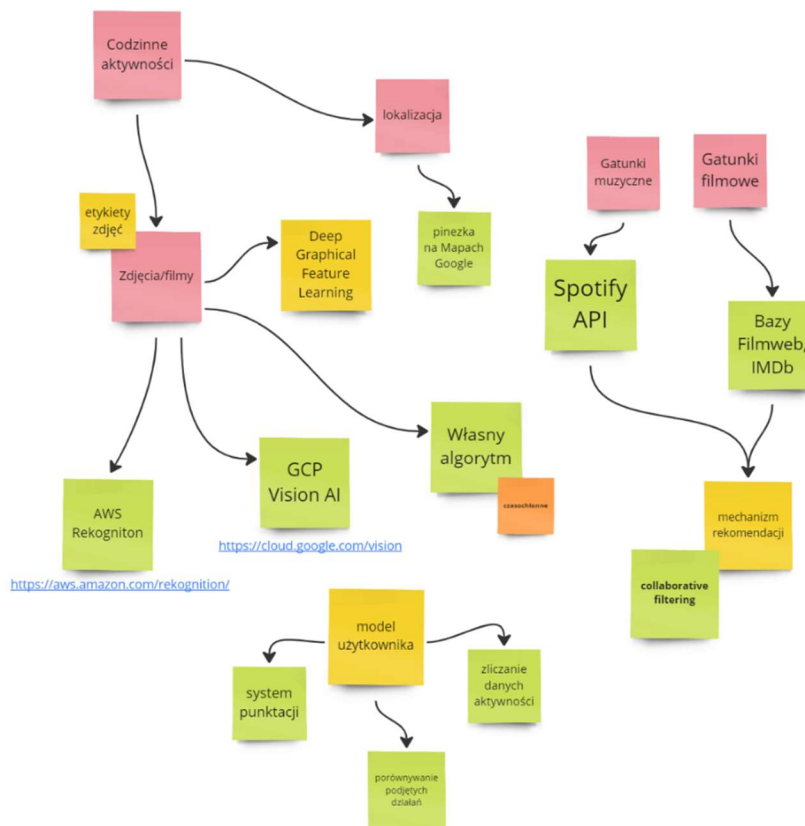
Graf. 6.1: zrzut ekranu z tablicy Miro prezentujący mapę myśli pomysłów na funkcjonalności

Zebrane myśli pokazały jakie funkcje będzie zawierać aplikacja, pozwoliły na ustanowienie struktury projektu i skupienie uwagi na rozwijaniu głównych jego elementów. Najwięcej pracy projektowej zespół spędził na dobraniu (lub stworzeniu) mechanizmów łączenia ludzi. Jak można

zauważyć, padło wiele propozycji co do wybrania składowych modelu użytkownika, na podstawie którego byłby łączony z innymi. Składowe te można rozumieć jako szeroko pojęte aktywności przechowywane na profilu, a następnie, raz dziennie algorytm na ich podstawie tworzyłby dopasowane pary użytkowników. Typowy przebieg takiego działania mógłby wyglądać następująco: „Użytkownik naszej aplikacji zabiera się za czytanie nowej powieści. Jest to jego zdaniem jedna z lepszych książek, jaka wpadła mu w ręce. Uruchamia aplikację, w której dodaje zdjęcie okładki ulubionej powieści. Następnie, tego samego dnia, inny użytkownik pracuje nad swoimi wierszami. Piszac kolejne fragmenty, postanawia sfotografować obecny stan prac. Podobnie jak wcześniej przedstawiony czytelnik, wstawia wykonane zdjęcie na swój profil. Tego dnia, aplikacja wyodrębni etykiety z tych zdjęć, odkryje, że na obu widnieją kartki, strony, tekst pisany i doda te wartości do modelu użytkownika. Następnie, zestawiając otrzymane wyniki, silnik oblicza „podobieństwo” tych dwóch modeli. Przy założeniu, że etykiety rzeczywiście były tożsame, użytkownicy zostają połączeni i widzą to na swoich profilach. Mogą wówczas się ze sobą skomunikować i toczyć rozważania o literaturze.”

Była to koncepcja, zaprezentowana na planowaniu projektu przez jednego z autorów, która rozpoczęła kolejną dyskusję. Mianowicie, w następnym etapie projektowania aplikacji, rozpoczęto rozważania dotyczące algorytmów łączenia użytkowników. Proces planowania tego elementu został przedstawiony na zrzucie ekranu.

6.2. Propozycje mechanizmów łączenia użytkowników



Graf. 6.2: zrzut ekranu z tablicy Miro prezentujący mapę myśli pomysłów na mechanizmy łączenia

Zapisano różnorodne aktywności i zanotowano, jaką techniką można je zebrać, aby przyczyniły się na rzecz mechanizmu łączenia. Wprowadzono również pojęcie modelu użytkownika, który stanowi encję, dzięki której można dokonywać porównywania z innymi użytkownikami.

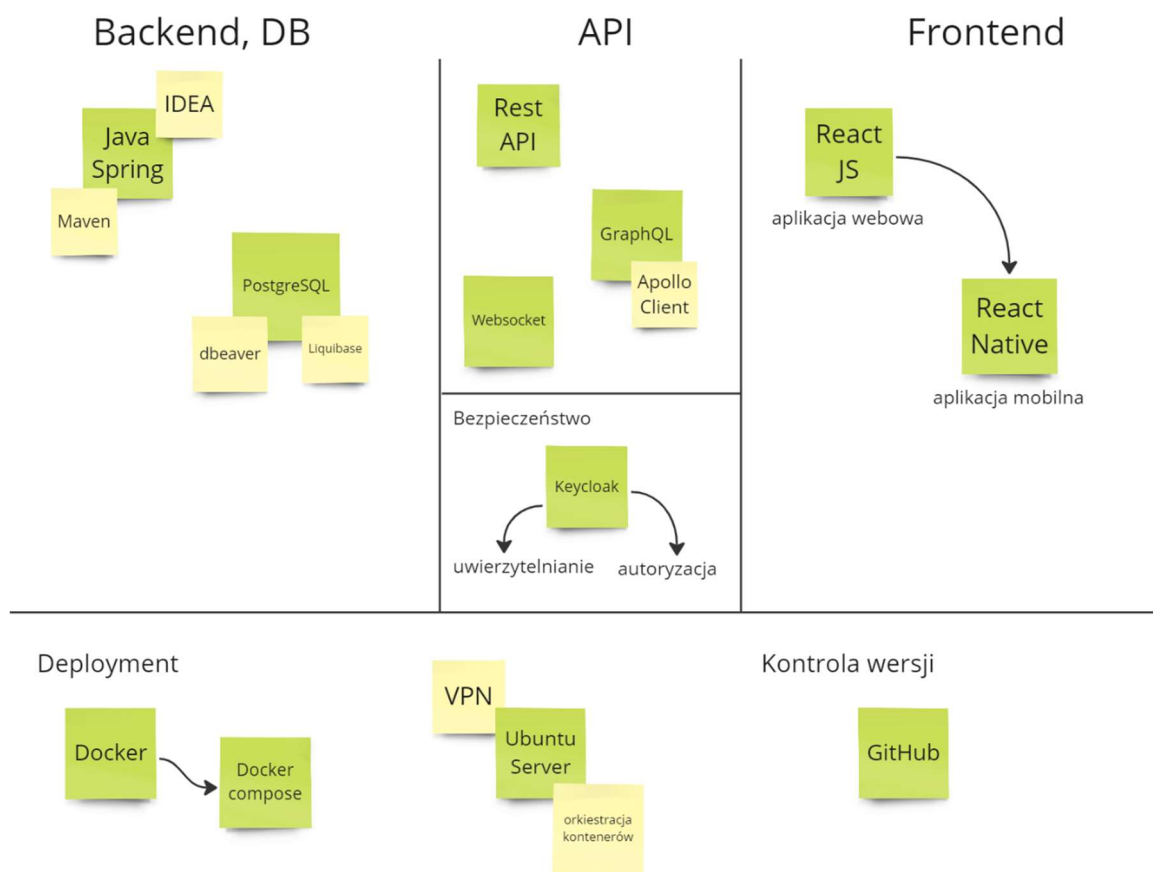
Najważniejsze wnioski:

- Etykietowanie zdjęć umieszczanych przez użytkowników pomoże zebranie statystyk o ich dziennych aktywnościach i czynnościach.
- Istnieją narzędzia, za pomocą których można etykietować zdjęcia, nie trzeba tej funkcjonalności implementować od zera.
- W celu porównywania użytkowników można skorzystać z idei filtrowania kolaboracyjnego, które dostarcza sposobów na określenie podobieństwa między docelowymi aktywnościami.
- Należy określić, w jaki sposób będzie budowany model użytkownika, co i w jakim stopniu będzie wpływało na jego charakterystykę.

7. Wybór technologii i proces wytwarzania

Mając wiedzę o koncepcji projektu, rozpoczęto proces wybierania technologii, w której aplikacja będzie wytwarzana. W tym rozdziale przedstawiono także sposoby pracy w zespole oraz jak przechowywano kod źródłowy.

7.1. Wybór technologii (Kamil Czepiel)



Graf. 7.1: zrzut ekranu z tablicy Miro prezentujący mapę myśli wyboru technologii

Planując konkretne rozwiązania technologiczne w projekcie, kierowano się znajomością dostępnych rozwiązań, ich innowacyjności i prostoty w obsłudze (i ewentualnie w nauce w obliczu technologii, z którą autorzy nie mieli doświadczenia). Odnośniki do wykorzystywanych dokumentacji zostaną umieszczone w dalszych częściach pracy.

Najważniejsze wnioski:

- Jako technologii do logiki aplikacji wybrano Java Spring oraz bazę danych PostgreSQL, z którymi autorzy mają doświadczenie. Dodatkowo Spring umożliwia tworzenie API, który w prosty sposób można wykorzystać w tworzeniu aplikacji. Pozwoli to na utworzenie interfejsu z logiką aplikacji, a jej interfejsem graficznym. Spring oferuje wiele udogodnień w kierunku łączenia z bazą danych (dzięki JPA) oraz zapewnia mechanizmy bezpieczeństwa.
- Do stworzenia aplikacji od strony interfejsu graficznego wybrano bibliotekę React JS. Za jej pomocą można wytworzyć aplikację przeglądarkową, a korzystając z wersji Native, można odtworzyć napisany kod dla aplikacji mobilnej.

- Przeanalizowano oferty rozwiązań chmurowych do wdrażania aplikacji, między innymi usługi AWS czy Google Cloud. Jednakże postawiono na skonfigurowanie i administrowanie lokalnego serwera, co wiąże się z mniejszymi kosztami i większymi możliwościami pod względem zarządzania.
- Do połączeń ze zdalnym serwerem skorzystano z technologii OpenVPN, która zestawia tunel z urządzeń końcowych deweloperów prosto na serwer. W ten sposób można łatwo pracować asynchronicznie, mając dostęp do tego samego środowiska deweloperskiego.

7.2. Wytwarzanie aplikacji (Jakub Grzybowski)

Autorzy podzielili się zadaniami w pracy nad projektem. Podział wyniknął z doświadczenia autorów w danych technologiach oraz ich wizji konkretnych części aplikacji.

Kamil Czepiel (KTI) odpowiada za poniższe części projektu:

- Logika biznesowa aplikacji (backend) – Java Spring
- Architektura – mikroserwisy w kontenerach Docker (Eureka, Kafka, Keycloak)
- Bazy danych – PostgreSQL, Liquibase
- AI – metody oraz modele silnika łączącego użytkowników
- Usługi chmurowe – AWS (S3, Rekognition, EC2)
- CI/CD github actions, zarządzanie repozytorium Docker Hub

Jakub Grzybowski (KTI) odpowiada za poniższe części projektu:

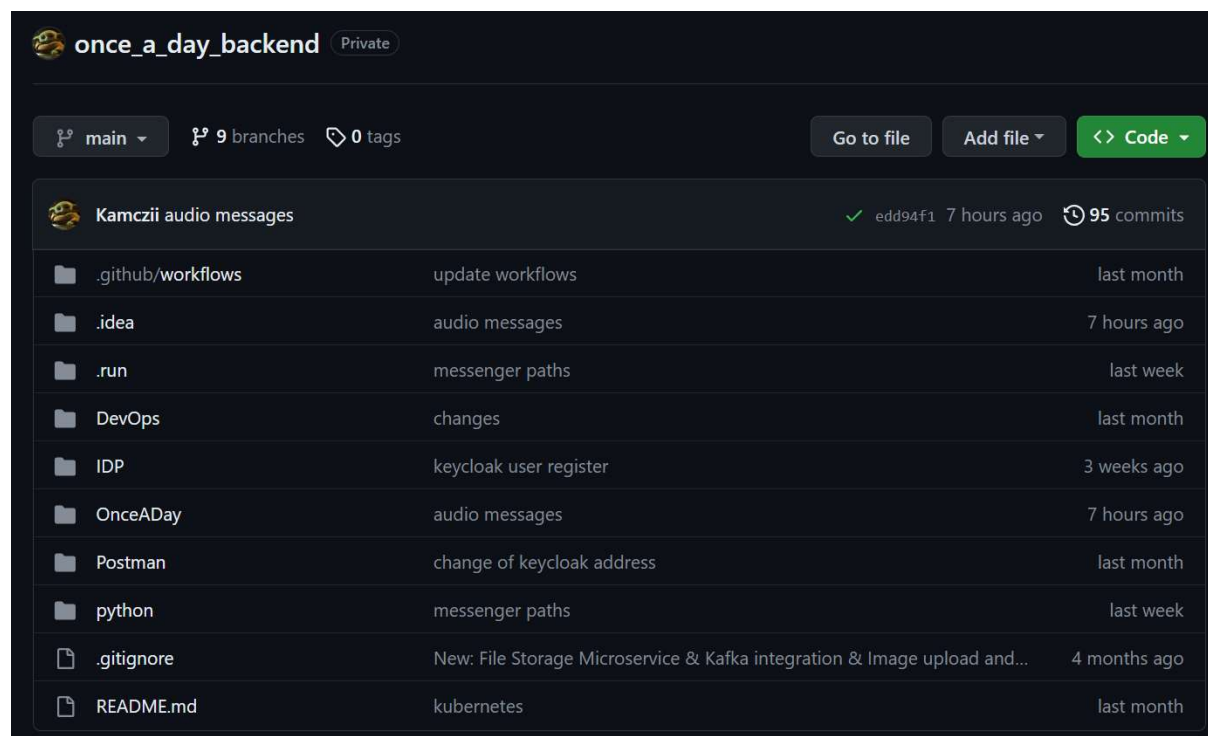
- AI – metody oraz modele silnika łączącego użytkowników
- Testowanie – Postman, Python
- Aplikacja przeglądarkowa – React
- Wsparcie przy pozostałych częściach aplikacji
- Zgłębianie tematów związanych z motywacją pracy
- Opracowanie części opisowej projektu

7.2.1. Repozytoria Git

W celu ułatwienia pracy nad projektem utworzono repozytoria na platformie GitHub celem kontroli wersji, jak i dokumentacji postępu. W rezultacie utworzono trzy repozytoria: oad_backend, oad_frontend, oad_automation. Każdy członek z zespołu inżynierskiego miał dostęp do każdego z nich i mógł na bieżąco sprawdzać aktualny stan prac oraz implementować swoje elementy. Ostatecznie, Kamil Czepiel wziął odpowiedzialność za backend, a Jakub Grzybowski za frontend. Zespół współpracował w formie, którą roboczo można nazwać „czerwone-zielone”. Kiedy jedna osoba wykonała swoją część pracy, która umożliwiała rozpoczęcie pracy drugiej, powiadamiała ją o tym, a następnie po zakończeniu prac wspólnie oceniano tak przygotowany przyrost. Metoda ta sprawdzała się dla dwuosobowego składu zespołu, ponieważ nie było tu zastosowania na wprowadzanie żadnych metodologii realizacji projektu informatycznego. Niemniej jednak zauważono pewne potrzeby i przeszkody, które wstrzymywały wzrost prac na pewnych jego etapach. Między innymi, blokujące defekty na pograniczu części logiki i interfejsu graficznego powodowały opóźnienia. Były to często problemy z dostępnością i spójnością odpowiednich

interfejsów. Zaburzało to koncepcję „czerwone-zielone”, kiedy jedna osoba z zespołu była już gotowa do pracy nad swoim komponentem, ale wyniknęły problemy z częścią przyrostu drugiej.

Struktura repozytorium oad_backend



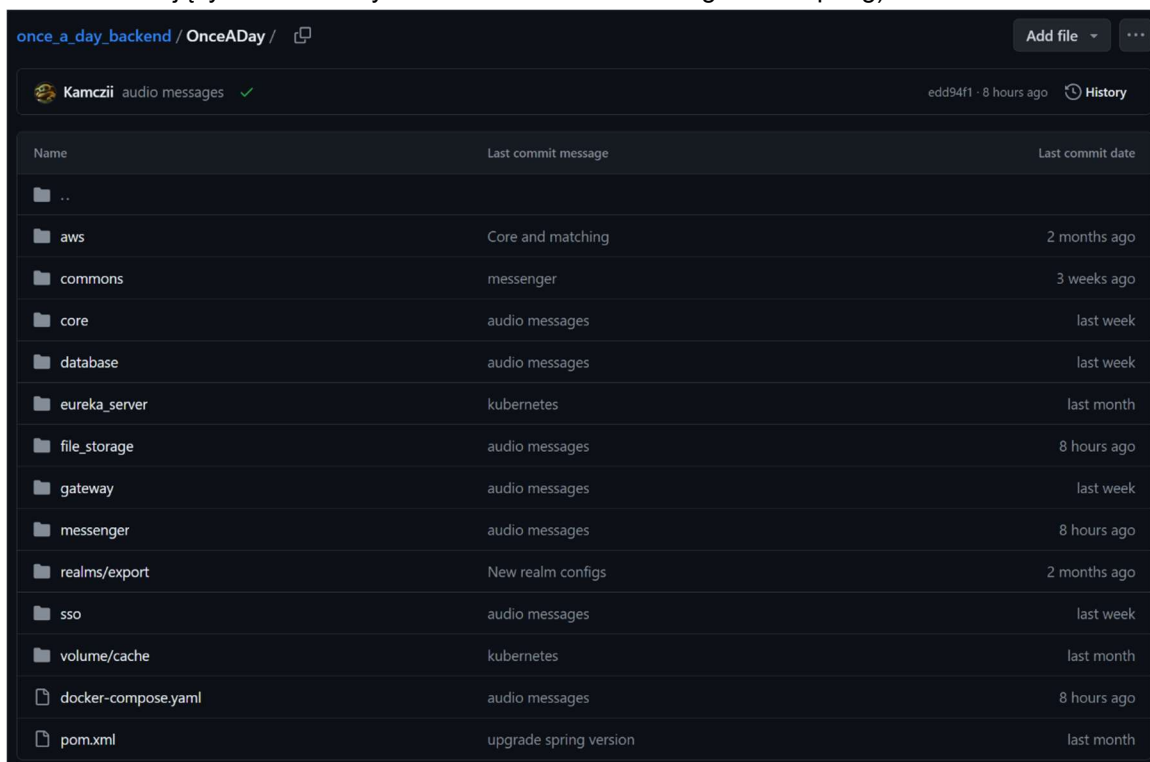
Graf. 7.2: struktura folderów w repozytorium oad_backend z logiką aplikacji

W widocznym na grafice repozytorium przechowywany jest kod źródłowy części logicznej aplikacji, na co składa się:

- Serwis profilu użytkownika
- Serwis do odczytu i zapisu plików (np. zdjęć z profilu użytkownika, nagrań audio)
- Serwis do przyjmowania aktywności użytkowników
- Silnik do łączenia użytkowników
- Inne komponenty wspierające testowanie aplikacji (Postman) oraz zarządzające wdrażaniem i uruchamianiem (DevOps)

Dodatkowo, wydelegowano część logiki aplikacji do serwisów zewnętrznych, tj. Keycloak, AWS czy Eureka. Sama architektura mikroservisów oparła się na jednym punkcie dostępu do aplikacji zwanym „API Gateway”. Umożliwiło to na hermetyzację wytworzonych serwisów, ich wspólną integracją oraz łatwiejsze zarządzanie. API Gateway, czyli brama – jest pośrednikiem dostępu do innych mikroservisów. Kieruje na nie ruch wedle ściśle określonego schematu. Przypisuje porty, które identyfikują mikroservisy i czuwa nad tym, aby każde żądanie do aplikacji najpierw przechodziło przez nią. Dalszy opis działania aplikacji od strony logicznej jest tematem rozważanych w kolejnych rozdziałach.

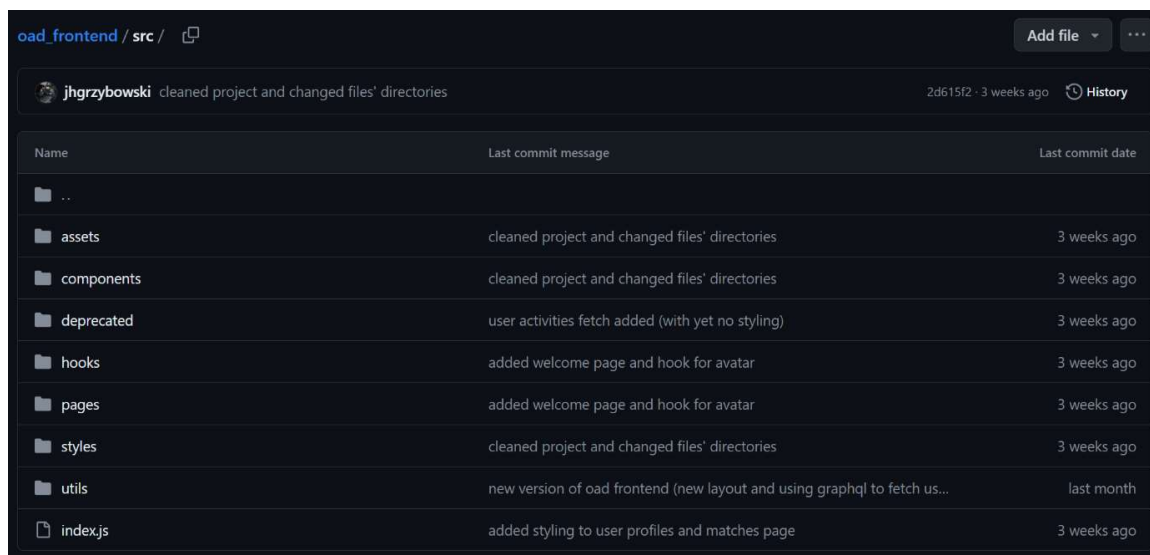
Poniżej przedstawiono strukturę głównego folderu projektu Maven w Java Spring (a więc folder zawierający kod źródłowy mikroservisów w technologii Java Spring):



Name	Last commit message	Last commit date
..		
aws	Core and matching	2 months ago
commons	messenger	3 weeks ago
core	audio messages	last week
database	audio messages	last week
eureka_server	kubernetes	last month
file_storage	audio messages	8 hours ago
gateway	audio messages	last week
messenger	audio messages	8 hours ago
realms/export	New realm configs	2 months ago
sso	audio messages	last week
volume/cache	kubernetes	last month
docker-compose.yaml	audio messages	8 hours ago
pom.xml	upgrade spring version	last month

Graf. 7.3: struktura folderów aplikacji w projekcie Maven w technologii Java Spring

Struktura kodu źródłowego w repozytorium oad_frontend



Name	Last commit message	Last commit date
..		
assets	cleaned project and changed files' directories	3 weeks ago
components	cleaned project and changed files' directories	3 weeks ago
deprecated	user activities fetch added (with yet no styling)	3 weeks ago
hooks	added welcome page and hook for avatar	3 weeks ago
pages	added welcome page and hook for avatar	3 weeks ago
styles	cleaned project and changed files' directories	3 weeks ago
utils	new version of oad frontend (new layout and using graphql to fetch us...	last month
index.js	added styling to user profiles and matches page	3 weeks ago

Graf. 7.4: struktura folderów interfejsu graficznego aplikacji w technologii React JS

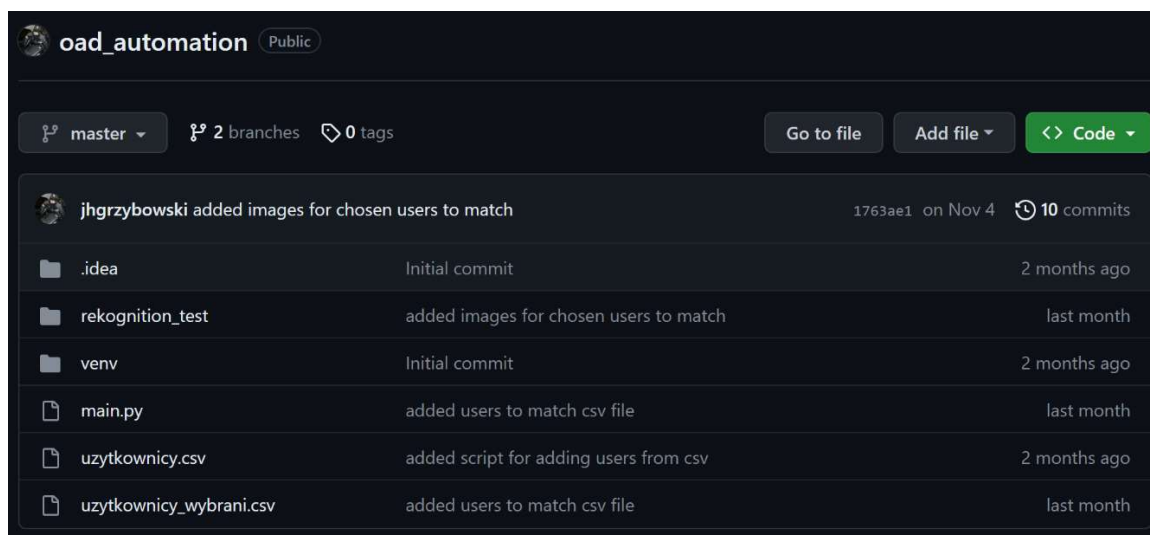
W projekcie interfejsu graficznego występują foldery, w których znajdują się:

- Elementy statyczne strony (logo i inne grafiki).
- Komponenty – składowe interfejsu graficznego (np. pasek nawigacji, profil użytkownika).
- Wyodrębniona grafika po stronie klienta – tzw. *hooks*, które odpowiadają m.in. za pobieranie zasobów z serwera.
- Strony – zdefiniowane widoki w aplikacji, do których użytkownik może się dostać.

- Style – pliki CSS odpowiadające za układ stron i jej część wizualną
- Komponenty łączące aplikację z serwisami zewnętrznymi (np. serwis autoryzacyjny Keycloak)

Omówienie poszczególnych części aplikacji i jej interfejsu graficznego zostało zamieszczone w następnych rozdziałach niniejszej pracy.

Struktura folderów w repozytorium oad_automation



Graf. 7.5: struktura folderów w repozytorium oad_automation

Ostatnie repozytorium, z którego zrzut ekranu widnieje powyżej, zawiera kod źródłowy w języku Python służący do automatycznego tworzenia kont użytkowników i umieszczania zdjęć na ich profilach. Zastosowano to w celu odciążenia twórców od ręcznego testowania aplikacji. Umożliwiło to na przeanalizowanie działania algorytmu na bazie wielu spreparowanych kont użytkowników. Kwestie na ten temat poruszono w dalszej części pracy.

7.2.2. Wersja aplikacji na zakończenie prac inżynierskich

W trakcie pracy nad aplikacją, włożono sporo starań w badanie i eksperymentowanie nad wytworzonym rozwiązaniem. Przyczyni się to na jakość pierwszej wersji produkcyjnej, co jest planowane w przyszłości. Na moment zakończenia pracy inżynierskiej, aplikacja pozostaje w wersji deweloperskiej, która stanowi bazę pod rozwinięcie projektu. Niemniej jednak udało się osiągnąć konkretne efekty, które ukazują jakość aplikacji i jej potencjał. Spełnione zostały główne założenia pracy:

- Przejrzano i przeanalizowano istniejące rozwiązania w celu zebrania wiedzy na temat aplikacji społecznościowych, ich obecnych możliwości, trendów, innowacyjnych funkcjonalności, lecz także dostrzeżenie problemów, z którymi się zmagają.
- Skupiono wiele uwagi na przygotowanie algorytmu, wedle którego użytkownicy byłiby łączeni. Podczas prób na dużym zbiorze danych testowych – wygenerowanych użytkowników z losowym przydziałem zdjęć z pogranicza wielu dziedzin zainteresowań, pasji czy codziennych aktywności uzyskano zadowalające rezultaty. Ostatecznie

zaprojektowano dwa algorytmy, których działanie oraz wyniki zostały opisane w następnych rozdziałach.

- Wytworzono logikę aplikacji, która zawiera wiele nowoczesnych technik programowania, odpowiednie mechanizmy bezpieczeństwa, interfejsy z zasobami na potrzebę ich wizualizacji w aplikacji oraz sposoby orkiestracji całym zbiorem mikroserwisów. Nakład pracy na tę część jest fundamentem dla przyszłego rozwoju projektu.
- Utworzono wstępny interfejs graficzny w formie aplikacji przeglądarkowej, który umożliwia na skorzystanie z wielu zaimplementowanych możliwości. Jego główną cechą jest elastyczność w refaktoryzacji do aplikacji mobilnej, która jest kluczowym celem projektu po zakończeniu prac inżynierskich. Mimo braku wydania wersji produkcyjnej zespół uważa pracę za udaną i wyraża ogromną chęć na jej dalszy rozwój. Członkowie przewidują ewentualne rozszerzenie badań nad jej różnymi aspektami w pracy magisterskiej.

Cały kod źródłowy znalazł się w załącznikach do tej pracy.

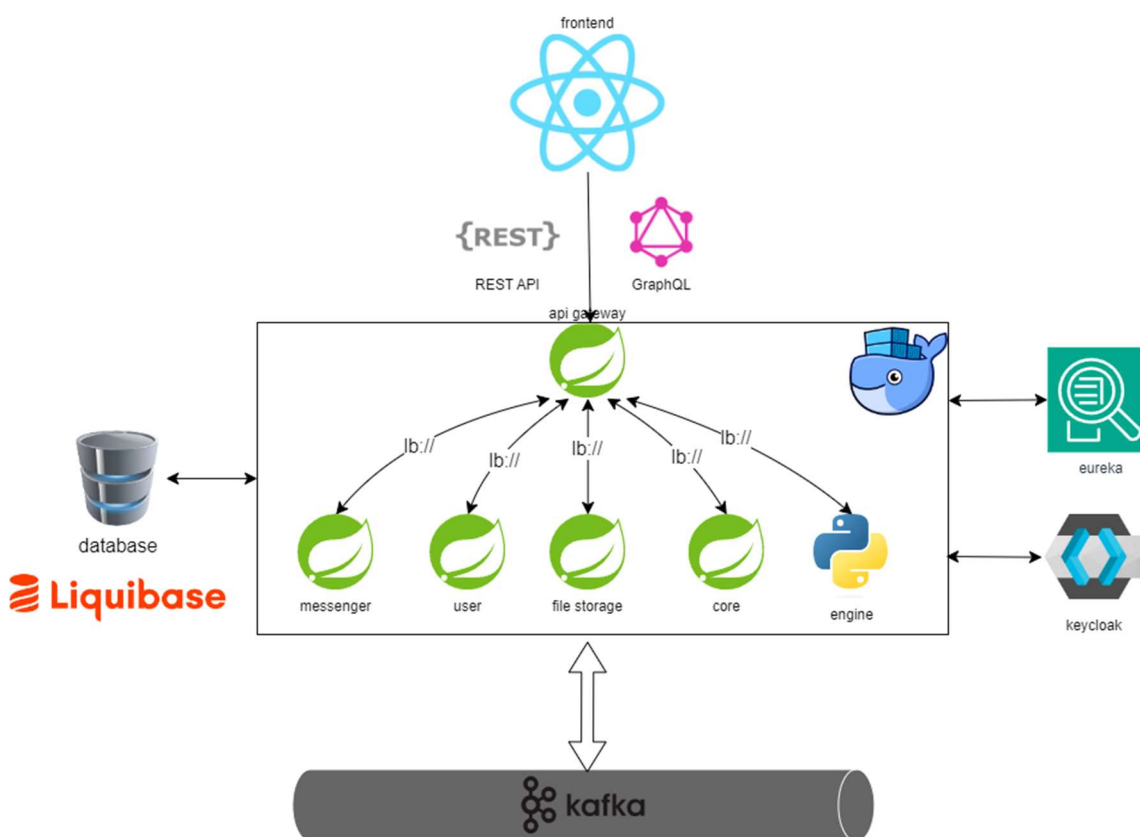
8. Architektura i logika aplikacji

W tej części pracy opisano poszczególne części aplikacji od strony architektonicznej oraz implementacyjnej. Schemat rozdziału jest oparty o krótkie przedstawienie danej technologii bądź różnych metod czy procesów i zjawisk zachodzących w inżynierii oprogramowania. Odbyna się to w ramach wprowadzenia do wytworzonych komponentów czy modułów, aby jak najlepiej zobrazować wykonaną pracę. Cała logika została oparta o framework Java Spring¹⁰.

8.1. Architektura mikroservisów

Jest to styl architektoniczny dla wytwarzania oprogramowania, gdzie tworzone są małe, niezależne programy dedykowane określonym zadaniom¹¹. Niezależność w tym przypadku oznacza, że mogą być uruchamiane i testowane niezależnie od innych komponentów systemu. Każdy mikroservis pełni wyznaczone zadania, lecz ściśle współpracuje z innymi.

Zakładając rozwój aplikacji w następnych latach, nie wykluczając produkcyjnego startu, łatwość skalowania stała się priorytetem. W tym celu funkcjonalność rozłożona została na dedykowane mikroservisy, co zostało zaprezentowane na grafice poniżej.



Graf. 8.1: wizualizacja architektury mikroservisów w aplikacji Once a Day

¹⁰ Źródło dokumentacji: <https://docs.spring.io/spring-framework/reference/index.html>

¹¹ Źródło dokumentacji: <https://spring.io/microservices>

8.1.1. Wyznaczone mikroserwisy

Każdy mikroserwis odpowiada za konkretne zadania:

- **Messenger** – komunikacja między użytkownikami w czasie rzeczywistym z wykorzystaniem protokołów WebSocket oraz STOMP.
- **User** – zarządzający użytkownikami oraz integracją z serwerem autoryzacyjnym Keycloak.
- **File Storage** – przechowywanie oraz odczyt plików.
- **Core** – przyjmowanie aktywności użytkowników.
- **Engine** – algorytmy parowania użytkowników.

8.1.2. Omówienie działania aplikacji w utworzonej architekturze

Komunikacja aplikacji klienckiej przechodzi przez Spring Cloud Gateway, będący bramą na sieć zewnętrzną dla strony serwerowej. Każde żądanie jest autoryzowane na podstawie *Bearer token* obecnego w nagłówku *Authorization* protokołu HTTP, który wydawany jest przez serwer autoryzacyjny Keycloak. Informacje o dostępnych serwisach, na które może przekierować ruch, brama czerpie z oddzielnego serwisu Eureka, do którego co określony interwał zgłaszają się aplikacje, informując tym samym, że są dostępne. W przypadku, gdy przez dłuższy czas *Eureka* nie otrzyma od serwisu informacji o stanie, wyklucza go z listy dostępnych serwisów. *Gateway* regularnie aktualizuje informacje o dostępności serwisów oraz ich liczbie w celu równomiernego rozproszenia ruchu. Zrezygnowano z dedykowanych baz danych dla każdego serwisu, ponieważ zapewnianie integralności w warstwie aplikacji skomplikowałoby kod, a tym samym wydłużyło wydanie pierwszej wersji aplikacji. Baza danych jest jedna, a kontrolę nad jej schematem sprawuje Liquibase, będący narzędziem kontroli wersji dla baz danych. Serwisy, gdy muszą wspólnie wykonać zadanie, komunikują się w sposób nieblokujący przy pomocy *brokera* wiadomości *Kafka*. Każda aplikacja działa jako niezależny kontener, zarządzany przez narzędzie Docker Compose.

8.2. Uwierzytelnianie i autoryzacja

W aplikacji zrezygnowano z implementowania funkcji bezpieczeństwa od zera. Jest to bowiem zadanie czasochłonne, trudne, a tym samym podatne na błędy. Warto w takim wypadku stosować gotowe rozwiązania, zaakceptowane przez społeczność, regularnie aktualizowane i testowane. Do kontroli dostępu i identyfikowania użytkowników wykorzystane zostały szeroko stosowane protokoły OAuth2.0 oraz OpenID Connect, które są zaimplementowane w serwerze autoryzacyjnym Keycloak¹². Poza najważniejszą funkcjonalnością, umożliwiającą zalogowanie oraz rejestrację użytkowników, w łatwy sposób można również wprowadzać określone polityki w zależności od potrzeb. Jednym kliknięciem panelu administratora lub jednym żądaniem HTTP możliwe jest wymuszenie na użytkowniku zmianę hasła przy następnym logowaniu.

¹² źródło dokumentacji: <https://www.keycloak.org/documentation>

8.2.1. OAuth2.0¹³

Protokół wykorzystywany do autoryzacji, czyli zarządzania dostępem do zasobów. Ustanowiony w celu bezpiecznego dzielenia się zasobami pomiędzy niepowiązanymi ze sobą stronami. Kontrola dostępu działa na podstawie o *Access Token* wydawany użytkownikowi, który najczęściej jest w formacie JWT (*JSON Web Token*).

JSON Web Token jest bezpiecznym sposobem na wymianę oświadczeń między stronami komunikacji. Token składa się z 3 części:

- **Header** – zawiera m.in. informację, jakim algorytmem podpisano token.
- **Payload** – składa się z tzw. *claims* (z ang. uznania, oświadczenia), czyli oświadczeń o posiadaczu tokena. Istnieje tutaj dowolność, jakie *claims* mają znaleźć się w tokenie. Nie powinny być to w żadnym wypadku dane poufne.
- **Signature** – to podpis, który utworzono, korzystając z prywatnego klucza serwera autoryzacyjnego.

Klientami są podmioty, które wykonują akcje wymagające autoryzacji. Wyróżnia się ich dwa rodzaje:

- **Confidential** – klienci, którzy są w stanie bezpiecznie uwierzytelnić się w serwerze autoryzacyjnym, mając możliwość bezpiecznego przechowania danych uwierzytelniających (np. *client secret*), są to np. aplikacje uruchomione na serwerze.
- **Public** – klienci, którzy nie mają możliwości przechowywania wrażliwych danych w kodzie aplikacji, są to np. aplikacje mobilne lub przeglądarkowe.

Komunikacja z serwerem odbywać się może poprzez:

- **Back Channel** – aplikacja uruchomiona na serwerze komunikuje się bezpośrednio z serwerem autoryzacyjnym, co jest bezpiecznym sposobem komunikacji.
- **Front Channel** – aplikacja uruchomiona na naszym urządzeniu komunikuje się bezpośrednio z serwerem autoryzacyjnym, będąc mniej bezpiecznym sposobem komunikacji.

W zależności od tego, jakim kanałem odbywa się komunikacja, istnieją różne *grant types*, czyli czynności, przez które należy przejść, aby otrzymać token. W aplikacji wykorzystano następujące *grant types*:

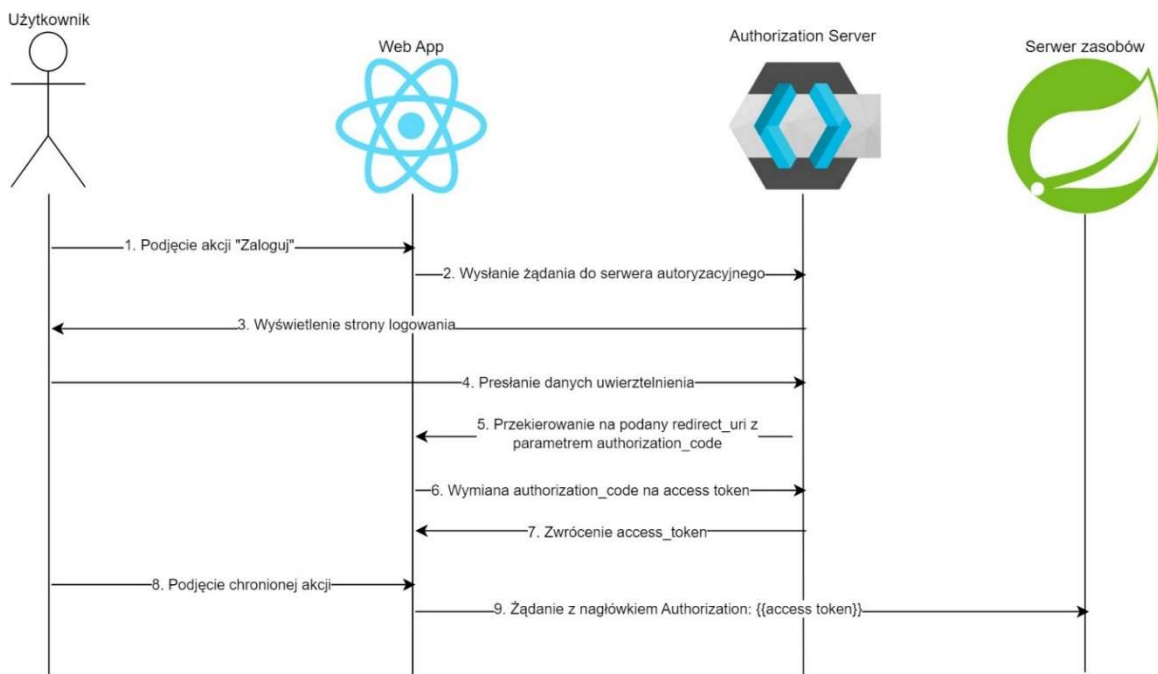
- **Client credentials (Back Channel)** – polega na wymianie *client id* oraz *client secret* na *access token*. Żeby taka wymiana była możliwa, aplikacja musi przechowywać *client secret*, co w przypadku aplikacji *Spring* jest dopuszczalne, ponieważ jest uruchomiona w bezpiecznym środowisku na serwerze. Najlepiej sprawdzi się w komunikacji *backend-to-backend*. W projekcie jest to komunikacja mikroservisu „User” z serwerem Keycloak.

¹³ źródło dokumentacji: <https://datatracker.ietf.org/doc/html/rfc6749>

- **Resource Owner Credentials** – wykorzystywany na czas prac deweloperskich. Z tego *grant type* korzysta Postman, narzędzie do testowania Rest API wykonujące żądania HTTP.

Z każdym testowym żądaniem uruchamiany jest *Pre-request Script*, który pobiera token, przekazując *client_id*, *username*, *password*. Nie zalecane w środowiskach produkcyjnych, przypomina bowiem sposób, w jaki udostępniano zasoby zewnętrznym serwisom przed protokołem OAuth.

- **Authorization Code (Back Channel + Front Channel)** – standardowy cykl, jaki przechodzą użytkownicy w celu zalogowania się do aplikacji. Po podjęciu akcji „Zaloguj”, użytkownik zostaje przekierowany na stronę logowania Keycloak, wyświetlającą formularz z nazwą użytkownika i hasłem. Po poprawnym rozpatrzeniu danych logowania następuje przekierowanie z powrotem na stronę aplikacji, otrzymując tym samym *authorization code*, który jest wymieniany na *access token*. Poniżej przedstawiono graficznie cykl autoryzacji.



Graf. 8.2: cykl autoryzacji użytkownika

8.2.2. OpenID Connect¹⁴

Warstwa tożsamościowa. Rozszerza OAuth2.0 o tzw. *Federated Authentication*, dając możliwość uwierzytelnienia się w jednym serwisie, korzystając z serwisu trzeciej strony.

8.3. Baza danych

8.3.1. PostgreSQL¹⁵

Otwarto-źródłowy system zarządzania relacyjnymi bazami danych. Obecnie szeroko stosowane rozwiązanie dla dużych projektów biznesowych, które swoją sławę zyskało dzięki dużej

¹⁴ źródło dokumentacji: <https://openid.net/developers/how-connect-works/>

¹⁵ źródło dokumentacji: <https://www.postgresql.org.pl>

wydajności i skalowalności. Wspiera język SQL, a także wprowadza szereg dodatkowych funkcjonalności.

8.3.2. Liquibase¹⁶

Otwarto-źródłowe narzędzie do wersjonowania baz danych. Zmiany schematu bazy realizowane są przez skrypty, zapisane w plikach, które są niezależne od silnika baz danych. Znacznie ułatwia to w razie potrzeby migrację do innych systemów. Historia zmian widoczna w repozytorium pozwala przeprowadzić recenzję kodu, zanim zostanie połączony z główną gałęzią repozytorium.

```
<createTable tableName="label_weights">
  <column name="label_weight_id" type="BIGINT" autoIncrement="true">
    <constraints primaryKey="true" nullable="false"/>
  </column>
  <column name="user_id" type="BIGINT"/>
  <column name="label_id" type="BIGINT"/>
  <column name="weight" type="int"/>
  <column name="updated_date" type="date"/>
  <column name="active" type="boolean" defaultValue="true"/>
</createTable>
<addForeignKeyConstraint
  baseColumnNames="user_id"
  baseTableName="label_weights"
  constraintName="label_weights_user_id_fkey"
  referencedColumnNames="user_id"
  referencedTableName="users"
/>
<addForeignKeyConstraint
  baseColumnNames="label_id"
  baseTableName="label_weights"
  constraintName="label_weights_label_id_fkey"
  referencedColumnNames="label_id"
  referencedTableName="rekognition_labels"
/>
<createIndex indexName="label_user_id_idx" tableName="label_weights">
  <column name="user_id"/>
</createIndex>
<createIndex indexName="label_id_idx" tableName="label_weights">
  <column name="label_id"/>
</createIndex>
```

Listing 8.1: skrypt w formacie XML tworzący tabelę `label_weights`, ograniczenia klucza obcego oraz indeksy na kluczowych pod względem zapytań tabel.

W przypadku, gdy uniwersalne znaczniki Liquibase nie wystarczą, istnieje możliwość natywnych zapytań dzięki znacznikowi `<sql>`. Tworząc tabelę `matches`, reprezentującą połączenia

¹⁶ źródło dokumentacji: <https://www.liquibase.org>

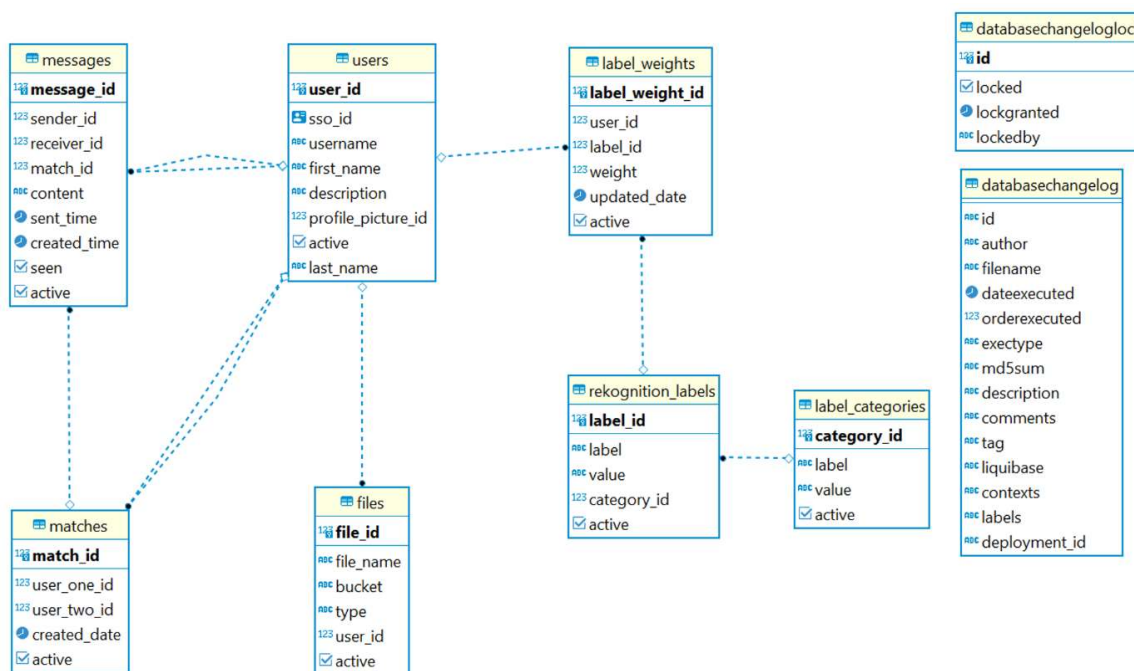
między użytkownikami, konieczne było wprowadzenie ograniczenia, aby niemożliwe było utworzenie dwóch połączeń pomiędzy dwoma tymi samymi użytkownikami.

```
<sql>
    create unique index unique_match on
    matches(greatest(user_one_id,user_two_id),
    least(user_one_id,user_two_id), active);
</sql>
```

Listing 8.2: ograniczenie w tabeli matches poprzez natywne zapytanie SQL

Powyższe ograniczenie nie pozwoli utworzyć drugiego połączenia pomiędzy dwoma użytkownikami, nawet jeśli kolejność identyfikatorów będzie inna.

8.3.3. Struktura tabel



Graf. 8.3: ER diagram wygenerowany przez narzędzie Dbeaver

W bazie danych można wyróżnić następujące tabele, co widać na powyższej grafice:

- **messages** – przechowuje wiadomości z komunikatorów tekstowych użytkowników.
- **matches** – informacja o połączeniach pomiędzy użytkownikami.
- **files** – przechowuje informacje o plikach, gdzie się znajdują i do kogo należą.
- **users** – informacje o użytkownikach, można zwrócić uwagę na kolumnę **sso_id** przechowującą identyfikator na serwerze Keycloak.
- **label_weights** – najważniejsza tabela przechowująca informacje o preferencjach użytkowników. To tutaj zapisywane jest, ile razy u danego użytkownika została wykryta określona etykieta.

- **rekognition_labels** – etykiety, wykrywane przez Amazon Rekognition, które brane są pod uwagę podczas łączenia użytkowników.
- **label_categories** – kategorie etykiet.
- **databasechangesloglock** – tabela utworzona w Liquibase, wykorzystywana do zakładania blokady podczas wykonywania skryptów.
- **databasechangeslog** – historia wykonanych przez Liquibase skryptów.

8.4. Application Programming Interface

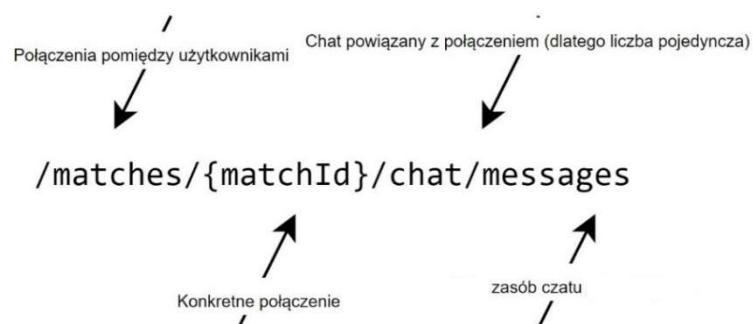
API to kontrakt, według którego obie strony komunikują się ze sobą. Stronami są dwa niezależne programy, inaczej niż w przypadku interfejsu użytkownika, gdzie po jednej stronie komunikacji stoi człowiek. Od strony korzystającej z API nie jest wymagana wiedza o implementacji. W aplikacji wydzielono klienta oraz serwer, dla których konieczne jest ustalenie kontraktu komunikacji. Zdecydowano się skorzystać z wielu jego form.

8.4.1. REST (Representational state transfer)

Informacje zawarte w każdym zapytaniu muszą wystarczyć do przetworzenia żądania, tzn. serwer nie może przechowywać stanu po swojej stronie. W aplikacji dostępne są kontrolery, jeden dla każdego domenowego obiektu. Zgodnie z dobrymi praktykami projektowania REST API, nie stosuje się czasowników w nazwach punktów *endpoints* (z ang. punkt zakończenia, skojarzona lokalizacja z zasobami). URI wskazuje na zasób, a to jaką akcję na zasobie należy wykonać, sugeruje wykorzystana metoda HTTP.

```
@GetMapping("/matches/{matchId}/chat/messages")
public ResponseEntity<Page<ResponseChatMessageDTO>> getChatMessages(@PathVariable Long matchId,
                                                                    @PageableDefault(sort = {"id"}, direction = Sort.Direction.DESC)
                                                                    Pageable pageable) {
    final var page = messageService.getMessages(matchId, pageable);
    return ResponseEntity.ok(page);
}
```

Listing 8.3: kod kontrolera zajmującego się wyświetlaniem wiadomości



Graf. 8.4: specyfikacja URI zapytania typu REST

Wykorzystana metoda HTTP GET służy do pobrania zasobu. Z racji, że *endpoint* zakończony jest fragmentem w liczbie mnogiej, klient może spodziewać się nie pojedynczego elementu, a zbioru elementów w odpowiedzi.

8.4.2. GraphQL¹⁷

GraphQL jest alternatywą dla REST API, a jego przewaga przejawia się w elastyczności. Jest to język zapytań, w którym każde zapytanie o konkretny zasób można dowolnie definiować. Klient korzystający z API ma możliwość odpytać serwer, jakie dane oraz w jakim formacie je zwraca. Następnie można zażądać tylko wybranego fragmentu danych. W przypadku, gdy z API korzysta więcej niż jeden klient np. aplikacja przeglądarkowa oraz aplikacja mobilna, mogą one wymagać innego zestawu danych, czego powodem może być różnica w obsługiwanej rozdzielczości przez klienta lub charakterystyka jego sieci. GraphQL pozwala bowiem na zredukowanie otrzymywanych danych, co nie zawsze jest możliwe w podejściu REST API. Dostosowanie API pod konkretnego klienta wiązałoby się z tworzeniem osobnych kontrolerów lub korzystania ze wzorca projektowego strategii, przez co kod w warstwie serwisowej znacznie by się rozrósł.

Jako przykład, poniżej zaprezentowano żądanie w GraphQL, które wyświetla profil użytkownika w aplikacji:

```
{
  userByUsername(username: "bbuilder") {
    id
    username
    avatar
    activities
    firstName
    lastName
    matches {
      id
      username
      avatar
    }
  }
}
```

Listing 8.4: żądanie wyświetlenia profilu użytkownika w aplikacji przeglądarkowej

¹⁷ źródło dokumentacji: <https://graphql.org>

```

{
  "data": {
    "userByUsername": {
      "id": "238",
      "username": "bbuilderr",
      "avatar": "http://10.9.0.1:8081/files/1708",
      "activities": [
        "http://10.9.0.1:8081/files/1110",
        "http://10.9.0.1:8081/files/1111",
        "http://10.9.0.1:8081/files/1112",
        "http://10.9.0.1:8081/files/1113",
        "http://10.9.0.1:8081/files/1114",
        "http://10.9.0.1:8081/files/1115",
        "http://10.9.0.1:8081/files/1116",
        "http://10.9.0.1:8081/files/1117"
      ],
      "firstName": "bbu",
      "lastName": "uilderr",
      "matches": [
        {
          "id": "191",
          "username": "natthorn",
          "avatar": null
        },
        {
          "id": "192",
          "username": "colthoma",
          "avatar": null
        },
        {
          "id": "239",
          "username": "bigfreakguy",
          "avatar": "http://10.9.0.1:8081/files/1709"
        },
        {
          "id": "289",
          "username": "amyricha",
          "avatar": null
        }
      ]
    }
  }
}

```

Listing 8.5: odpowiedź serwera dla aplikacji przeglądarkowej (w formie JSON)

```
{
  userByUsername(username: "bbuilderr") {
    id
    username
    avatar
    firstName
    lastName
  }
}
```

Listing 8.6: żądanie wyświetlenia profilu użytkownika w aplikacji mobilnej

```
{
  "data": {
    "userByUsername": {
      "id": "238",
      "username": "bbuilderr",
      "avatar": "http://10.9.0.1:8081/files/1708",
      "firstName": "bbu",
      "lastName": "uilderr"
    }
  }
}
```

Listing 8.7: odpowiedź serwera dla aplikacji mobilnej (w formie JSON)

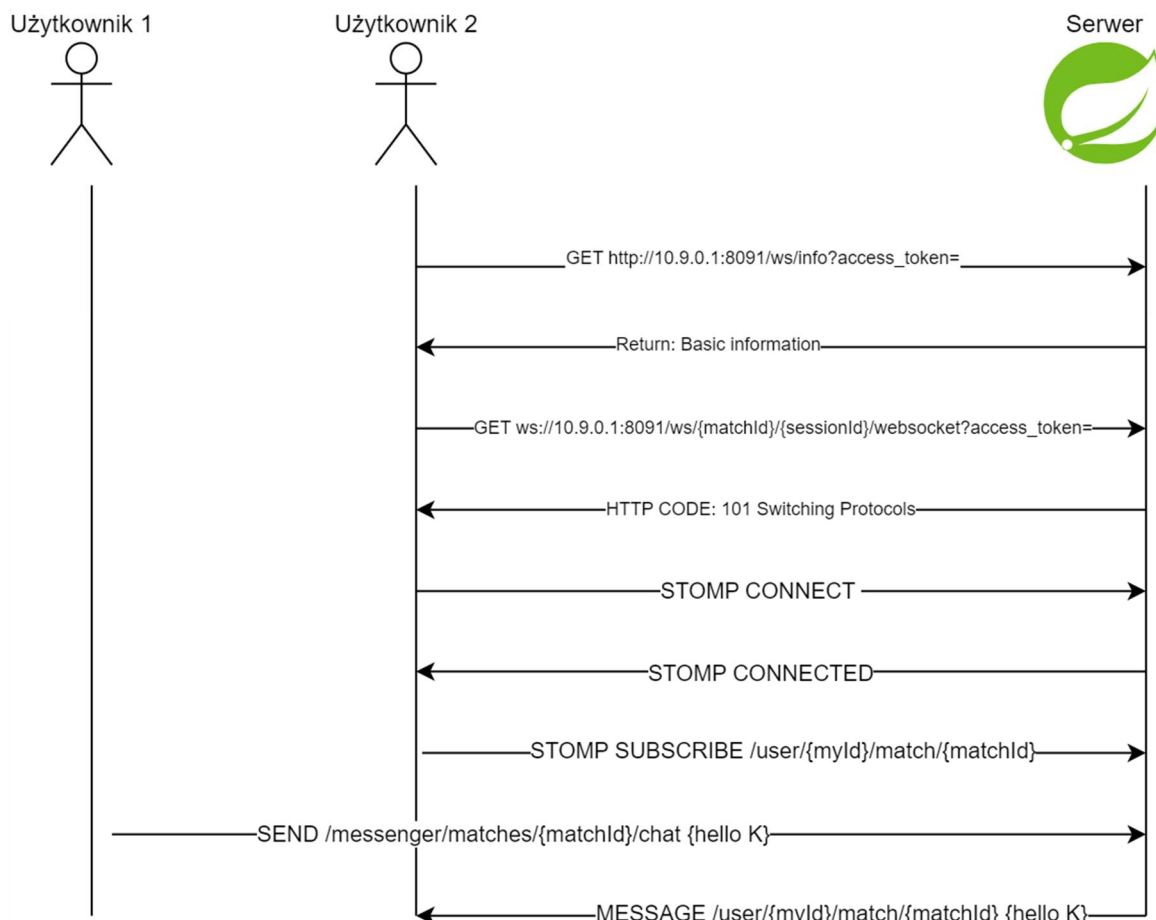
Aplikacja przeglądarkowa z racji, że korzysta z większego ekranu niż ekran smartfona, jest w stanie wyświetlić więcej informacji na jednej stronie, a bardziej stabilne, domowe łącze jest w stanie wysłać więcej danych w krótszym czasie. Podsumowując, istnieje duża swoboda przy tym jakie dane można pobrać z zasobu i umieszczać na profilu użytkownika w interfejsie graficznym.

8.4.3. WebSocket¹⁸

Protokół umożliwiający dwukierunkową komunikację aplikacji klienckiej z serwerem w jednym połączeniu TCP. Wiadomości powinny w małym stopniu wykorzystywać przepustowość łącza, dlatego nagłówki wysyłane są tylko przy zestawianiu połączenia. WebSocket sam w sobie jest lekkim protokołem, nie definiuje formatu przesyłanych danych. Jako protokół wyższej warstwy w aplikacji wykorzystano STOMP, który jest prostym protokołem definiującym komunikację w formacie tekstu.

¹⁸ źródło dokumentacji: <https://docs.spring.io/spring-framework/docs/4.3.x/spring-framework-reference/html/websocket.html>

Protokół STOMP został wykorzystany w implementacji komunikatora, umożliwiając połączonym użytkownikom wymianę wiadomości w czasie rzeczywistym, jego cykl przedstawiono poniżej.



Graf. 8.5: cykl wymiany wiadomości użytkowników za pomocą WebSocket i protokołu STOMP

8.5. Logika mikroservisów w Spring Framework¹⁹

Do budowania małych, łatwo zarządzanych komponentów całej aplikacji wykorzystano Spring Framework, najczęściej wybierany i najszerzej wspierany *framework*²⁰ dla języka Java. Spring zapewnia usprawnienia w pisaniu aplikacji takie jak *Inversion of Control* (z ang. odwrócenie sterowania) czy *Dependency Injection* (z ang. Wstrzykiwanie Zależności), ale również dostarcza dedykowanych projektów pod określone cele. Z wykorzystaniem Springa w bardzo prosty i szybki sposób można wytworzyć zabezpieczoną aplikację przeglądarkową komunikującą się z bazą danych czy reaktywne API Gateway. Poza samym frameworkiem język Java bogaty jest w biblioteki usprawniające pracę programisty.

8.5.1. Maven²¹

Do zarządzania projektem i jego zależnościami w projekcie wykorzystywane jest narzędzie Maven. Bazuje na plikach `pom.xml` definiujących moduły i ich zależności. Dodatkowo usprawnia proces budowania aplikacji, a przez różne wtyczki pozwala wpłynąć na jego przebieg.

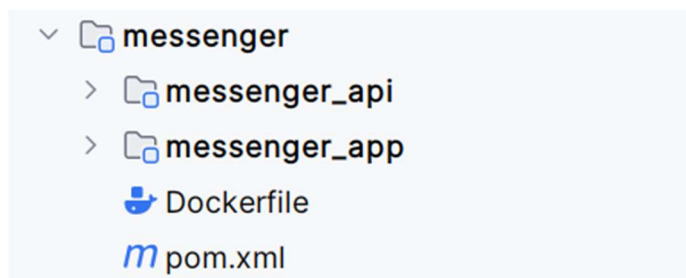
¹⁹ źródło dokumentacji: <https://docs.spring.io/spring-framework/reference/index.html>

²⁰ rodzaj struktury, szkieletu oprogramowania oraz zestaw narzędzi, którego używa się w procesie tworzenia aplikacji, źródło: <https://thestory.is/pl/journal/co-to-jest-framework/>

²¹ źródło dokumentacji: <https://maven.apache.org/what-is-maven.html>

8.5.2. Struktura projektu

W każdym z serwisów zastosowano określoną hierarchię plików, aby zarządzanie zależnościami było jak najprostsze. Każdy projekt ma podstawową strukturę, która prezentuje się następująco:



Graf. 8.6: przykładowa struktura jednego z mikroservisów aplikacji

Każdy z modułów, podzielony został na dwa mniejsze: „api” oraz „app”. Ten drugi zawiera kod źródłowy wykonujący dedykowaną logikę mikroservisów. Moduł API nie zawiera kodu wykonywalnego, a definicję obiektów DTO, wiadomości Kafka. Taki podział jest konieczny, ponieważ istnieje wtedy możliwość importowania definicji z innych mikroservisów, nie tworząc zależności cyklicznych (ang. *circular dependency*), gdy zachodzi potrzeba komunikacji między nimi poprzez HTTP lub Kafka.

Pliki źródłowe podzielone są na dedykowane pakiety, z których najczęściej powtarzającymi się są:

- **advice** – aspekty
- **controller** – kontrolery
- **service** – serwisy
- **repository** – repozytoria JPA
- **config** – klasy konfiguracyjne
- **producer** – producenci wiadomości Kafka
- **listener** – konsumenci wiadomości Kafka

8.5.3. Techniki wytwarzania oprogramowania w projekcie

SOLID²²

Są to określone zasady projektowania obiektowego pozwalające utrzymać kod w czytelnej i elastycznej do modyfikacji postaci, czyniąc go łatwiejszym w zarządzaniu. Wraz z rozwojem programu istnieje ryzyko, że stanie się on coraz trudniejszy w rozszerzaniu, a zmiany będą wiązać się z modyfikacją już istniejącego kodu, co jest podatne na błędy.

Aby zapobiec tym problemom, dwóch bogatych w doświadczenie programistów Robert C. Martin oraz Michael Feathers opracowało zestaw dobrych praktyk SOLID. Każda litera odpowiada jednej zasadzie:

S – Single Responsibility – klasa mająca jedną odpowiedzialność jest łatwiejsza w zrozumieniu. Ma również mniej przypadków testowych, które łatwiej jest pokryć.

²² źródło dokumentacji: <https://www.baeldung.com/solid-principles>

O – Open/Closed – klasa powinna być zaprojektowana w taki sposób, aby modyfikacja jej kodu była konieczna jedynie podczas naprawy błędów. Powinna jednak być otwarta na rozszerzanie.

L – Liskov Substitution²³ – zależności pomiędzy klasami powinny być na odpowiednim poziomie abstrakcji, tak żeby klasa pochodna mogła być użyta w miejscu klasy bazowej.

I – Interface Segregation – interfejsy powinny być jak najmniejsze, aby koszt ich implementacji był jak najmniejszy. Dzięki temu można implementować interfejsy, które rzeczywiście są potrzebne.

D – Dependency Inversion – klasy powinny zależeć od interfejsów lub klas abstrakcyjnych zamiast od konkretnych ich implementacji.

W ciągu prac nad kodem źródłowym kierowano się również podręcznikiem do zachowania „czystego kodu” autorstwa przytoczonego wcześniej informatyka Roberta C. Martina [14].

Kolejkowanie²⁴

Mikroserwisy często współpracują w celu przetworzenia żądania od klienta. Komunikacja synchroniczna w takim wypadku nie jest najlepszym rozwiązaniem, wymaga bowiem rezerwowania zasobów każdej strony komunikacji na czas trwania połączenia.

Klient ↔ SERWIS A ↔ SERWIS B ↔ SERWIS C

Przypuszczając hipotetyczne żądanie powyżej, każda strona chcąc połączyć się z kolejnym serwisem musiałaby czekać na gotowość drugiej strony na przyjęcie połączenia, a następnie to połączenie utrzymywać przez cały czas przetwarzania żądania. Jeśli operacja serwisu C byłaby czasochłonna, klient oraz serwisy A i B są zmuszone utrzymywać cały czas połączenie. Alternatywą dla tak ścisłego powiązania jest komunikacja asynchroniczna poprzez kolejkowanie. Do takiej właśnie komunikacji wykorzystana została platforma Apache Kafka²⁵. Serwisy komunikują się poprzez wiadomości nazywane zdarzeniami, które umieszczane są w odpowiednich *topics*. Komunikacja może być:

- jednokierunkowa – nie spodziewa się odpowiedzi w najbliższym czasie
- dwukierunkowa – spodziewa się wiadomości zwrotnej wysłanej na dedykowany *topic*

²³ źródło dokumentacji: <https://www.baeldung.com/cs/liskov-substitution-principle>

²⁴ źródło dokumentacji: <https://microservices.io/patterns/communication-style/messaging.html>

²⁵ źródło dokumentacji: <https://kafka.apache.org/documentation/>

Programowanie zorientowane aspektowo

To paradygmat, w którym stosuje się separację niezależnych od siebie zagadnień. W aplikacji znalazł zastosowanie w metodach, które poza główną funkcjonalnością wykonują też zadania poboczne. Zadania te można wykonywać przed lub po wykonaniu funkcji, a wyrażenia pozwalają zawęzić metody objęte aspektem. Miejsce zastosowania aspektu przed wykonaniem właściwej metody jest funkcjonalność komunikatora, gdzie przed przetworzeniem wiadomości użytkownika czy został połączony z docelowym użytkownikiem, co widać na poniższym listingu.

```
@Before("execution(* org.once_a_day.messenger.messenger_app.service.MessageService.process(..) " +
    "&& args(matchId,message)")
public void beforeMessageSend(Long matchId, InputChatMessageDTO message) {
    final var receiver = message.getReceiverId();
    final var sender = message.getSenderId();
    Optionals.firstNonEmpty(
        () -> matchRepository.findByIdAndUserOneIdAndUserTwoId(matchId, sender, receiver),
        () -> matchRepository.findByIdAndUserOneIdAndUserTwoId(matchId, receiver, sender)
    ).orElseThrow(() -> new ApplicationException(ExceptionCode.USERS_NOT_MATCHED));
}
```

Listing 8.8: przykład zastosowania aspektu (przed podjęciem czynności)

Aspekt po wykonaniu właściwej metody zastosowano w mikroserwisie **FileStorage**, gdzie po zapisaniu zdjęcia wysyłany jest komunikat za pomocą zdarzenia Kafka. Kod źródłowy spełniający to zadanie wylistowano poniżej.

```
@AfterReturning(pointcut = "execution(* org.once_a_day.file_storage.file_storage_app.service.FileService.uploadProfilePicture(..) && args(userId, fileName, bytes))", returning="fileId")
public void afterUploadProfileImage(Long userId, String fileName, byte[] bytes, Long fileId) {
    final var message = buildProfilePictureChangedMessage(userId, fileId);
    uploadFileKafkaProducer.send(message);
}
```

Listing 8.9: przykład zastosowania aspektu (po podjęciu czynności)

Technika ta pozwala trzymać się zbioru dobrych praktyk SOLID, a dokładnie zasady **Single Responsibility Principle**. Dzieje się tak, ponieważ metody serwisów odgrywają ściśle zdefiniowaną rolę, mają jedno zadanie. Zadaniem metody **uploadProfilePicture** w serwisie **FileService** jest zapis plików, co możemy wyczytać z nazwy metody. Nazwa metody nie mówi jednak nic o emitowaniu odpowiedniego zdarzenia.

Wzorce projektowe²⁶

Są rozwiązaniem często występujących problemów w projektowaniu systemów IT. Dobrze wykorzystane mogą polepszyć czytelność, a tym samym lepsze zrozumienie tak napisanego kodu. W wielu przypadkach pomogą trzymać się zasad SOLID. Umiejętność zastosowania odpowiedniego wzorca jest tak samo ważna, jak ich właściwa znajomość, ponieważ źle zastosowany wzorec może przynieść skutki odwrotne do zamierzonych.

Wzorce projektowe znalazły zastosowanie w wytwarzanej aplikacji. *Chat* (komunikator), który daje użytkownikom możliwość wysłania dwóch typów wiadomości CHAT oraz AUDIO. Każdy typ wiadomości powinien być przetwarzany w inny sposób, wiadomości tekstowe są zapisywane w bazie danych i jeśli to możliwe wysyłane do odbiorcy. Wiadomości AUDIO natomiast wymagają

²⁶ źródło dokumentacji: <https://refactoring.guru/design-patterns/strategy>

przetworzenia i zapisu na dysku przez osobne mikroserwisy, do których plik jest przekazywany z wykorzystaniem kolejkowania Kafka.

Najprostszym rozwiązaniem w tym wypadku byłoby zastosowanie instrukcji warunkowej w zależności od typu wiadomości, gdzie w blokach wykonywana byłaby specjalistyczna funkcjonalność. Skutkiem takiego podejścia byłaby przerośnięta klasa. Chęć dodania kolejnych typów wiadomości, jakimi mogłyby być IMAGE, LINK czy GIF byłaby równoznaczna z rozrostem obszernej klasy, wiążąc się z ryzykiem naruszenia działającej już funkcjonalności.

Rozwiązaniem jest wzorec projektowy strategii, gdzie wydzielono interfejs, który wykonuje konkretne zadanie na wiele różnych sposobów, w tym przypadku przetworzenie wiadomości, a każdy specyficzny algorytm jest implementacją tego właśnie interfejsu.

```
public interface ProcessMessageStrategy {  
    void process(final Long matchId, InputChatMessageDTO message);  
}
```

Listing 8.10: interfejs wspólny dla każdej wiadomości `ProcessMessageStrategy.java`

```
@Override  
public void process(final Long matchId, final InputChatMessageDTO message) {  
    final var entity = createAudioMessage(matchId, message);  
    final var save = save(entity);  
    final var processAudioRequest = buildProcessAudioRequest(message, save);  
    messengerKafkaProducer.send(processAudioRequest);  
}
```

Listing 8.11: implementacja metody `process` dla wiadomości AUDIO
`ProcessAudioMessageImpl.java`

```
@Override  
public void process(final Long matchId, final InputChatMessageDTO message) {  
    final var entity = createRegularMessage(matchId, message);  
    final var save = save(entity);  
    sendMessageService.send(save.getId());  
}
```

Listing 8.12: implementacja metody `process` dla wiadomości CHAT
`ProcessChatMessageImpl.java`

Dodatkowo klasa kontekstu, która na podstawie parametru, w tym przypadku typu wiadomości zwraca właściwą implementację.

```
@PostConstruct
void init() {
    map.put(MessageType.AUDIO, processAudioMessageImpl);
    map.put(MessageType.CHAT, processChatMessageImpl);
}

@Override
public ProcessMessageStrategy resolve(MessageType type) {
    if (map.containsKey(type)) {
        return map.get(type);
    } else {
        throw new ApplicationException(ExceptionCode.CUSTOM_MESSAGE, "Cant handle this type of message");
    }
}
```

Listing 8.13: kontekst rozwiązujący strategię na podstawie typu wiadomości

`ProcessMessageContextImpl.java`

Skorzystanie z takiej struktury klas jest proste i czytelne ze względu na określenie typu wiadomości.

```
@Override
public void process(final Long matchId, InputChatMessageDTO message) {
    processMessageContext.resolve(message.getType())
        .process(matchId, message);
}
```

Listing 8.14: klasa `MessageService.java`

W ten sposób zachowana została zasada **Open/Close Principle**, dodanie kolejnych typów wiadomości nie będzie wiązać się z modyfikacją już istniejącego kodu, a stworzeniem zupełnie nowej klasy implementującej wspólny interfejs, która będzie uwzględniona w kontekście.

9. Zaproponowane mechanizmy łączenia użytkowników

Posiadając już strukturę i logikę aplikacji, trzeba zastanowić się jak zaimplementować algorytm łączenia użytkowników. Wytworzona aplikacja umożliwia użytkownikom dodanie zdjęć, z których wydobywane są ich etykiety. Posłużą one jako dane wejściowe algorytmu do skonstruowania mechanizmu. Za rozwiązanie referencyjne w projektowaniu silnika łączącego posłuży *collaborative filtering* (z ang. filtrowanie kolaboracyjne) [15], które jest techniką rekomendacji treści użytkownikom. W autorskim podejściu, wykorzystanie techniki rekomendacji będzie bazą dla utworzenia modelu użytkownika. Następnie takie modele będą porównywane i wykorzystując ideę algorytmu stabilnych połączeń Gale-Shapleya, tworzone będą pary.

9.1. Model użytkownika

Użytkownicy, korzystając z aplikacji, każdego dnia mają możliwość przesłania jednego zdjęcia opisującego minione 24 godziny. Może to być zdjęcie z boiska, książki lub plakat filmu w kinie. Po udanym zapisie na dysku wysyłane jest żądanie do usługi AWS Rekognition²⁷ o wyciągnięcie etykiet ze zdjęcia. Każda z otrzymanych w odpowiedzi etykiet jest zapisywana do bazy danych. Jeśli wcześniej nie wykryto u użytkownika danej etykiety, w tabeli *label_weights* tworzony jest nowy wiersz z początkową wagą 1 reprezentujący powiązanie pomiędzy etykietą a użytkownikiem. W sytuacji, gdy etykieta została wykryta kolejny raz u użytkownika, waga połączenia inkrementowana jest o 1.

Z biegiem czasu tabela rozrasta się, dlatego otrzymywany jest coraz dokładniejszy obraz zainteresowań użytkowników.

Tabela 9.1: Zestawienie wykrytych etykiet z aktywności użytkowników

label_weight_id	user_id	label_id	weight
201	5	5	4
248	5	3	1
233	7	5	2
203	8	123	11

Informacja o tym, jakie etykiety wykryto u użytkownika, trzymana jest w bazie danych i uzupełniana na bieżąco przez cały dzień. Na podstawie tych informacji tworzona jest macierz cech użytkowników. Wierszem jest identyfikator użytkownika, kolumnami etykiety, a wartościami wagi, czyli liczba określająca, ile razy u danego użytkownika wykryto etykietę. Do budowy macierzy wykorzystywane są tylko etykiety, które zostały wykryte przynajmniej raz, aby skrócić czas obliczeń. Macierz jest rzadka, tzn. większość jej elementów jest równa zero.

²⁷ źródło dokumentacji: <https://docs.aws.amazon.com/rekognition/latest/dg/what-is.html>

Tabela 9.2: Macierz cech użytkowników

	American Football	Art			Ping Pong paddle	Pizza
1	1	0			3	0
2	0	1			0	5
....						
....						
360	3	1			5	2
361	0	0			1	3

9.2. Algorytmy łączenia

Celem programu jest utworzenie najbardziej optymistycznych połączeń z puli wszystkich użytkowników tzn. uzyskać taki zbiór połączeń, gdzie każdy użytkownik będzie dopasowany do najbliższego sobie użytkownika. Rozwiązanie tego problemu jest nietrywialne, a w wielu przypadkach niemożliwe. Dlatego spośród $\frac{n!}{2(n-2)!}$ kombinacji połączeń trzeba znaleźć najlepsze możliwe. Metoda *brute force* nie sprawdzi się w tym przypadku, ponieważ złożoność algorytmu $n!$ dla dużej liczby użytkowników jest nie do zaakceptowania pod względem efektywności działania algorytmu. W związku z tym należy wziąć pod uwagę, że wynik algorytmu będzie tylko przybliżeniem.

9.2.1. Odległość cosinusowa

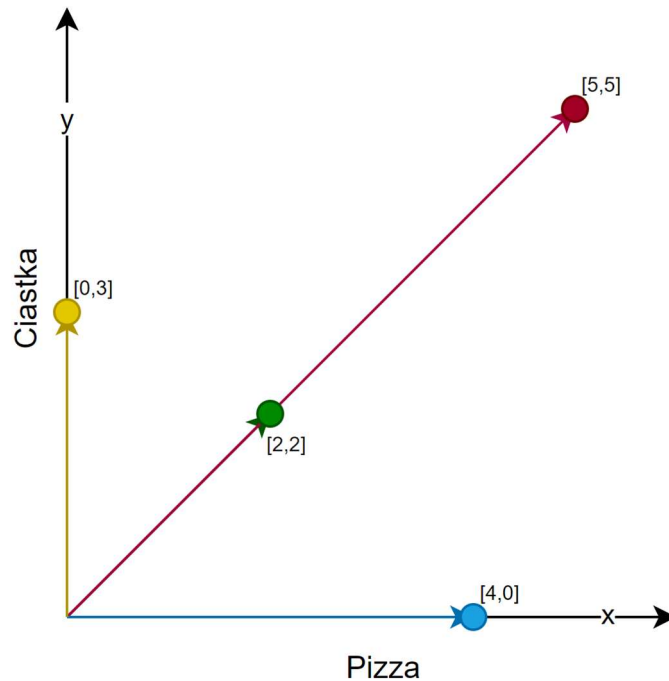
Podobieństwo użytkowników wyrażamy w odległości cosinusowej. Cosinus kąta pomiędzy wektorami opisującymi upodobania użytkowników w przestrzeni n -wymiarowej mającymi swój początek w punkcie $[0,0, \dots, 0]$ przestrzeni, a koniec w punktach $[x_1, x_2, \dots, x_n]$ jest wyliczaną miarą odległości, im wartość bliższa 1, tym kąt jest mniejszy, tym użytkownicy są do siebie bardziej podobni.

Dla lepszego zrozumienia, w jaki sposób cosinus kąta może wyrazić podobieństwo można uprościć sytuację na przykładzie czterech użytkowników (czerwony, zielony, niebieski, żółty) w dwuwymiarowej przestrzeni etykiet „Pizza” (oś odciętych) oraz „Ciastka” (oś rzędnych).

Ze zdjęć użytkowników poszczególne etykiety zostały wykryte następująco:

- Czerwony (C) - Pizza: 5, Ciastka: 5
- Zielony (Z) - Pizza: 2, Ciastka: 2
- Niebieski (N) - Pizza: 4, Ciastka: 0

- Żółty (Y) - Pizza: 0, Ciastka: 3



Graf. 9.1: wykres odległości cosinusowej w zadanej przestrzeni

Z wykresu można zaobserwować, że kąt pomiędzy użytkownikiem C i Z wynosi 0, cosinus więc równy jest 1, co według odległości cosinusowej oznacza, że użytkownicy są identyczni. Biorąc samo położenie punktów w przestrzeni, nie można określić identyczności. W rezultacie, co trzeba podkreślić, odległość cosinusowa dobrze porównywała proporcje cech względem siebie, a nie ich moc. Jeśli skorzystano by z odległości euklidesowej, najbliższym użytkownikowi Z jest użytkownik Y, co nie byłoby zadowalającym wynikiem. Użytkownicy G i R lubią tak samo pizzę i ciastka. Użytkownik Y jest zdecydowanym fanem ciastek, nieuwzględniającym pizzy w swojej diecie. Tak więc użytkownikowi G zdecydowanie bliżej do użytkownika R, mimo że algorytm wykrył ciastka u G i Y podobną ilość razy.

Odległość cosinusowa wyraźnie może zasignalizować zupełne przeciwieństwa, tak jak w przypadku użytkowników Y i N, gdzie cosinus wyniósłby 0. Korzystając z jej właściwości, zaprojektowano dwa algorytmy, które zostaną przebadane pod kątem wydajności mechanizmu łączenia.

9.2.2. Algorytm losowy

1. Z macierzy wybiera się losowo jednego użytkownika
2. Poszukiwane jest dla niego najlepsze dopasowanie (najmniejsza odległość cosinusowa)
3. Utworzona para jest zapisywana i nie jest brana pod uwagę przy następnych iteracjach
4. Powrót do K1 do momentu aż każdy użytkownik zostanie sparowany*

**w przypadku parzystej liczby użytkowników, w przeciwnym wypadku jeden użytkownik pozostaje bez pary*

9.2.3. Algorytm ze wstępnym posortowaniem

1. Obliczana jest odległość cosinusowa pomiędzy każdą możliwą parą
2. Sortowana jest lista par według odległości
3. Dokonywana jest iteracja po każdym elemencie listy
 - a. Jeśli co najmniej jeden użytkownik tworzący parę został wcześniej wybrany, kombinacja jest pomijana, powrót do K3
 - b. Para jest zapisywana i brana pod uwagę przy następnych iteracjach

10. Analiza opracowanych algorytmów

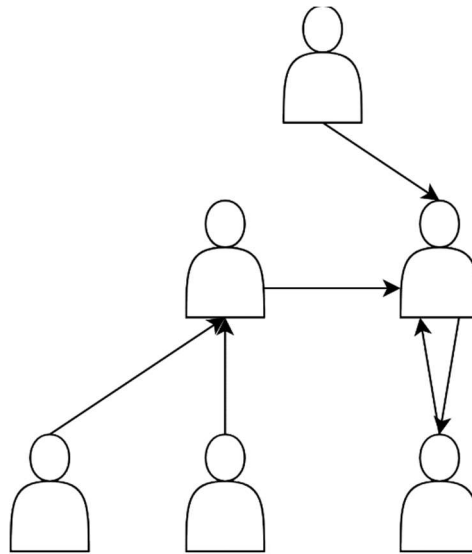
W celu badań skuteczności przygotowanych algorytmów dokonano analizę ich działania na zbiorze wygenerowanych użytkowników, którzy będą stanowić symulację dla środowiska produkcyjnego.

10.1. Miara skuteczności

Optymalizacja algorytmów wymaga przyjęcia pewnej miary, według której będzie można stwierdzić czy usprawnienie algorytmu pozwoliło osiągnąć zamierzony efekt tzn. sprawić, że jest bardziej wydajny. Przez wydajność w tym przypadku trzeba rozumieć satysfakcję użytkowników z dobieranych im par. Uczucia użytkowników są jednak bardzo subiektywne, ciężko jest określić bowiem obiektywną miarę, która byłaby niezależna od osoby i jej odczuć.

10.2. Perfekcyjny przyjaciel

Jest abstrakcyjnym pojęciem, które zostaje wprowadzone w celu pomiaru skuteczności algorytmu. Perfekcyjny przyjaciel to osoba najbliższa drugiemu użytkownikowi, w matematycznym tego słowa znaczeniu. To jest taka, której odległość cosinusowa jest najmniejsza. Odrzucając możliwość, że w grupie osób będą dwie tak samo odległe, co jest możliwe, ale z bardzo małym prawdopodobieństwem można założyć, że jeden użytkownik ma tylko jednego perfekcyjnego przyjaciela. Użytkownik jednak w tym samym czasie może być najlepszym przyjacielem dla zera lub większej liczby użytkowników.



Graf. 10.1: koncepcja perfekcyjnego przyjaciela na grafie

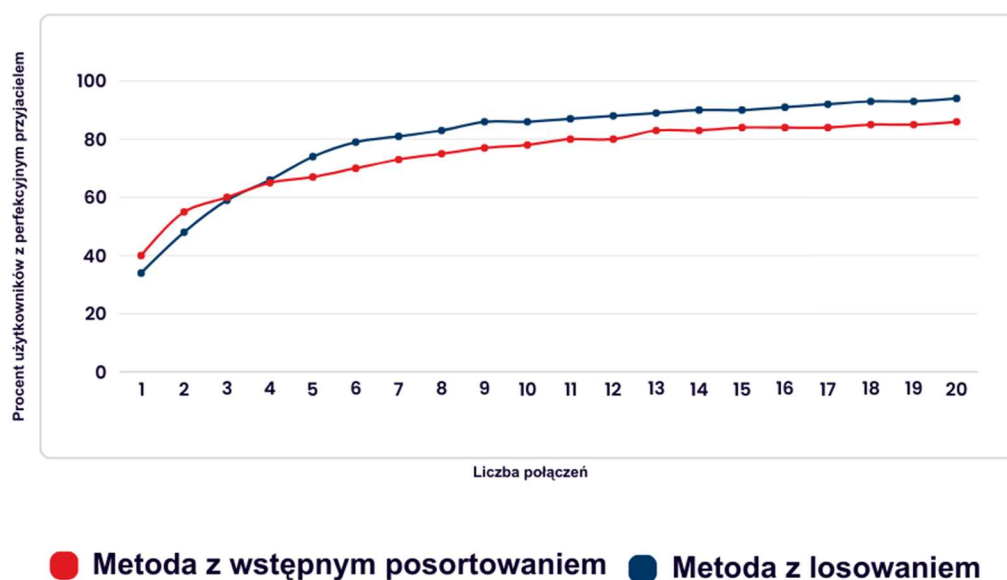
Miara z użyciem perfekcyjnego przyjaciela

Skuteczność algorytmu wyrażamy procentem użytkowników, którzy w grupie swoich połączeń mają perfekcyjnego przyjaciela.

10.3. Wyniki pomiaru algorytmów

Przeprowadzono eksperyment dla 360 wygenerowanych użytkowników, który przedstawił, jak ze wzrostem liczby połączeń rośnie szansa na sparowanie z perfekcyjnym przyjacielem. Nie jest to miara idealna, nie uwzględnia, że liczba użytkowników będzie wzrastać. Ponadto użytkownicy każdego dnia mogą przesłać nowe zdjęcie, niektórzy skorzystają z tej okazji, niektórzy nie, czego efektem może być zmiana perfekcyjnych przyjaciół każdego dnia. Można zauważyć, że jest to środowisko dynamiczne, ale jego uproszczenie powinno dać pewien pogląd, w jakim kierunku zmierzają wyniki. Użytkownicy byli generowani automatycznie oraz wykorzystano bazę około 1000 zdjęć (przedstawiające zainteresowania, codzienne aktywności, ulubione przedmioty), które w sposób losowany były umieszczane na ich profilach.

Szansa, że perfekcyjny przyjaciel jest z nami połączony w procentach przy 360 użytkownikach



Graf. 10.2: wykres prezentujący wyniki badania algorytmów

Z wykresu można odczytać, że metoda ze wstępnym posortowaniem jest lepszym rozwiązaniem w przypadku, gdy użytkownicy posiadają małą liczbę połączeń. Metoda losowa zyskuje przewagę dopiero od piątego połączenia.

Założeniem aplikacji jest jakość, a nie ilość spędzonego czasu w cyfrowym świecie, dlatego też ograniczając liczbę połączeń, jaką użytkownik będzie mieć w danym momencie np. do 5. Dlatego wybór metody ze wstępnym posortowaniem będzie lepszy dla rezultatów projektu. Niemniej jednak trzeba wziąć pod uwagę czynniki, których nie da się przewidzieć matematycznie, a więc upodobania użytkowników. W miarę postępu prac w przyszłości, planowane jest przeprowadzenie badania na rzeczywistej próbie użytkowników, którzy ocenią, czy sugerowane połączenia są dla nich jakościowe.

10.4. Połączenia zaproponowane przez algorytm

Przybliżoną odpowiedź na to, czy zaproponowany algorytm ma sens z perspektywy użytkownika, są w stanie udzielić dopasowania, jakie udało się uzyskać. Na szczególną uwagę zasługuje pierwsza tura łączenia, gdzie liczba kombinacji jest największa. W jej przebiegu wykorzystany został algorytm ze wstępnym posortowaniem. Kolejność, w jakiej pary zostały dobrane, została zachowana i w bazie danych reprezentowana jest przez unikalny identyfikator liczbowy. Im identyfikator jest większy, tym połączenie nastąpiło później.

Z wykorzystaniem zapytania SQL można prześledzić kolejność, w jakiej dobierani byli użytkownicy.

```
select m.match_id, u1.user_id, u1.username, u2.user_id, u2.username, m.created_date from matches m
join users u1 on u1.user_id = m.user_one_id
join users u2 on u2.user_id = m.user_two_id order by m.match_id asc;
```

Listing 10.1: zapytanie zwracające połączenia według chronologii

Wynik zapytania z ograniczeniem do pięciu rezultatów zaprezentowano w tabeli.

Tabela 10.1: Rezultat wyświetlenia pięciu połączeń

Match ID	Left ID	Left Username	Right ID	Right Username	Time
1	249	hondadrive	248	suebaru	12:03:43.454
2	244	truckfreak	245	pudziangurom	12:03:43.454
3	242	tcyclerbike	243	glrider	12:03:43.454
4	351	caiwashi	135	jesperez	12:03:43.454
5	239	bigfreakguy	238	bbuilderr	12:03:43.454

Wszystkie z wyjątkiem czwartej pary składają się ze starannie przygotowanych przez zespół użytkowników, którzy mieli prezentować konkretne, zazębiające się między sobą zainteresowania.

Zrezygnowano z przechowywania odległości pomiędzy użytkownikami w bazie danych, z racji braku jej biznesowego zastosowania. Biorąc jednak pod uwagę jej znaczenie podczas analizy, była zapisywana do pliku podczas przeprowadzania testów. W ten sposób uzupełniamy tabelę o brakującą informację.

Tabela 10.2: Odległość cosinusowa dla poszczególnych połączeń

Match ID	Odległość cosinusowa
1	0.155
2	0.178
3	0.194
4	0.224
5	0.250

Odległość cosinusowa reprezentująca podobieństwa pomiędzy wektorami reprezentującymi użytkowników wyrażona w przedziale $< 0,1 >$ rośnie dość dynamicznie. Oznacza to, że osób do siebie bardzo podobnych w testowanym zbiorze jest relatywnie mało.

10.4.1. Wybrane połączenia

Poniżej zostaną przedstawione połączenia, a więc zdjęcia, na podstawie których dobrano użytkowników. Dodatkowo zostaną wpisane etykiety wyodrębnione z tych zdjęć.

Połączenie 1.

Tabela 10.3: Połączenie nr 1, użytkownik nr 1

Użytkownik	(249) hondadrive
Przesłane zdjęcia	
Najczęściej wykrywane etykiety	Transportation, Car, Vehicle, Car Wheel, Sedan, Bumper, Spoke, Machine, Wheel, Tire, Alloy Wheel, City, Headlight, Urban, License Plate

Tabela 10.4: Połączenie nr 1, użytkownik nr 2

Użytkownik	(248) suebaru
Przesłane zdjęcia	
Najczęściej wykrywane etykiety	Machine, Tire, Wheel, Vehicle, Alloy Wheel, Car, Bumper, Car Wheel, Coupe, Transportation, Spoke, Sports Car, Urban, Windshield, Sedan

Połączenie 2.

Tabela 10.5: Połączenie nr 2, użytkownik nr 1

Użytkownik	(244) truckfreak
Przesłane zdjęcia	   
Najczęściej wykrywane etykiety	Transportation, Vehicle, 18-wheeler Truck, Truck, Car, Trailer Truck, Machine, Wheel, Photography, Frown, Sad, Moving Van, Advertisement, Bumper, Bus

Tabela 10.6: Połączenie nr 2, użytkownik nr 2

Użytkownik	(245) pudziangurom
Przesłane zdjęcia	  
Najczęściej wykrywane etykiety	Transportation, Vehicle, Photography, Wheel, Machine, Truck, Trailer Truck, Van, Snow, Outdoors, Advertisement, Moving Van, 18-wheeler Truck, T-Shirt, Chair

Połączenie 4.

Tabela 10.7: Połączenie nr 4, użytkownik nr 1

Użytkownik	(351) caiwashi
Przesłane zdjęcia	
Najczęściej wykrywane etykiety	Outdoors, Leisure Activities, Nature, Plant, Snow, Water, Portrait, Coat, Sport, Photography, Food, Hat, Bottle, Racket, Grapefruit

Tabela 10.8: Połączenie nr 4, użytkownik nr 2

Użytkownik	(135) jesperez
Przesłane zdjęcia	   
Najczęściej wykrywane etykiety	Sport, Glove, Food, Adventure, Hat, Nature, Leisure Activities, Outdoors, Plant, Pilates, Fitness, Hosiery, Sitting, Slope, Blonde

Połączenie 179.

Tabela 10.9: Połączenie nr 179, użytkownik nr 1

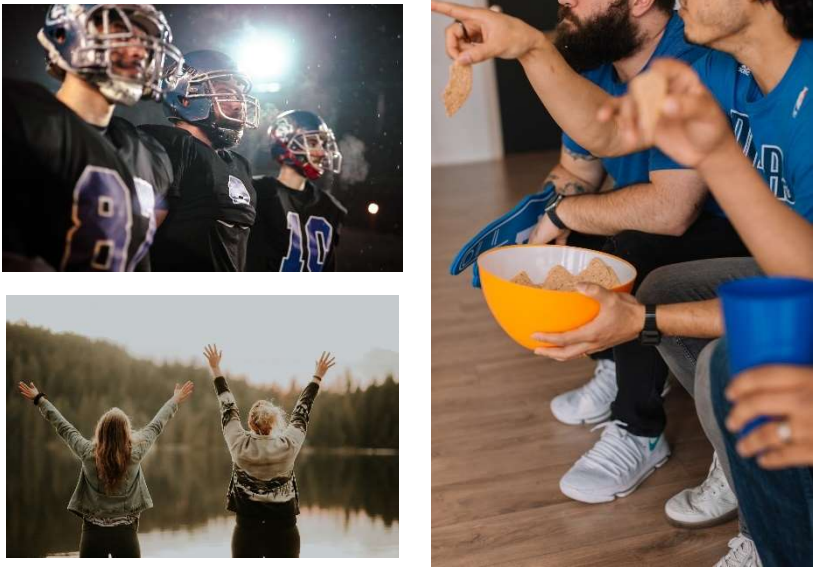
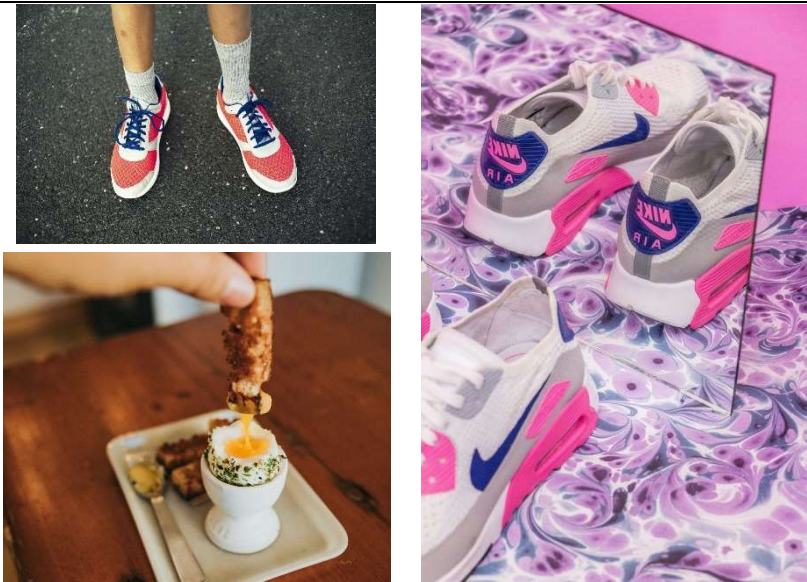
Użytkownik	(230) samadams
Przesłane zdjęcia	
Najczęściej wykrywane etykiety	American Football, Playing American Football, Football, Crash Helmet, Sport, Helmet, Football Helmet, Eating, Wristwatch, Disposable Cup, Cereal Bowl, Bread, Bowl, Food, Cup

Tabela 10.9: Połączenie nr 179, użytkownik nr 2

Użytkownik	(279) lauhowar
Przesłane zdjęcia	
Najczęściej wykrywane etykiety	Running Shoe, Sneaker, Foam, Purple, Ice Cream, Food Presentation, Brunch, Egg, Cream, Spoon, Dessert, Food, Bread, Plate, Sock

11. Interfejs graficzny aplikacji

Do utworzenia interfejsu graficznego zdecydowano się na użycie ReactJS²⁸. Jest to biblioteka języka JavaScript do budowania aplikacji przeglądarkowych z wykorzystaniem tego języka. Charakteryzuje się modularnością i możliwością ponownego użycia kodu źródłowego we wielu miejscach aplikacji. Jest jedną z najbardziej wspieranych obecnie bibliotek w swojej dziedzinie, społeczność dostarcza wiele zewnętrznych modułów, które wspomagają pracę nad projektami opartymi o ReactJS.

11.1. Budowanie aplikacji w React

React dostarcza pakiet inicjalizujący, który zestawia pierwszy projekt. W tym celu skorzystano z menadżera pakietów do Node.js (będącego środowiskiem uruchomieniowym dla języka JavaScript²⁹, odpowiedzialnym za utworzenie serwera, w którym działać będzie aplikacja w React) o nazwie npm³⁰. Za pomocą linii poleceń (w systemie Windows – „Linia poleceń”, a w systemie z dystrybucji Linux – „Terminal”), można wywołać polecenie `npx create-react-app once_a_day`. Zostanie wówczas utworzony folder o nazwie „once_a_day”, w którym znajdą się wszystkie pliki i inne foldery projektu React.

11.1.1. Konfiguracja i zależności w package.json

Plik `package.json` stanowi główne źródło wszelkich ustawień dla aplikacji. Zawiera informacje o projekcie, jego zależnościach, skryptach do uruchomienia, testowania czy budowania. Najważniejsze pola, które ten plik zawiera to:

```
"name": "oad_frontend",
```

Listing 11.1: pole `name` specyfikuje nazwę projektu.

```
"version": "0.1.0",
```

Listing 11.2: pole `version` określa aktualną wersję aplikacji.

```
"dependencies": {  
  "@apollo/client": "^3.8.7",  
  "@react-keycloak/web": "^3.4.0",  
  "graphql": "^16.8.1",  
  "keycloak-js": "^22.0.5",  
  "react": "^18.2.0",  
  "react-dom": "^18.2.0",  
  "react-hook-form": "^7.48.2",  
  "react-router-dom": "^6.20.1",  
  "react-scripts": "^5.0.1",  
  "react-stomp-hooks": "^2.1.0",  
  "web-vitals": "^2.1.4"  
},
```

Listing 11.3: pole `dependencies` przechowuje obecne zależności projektu, a więc konkretne biblioteki i pakiety wgrane do projektu. Dodatkowo jest widoczna wersja każdej z zależności.

²⁸ źródło dokumentacji: <https://react.dev/reference/react>

²⁹ źródło dokumentacji: <https://nodejs.org/en/about>

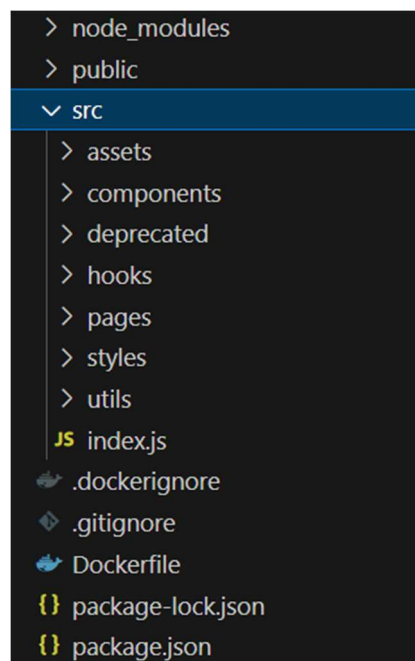
³⁰ źródło dokumentacji: <https://docs.npmjs.com/>

```
"scripts": {
  "start": "react-scripts start",
  "build": "react-scripts build",
  "test": "react-scripts test",
  "eject": "react-scripts eject"
},
```

Listing 11.4: pole `scripts` definiuje polecenia, które można wykonać z poziomu terminala przy pomocy menadżera npm. Poszczególne polecenia zajmują się uruchomieniem nowego serwera deweloperskiego, zbudowaniem aplikacji do wersji produkcyjnej oraz przetestowaniem całej aplikacji.

Aby w projekcie umieścić kolejną zależność, trzeba ponownie skorzystać z linii poleceń. Wpisując tam frazę `npm install <nazwa_zaleznosci> --save` wpisywana jest kolejna zależność do pliku `package.json`.

11.1.2. Struktura projektu



Graf. 11.1: struktura folderów w projekcie aplikacji w React JS

Poszczególne foldery wykazane powyżej zawierają:

- `node_modules` – zainstalowane pakiety na bazie `package.json`
- `src` – folder z plikami źródłowymi projektu
 - `assets` – zasoby statyczne (w tym grafiki na stronie)
 - `components` – komponenty aplikacji (w tym pasek nawigacyjny, profil użytkownika)
 - `deprecated` – nieużywane już części aplikacji
 - `hooks` – wyodrębnione moduły z logiką aplikacji
 - `pages` – poszczególne widoki wyświetlane w aplikacji
 - `styles` – pliki ze stylami aplikacji
 - `utils` – moduły odpowiedzialne za komunikację z zewnętrznymi interfejsami

11.1.3. Komponenty

Stanowią one podstawowy element w budowaniu aplikacji Reacta. Mogą dynamicznie definiować zawartość interfejsu graficznego, generując niezależne części całej strony w strukturze DOM³¹. Do wyrenderowania dynamicznych fragmentów mogą przyjmować wartości przekazane z komponentu nadrzędnego, są to tzw. *props*.

Konstrukcja

Komponenty tworzone są na podstawie szablonów JSX³² (JavaScript XML). Umożliwia to wstawianie dynamicznych wartości (takich jak zmienne) do natywnych znaczników HTML. Poniżej, zaprezentowany został kod źródłowy komponentu `Main.js`.

```
src > pages > JS Main.js > ...
1  import UserList from "../components/UserList";
2  import { useUsers } from "../hooks/useUsers";
3  import "../styles/loader.css"
4  import React from "react";
5
6  const Main = () => {
7    const { data, loading, error } = useUsers();
8    return (
9      <div className="main">
10        {loading && <div className="loading-page"></div>}
11        {error && <div className="error-page">Błąd wczytywania danych z serwera</div>}
12        {data && <UserList users={data.users} />}
13      </div>
14    );
15  };
16
17  export default Main;
18
```

Listing 11.5: komponent funkcyjny `Main`, generuje dynamiczną zawartość strony, w zależności od stanu zmiennych dynamicznych (`data`, `error` oraz `loading`).

Cała logika komponentu umieszczana jest w środku jego funkcji. Po słowie kluczowym `return` wpisywany jest szablon JSX, który może korzystać z przygotowanych lub obliczonych danych. W tym przypadku generowany jest inny komponent zagnieżdżony `<UserList>` z wart. props `users`. Wykorzystano dodatkowo tzw. szablonowy warunkowe (po ang. *conditional templates*). Służą one do umieszczenia elementów w drzewie DOM w zależności od pewnych zmiennych. W komponencie `Main` mogą wystąpić trzy różne sytuacje, w jednej zmienna `loading` jest prawdziwa (wartość boolowska `true`), dlatego wygeneruje się znacznik `<div>` z klasą `loading-page` wyświetlającą anonimowany pasek logowania. Natomiast kiedy wystąpi wartość `error`, ukaże się komunikat o błędzie. Ostatecznie, kiedy dane zostaną wczytane prawidłowo z serwera, aplikacja wygeneruje poprawny widok z załadowaną treścią. Składnia szablonów warunkowych opiera się na wyrażeniu boolowskim z logicznym „i”. Interpreter JavaScript odczytuje wartości w szablonie i sprawdza, czy po drodze wszystkie składowe wyrażenia mają wartość `true`. Jeśli tak to generuje odpowiedni znacznik. Jest to przydatne w tworzeniu dynamicznych części aplikacji.

³¹ źródło dokumentacji: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

³² źródło dokumentacji: <https://react.dev/learn/writing-markup-with-jsx>

11.1.4. Props

Wartości, które przekazywane są do komponentów-dzieci. Mogą być to proste typy danych, ale również można podać obiekty. Poprzez destrukuryzację wartości *props* można łatwo określić jakie obiekty są dostępne dla komponentu-dziecka. Można to zauważyć w komponencie `<UserList>`.

```
src > components > JS UserList.js > ...
1  import React from "react";
2  import "../styles/user-list.css";
3  import UserLink from "../UserLink";
4
5  const UserList = ({ users }) => {
6
7    return (
8      <div className="users-list">
9        {users.map((user) => (
10          <UserLink key={user.id} user={user}/>
11        ))}
12      </div>
13    );
14  };
15
16  export default UserList;
17
```

Listing 11.6: komponent funkcyjny `UserList`, w jego nawiasach podane zostaje obiekt `users`, który został zdestrukuryzowany z *props*. Następnie pola obiektu są wykorzystywane w szablonie tego komponentu do wyświetlania informacji o użytkownikach.

11.1.5. Stany oraz useHook

Każda wartość dynamiczna w obrębie jednego komponentu to stan. Sposób ich definiowania jest specyficzny w React, ponieważ oprócz nadawania im wartości początkowej, określone jest również funkcja ich zmiany.

```
src > hooks > JS useFetch.js > useFetch
1  import { useState, useEffect } from "react";
2
3  const useFetch = (endpoint, headers) => {
4    const [data, setData] = useState(null);
5    const [loading, setloading] = useState(true);
6    const [error, setError] = useState(null);
7
8    useEffect(() => {
9      fetch(endpoint, {
10        headers: headers,
11      })
12        .then((res) => {
13          if (!res.ok) {
14            throw Error("Błąd ładowania danych z serwera");
15          }
16          return res.blob();
17        })
18        .then((data) => {
19          const objectURL = URL.createObjectURL(data);
20          setData(objectURL);
21          setloading(false);
22          setError(null);
23        })
24        .catch((err) => {
25          setloading(false);
26          setError(err.message);
27        });
28    }, [endpoint]);
29  }
```

Listing 11.7: hook `useFetch`, zawiera logikę pobierania zdjęć z serwera aplikacji.

useState

Jak można zaobserwować, w definicji stanu `data` występuje również metoda `setData`. Jest to przykład `useState`³³, czyli *hooka* odpowiedzialnego za zmianę wartości stanu. Jako parametr do niego, przekazuje się wartość początkową stanu. W przypadku przedstawionego kodu źródłowego można zauważyć, że tak właśnie zdefiniowane są zmienne, które w następnej kolejności są przetwarzane w funkcji.

useEffect

Reaktywność zostaje zapewniona poprzez kolejny *hook* o nazwie `useEffect`³⁴. Jego zadaniem jest kontrolowanie listy zależności i odnotowanie zmiany stanu. Kiedy taka nastąpi, `useEffect` zajmuje się wykonywaniem logiki, która jest w nim zaimplementowana przez programistę. W przypadku pokazywanego kodu źródłowego `useEffect` służy do kontrolowania czy dany endpoint się nie zmienił i pobieranie nowego zasobu wówczas, kiedy do tego doszło. `useFetch` zwraca odebrane dane z serwera, co następnie może zostać wyświetlone w dedykowanym komponencie.

11.1.6. React Router

Wytwarzana aplikacja posiada wiele widoków, do których użytkownik może się dostać. React dostarcza mechanizmów, aby można było ustalić konkretne ścieżki w aplikacji, które prowadzą do wybranych widoków. Mechanizm ten nazywa się „React Router”, odpowiedzialny jest za hierarchię oraz przekierowywania w obrębie całego systemu. Programista może zdefiniować konkretne URI, na których przechowywane są widoki z zasobami. Taka struktura w aplikacji *Once a Day* wygląda następująco:

```
src > pages > JS OadApp.js > [e] default
14  function OadApp() {
15    return (
16      <div className="app">
17        <ReactKeycloakProvider authClient={keycloak}>
18          <BrowserRouter>
19            <Navbar />
20            <Routes>
21              <Route path="/" element={<WelcomePage />} />
22            </Routes>
23            <PrivateRoute>
24              <Routes>
25                <Route path="/users" element={<Main />} />
26                <Route path="/users/:id" element={<UserProfile />} />
27                <Route path="/me" element={<MeProfile />} />
28                <Route path="/matches" element={<MyMatches />} />
29                <Route path="/chats" element={<MyChats />} />
30              </Routes>
31            </PrivateRoute>
32          </BrowserRouter>
33        </ReactKeycloakProvider>
34      </div>
35    );
36  }
```

Listing 11.8: fragment komponentu `OadApp`, w którym umieszczono instancję React Router.

³³ Źródło dokumentacji: <https://react.dev/reference/react/useState>

³⁴ Źródło dokumentacji: <https://react.dev/reference/react/useEffect>

Aby ustalić odpowiednią hierarchię w aplikacji, przypisuje się znacznik `<Routes>`, w którym umieszcza się określone ścieżki w znaczniku `<Route>`. Pozwala to na ustalenie nazw punktów końcowych aplikacji i przypisanie im konkretnych komponentów, ale także może wprowadzić mechanizmy bezpieczeństwa (np. ścieżki prywatne po dokonaniu uwierzytelniania w aplikacji).

11.2. Integracja z logiką aplikacji

11.2.1. API

Głównym interfejsem między logiką a częścią graficzną aplikacji jest GraphQL. Aplikacja posiada wbudowanego klienta i za jego pomocą wysyła żądania po konkretne zasoby. Zaimplementowane API zwraca odpowiednie dane, a ich format oraz zakres określa aplikacja przeglądarkowa. Poniżej zaprezentowano sposób pozyskiwania zasobów językiem zapytań GraphQL.

```
src > hooks > JS useUserProfileById.js > ...
1  import { gql, useQuery } from "@apollo/client";
2
3  const GET_USER_PROFILE = gql`
4    query GetUserProfile($id: ID!) {
5      userById(id: $id) {
6        id
7        firstName
8        lastName
9        username
10       avatar
11       activities
12       weights {
13         weight
14         label
15       }
16       matches {
17         id
18         username
19       }
20     }
21   `;
22
23  export const useUserProfileById = (id) => {
24    const { data, loading, error } = useQuery(GET_USER_PROFILE, {
25      variables: {
26        id,
27      },
28    });
29
30    return {
31      data,
32      loading,
33      error,
34    };
35  };
36
```

Listing 11.9: stworzony *hook* na potrzeby pobrania danych z interfejsu GraphQL.

Za pomocą zainstalowanego klienta *apollo*³⁵, zdefiniowanie zapytanie do serwera w odpowiedniej formie. Funkcjonalność klienta pozwala już na wyodrębnienie samych danych i zapisanie ich w obiektach, które są następnie zwracane do aplikacji.

11.2.2. Keycloak

React ma także wsparcie dla serwisu autoryzacyjnego Keycloak. Ponownie zdefiniowany jest klient, któremu zleca się logikę i wykorzystanie mechanizmów, aby móc uwierzytelić lub dokonać autoryzacji użytkownika.

³⁵ źródło dokumentacji: <https://www.apollographql.com/docs/react/>

```

src > components > JS MeProfile.js > ...
1  import { useUserProfileByUsername } from "../hooks/useUserProfileByUsername";
2  import { useKeycloak } from "@react-keycloak/web";
3  import "../styles/loader.css";
4  import "../styles/user-profile.css";
5  import Activity from "../Activity";
6  import UserContent from "../UserContent";
7
8  const MeProfile = () => {
9      const { keycloak } = useKeycloak();
10     const { data, loading, error } = useUserProfileByUsername(
11         keycloak.tokenParsed.preferred_username
12     );
13
14     return (
15         <div className="main">
16             {loading && !data && <div className="loading-page"></div>}
17             {error && !data && (
18                 <div className="error-page">Błąd wczytywania danych z serwera</div>
19             )}
20
21             {data && <UserContent data={data.userByUsername} />}
22         </div>
23     );
24 }

```

Listing 11.10: fragment komponentu MeProfile, w którym zawarto dane uwierzytelniające z serwisu Keycloak, aby odpowiednio wyświetlać nazwę użytkownika. Ponownie samą logikę zajmuje się zewnętrzny pakiet, a z poziomu aplikacji jest dostęp do odpowiednich pól i metod, które Keycloak umożliwia.

```

src > components > JS LoginButton.js > ...
1  import { useKeycloak } from "@react-keycloak/web";
2
3  const Login = () => {
4      const { keycloak, initialized } = useKeycloak();
5
6      return (
7          <div className="navbar-button my-profile" onClick={() => keycloak.login()}>
8              LOGIN
9          </div>
10     );
11 };
12
13 export default Login;
14

```

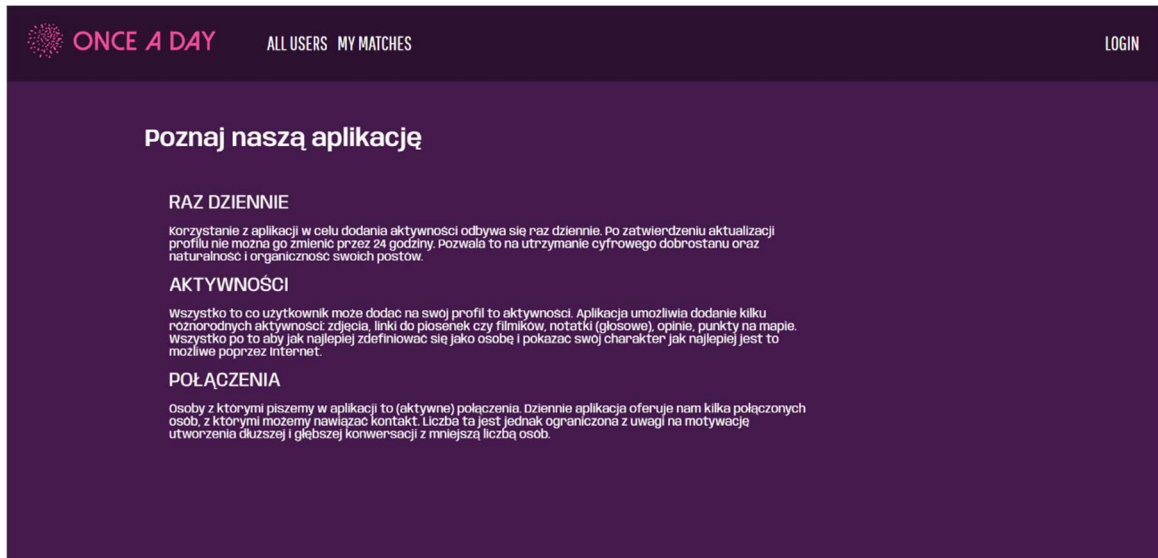
Listing 11.11: komponent LoginButton, w którym wykorzystana jest metoda `keycloak.login()`. Jej zadaniem jest przekierowanie użytkownika do serwisu logowania, a następnie przekierowania go z powrotem do aplikacji po udanym uwierzytelnieniu.

React wspiera wiele modułów ułatwiających pracę z serwisami zewnętrznymi. Tworzenie interfejsu graficznego posiadając te wszystkie narzędzia bardzo usprawniło pracę. Dzięki temu, autorzy nie musieli implementować swoich interfejsów, aby połączyć się z serwerem i otrzymać zasoby.

11.3. Graficzny Interfejs Użytkownika

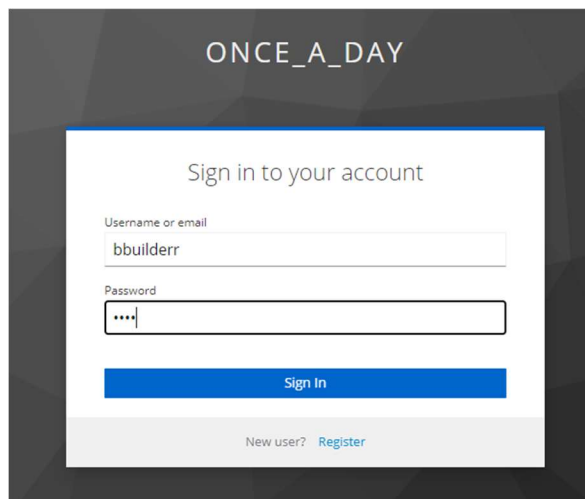
11.3.1. Strona główna i logowanie

Mimo deweloperskiego charakteru aplikacji posiada już formę przystępną dla użytkownika. Wchodząc na główną stronę ukazuje się krótkie streszczenie aplikacji, które widać na dołączonym zrzucie ekranu.



Graf. 11.2: strona domowa aplikacji

Aby przejść do ekranu uwierzytelniania, należy nacisnąć przycisk „LOGIN” w prawym górnym rogu ekranu. Przeniesie on użytkownika do zewnętrznego serwisu Keycloak, który poprosi o wpisanie loginu i hasła, co zostało zaprezentowane poniżej.

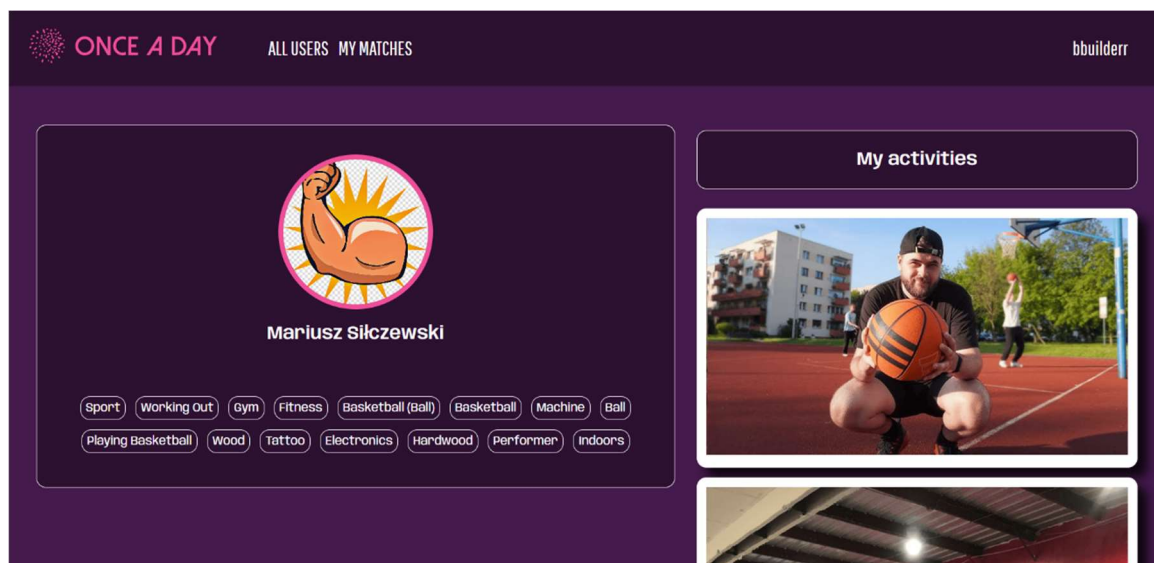


Graf. 11.3: ekran logowania aplikacji

11.3.2. Profil użytkownika i aktywności

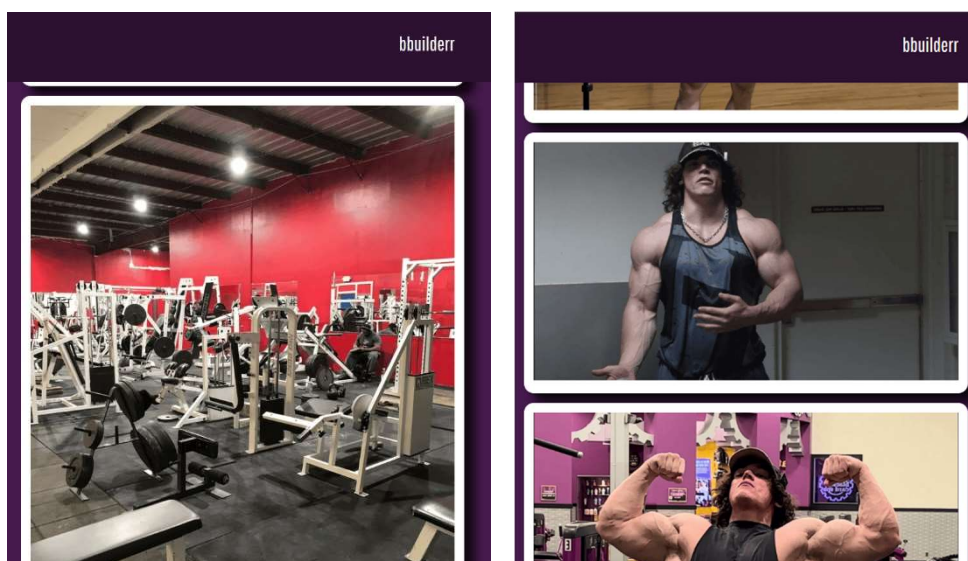
Po udanym logowaniu użytkownik ma możliwość przejścia na swój profil. Jeśli publikował już jakieś aktywności to znajdą się one w sekcji „My Activities”. Oprócz zdjęcia profilowego i nazwy użytkownika, na jego profilu widoczne są jego najbardziej popularne etykiety. Dopisane zostały, po co dobowym cyklu uruchamiania algorytmu aplikacji. Po zestawieniu zdjęć mechanizm wyodrębnił

odpowiednie etykiety i dopisał do macierzy cech użytkownika. Są one pokazywane w kolejności od najczęściej wykrytego na podstawie aktywności. Można to zauważyć na dołączonym obrazie.



Graf. 11.4: widok profilu użytkownika utworzonego na potrzeby badań algorytmów

Aplikacja wyświetla aktywności w sposób rolki zdjęć, jeden pod drugim. Można przeglądać zdjęcia wciąż mając widok na profil. Niektóre zdjęcia z profilu przedstawiono niżej.



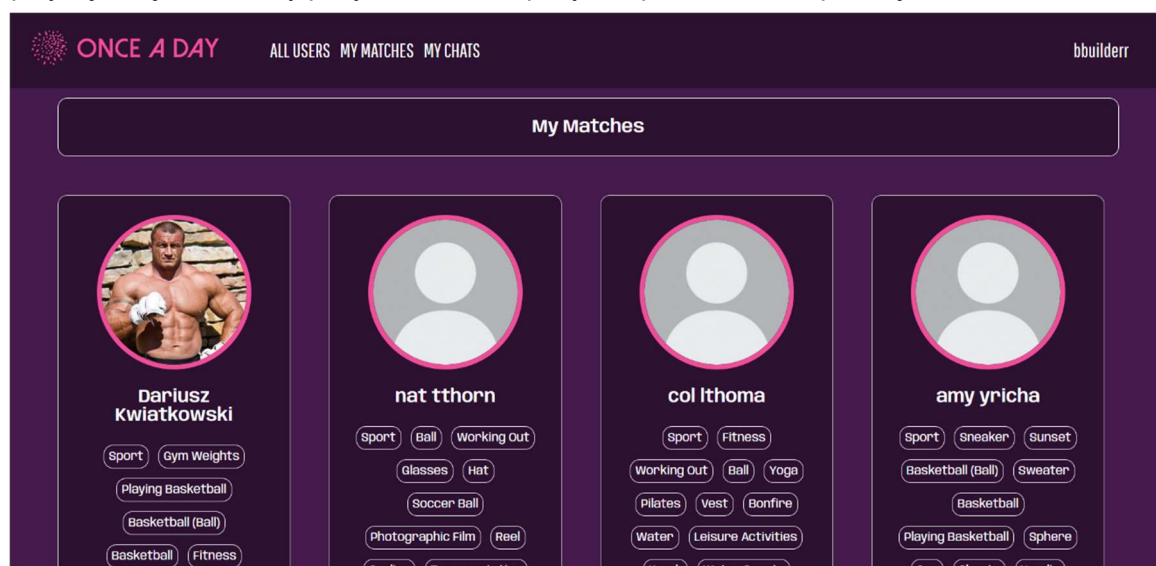
Graf. 11.5: pozostałe aktywności użytkownika

Można zauważyć, że ten użytkownik (Mariusz Siłczewski) umieszczał zdjęcia z siłowni oraz z meczów koszykówki. Określa to jego zainteresowania na profilu, niektóre z wykrytych etykiet w tłumaczeniu na polski to: „Sport”, „Trening”, „Siłownia”, „Koszykówka”, „Maszyna”, „Piłka” itd.

11.3.3. Połączenia

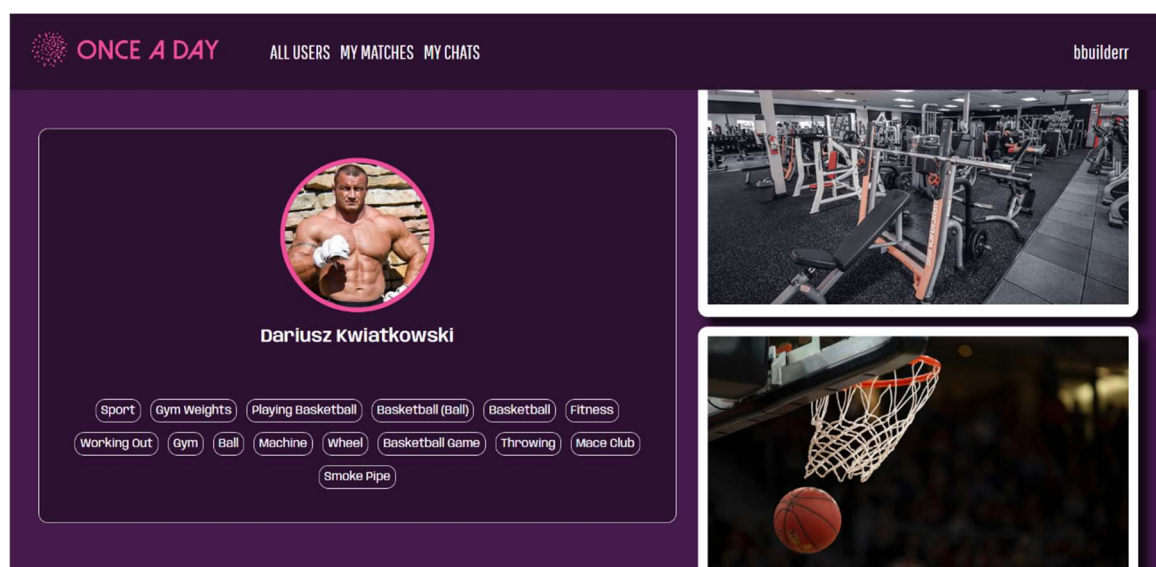
Następnie użytkownik ma możliwość podejrzenia swoich połączeń. W tym celu należy wybrać opcję „MY MATCHES” z paska nawigacyjnego i kliknąć go. Zostanie wyświetlona lista użytkowników, których profile są dostępne po naciśnięciu każdego z nich. W badaniach

przeprowadzono kilka iteracji działania algorytmu łączącego użytkowników, toteż każdy z nich ma przynajmniej dwie osoby połączone. Widok połączeń przedstawiono poniżej.



Graf. 11.6: lista połączonych użytkowników

Jeden z połączonych użytkowników (Dariusz Kwiatkowski), jak widać po zainteresowaniach z jego profilu, również mógł umieszczać zdjęcia z siłowni, jak i z koszykówki. Można zobaczyć jego aktywności klikając w jego panel.

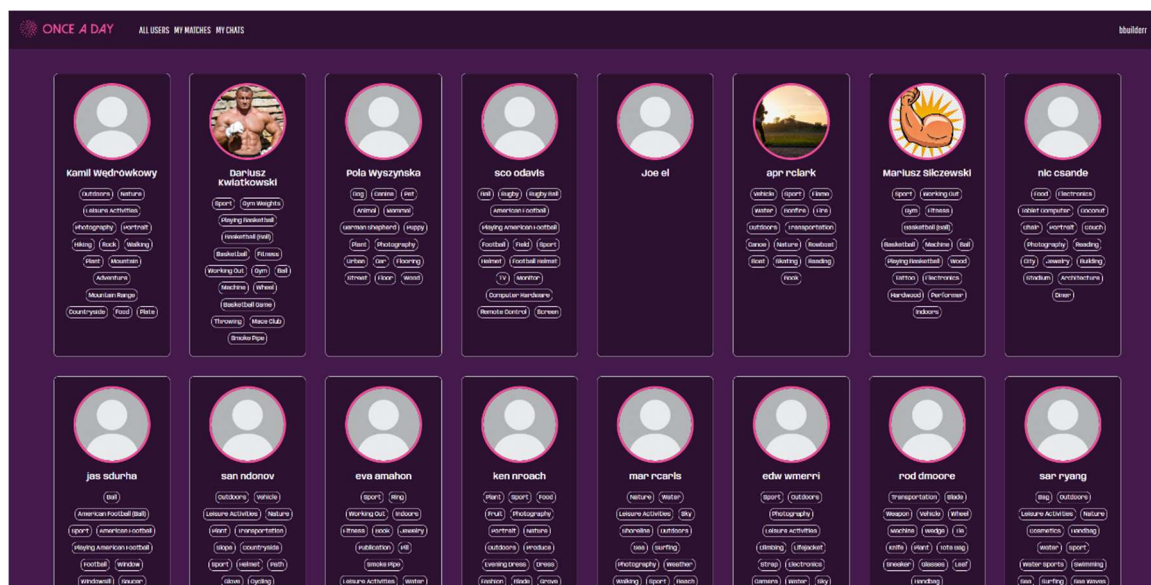


Graf. 11.7: profil połączonego użytkownika

Ponownie można rozpoznać te same aktywności na profilu drugiego użytkownika. Sięgając po kolejne połączenie z listy również widać podobieństwo w publikowanych treściach. Jako załącznik do tej pracy zostaną dołączone zrzuty ekranu aplikacji ukazujące najlepsze według autorów połączenia na podstawie przesłanych eksperymentalnie zdjęć. Tak jak wspomniano, opracowany algorytm, który zestawia dwie podobne osoby odnosi całkiem dobre efekty pod kątem zgodności matematycznej, jednakże ludzka intuicja może nie zawsze być spójna z tym, co wyliczono na podstawie pewnego wzoru. Niemniej jednak, we wielu sytuacjach te dwie wizje się ścierają i dochodzi do sukcesu.

11.3.4. Podgląd wszystkich profili użytkowników

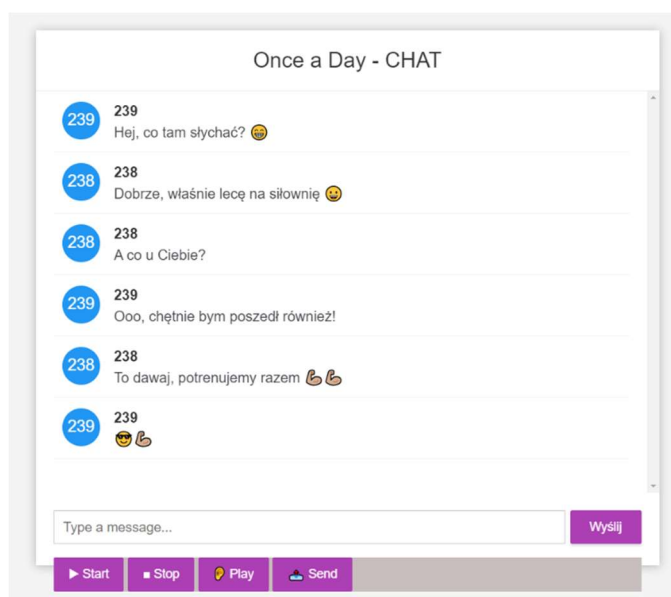
Jako dodatek deweloperski umożliwiono podgląd listy wszystkich użytkowników. Stosowano to w celu przejrzania dodanych aktywności użytkowników i porównywanie tych treści. Aby przejść do widoków wszystkich użytkowników, należy wybrać „ALL USERS” w pasku nawigacji i nacisnąć ten przycisk. Potem powinien pokazać się ten ekran:



Graf. 11.8: strona deweloperska z widokiem na wszystkich użytkowników

11.3.5. Chat

Ostatnim elementem zaimplementowanym od strony interfejsu graficznego przez zespół był *chat*, który umożliwiał użytkownikom wymianę wiadomości. Dodatkowo powstała opcja dodania wiadomości głosowych. Specjalny silnik przetwarza nagranie przed wysłaniem do odbiorcy w celu usuwania ciszy w nagraniach, co ułatwia odsłuch i skraca nagranie audio jedynie do wypowiedzi wysyłającego. Sama funkcjonalność jest jeszcze w fazie rozwoju, ale ostatecznie znalazła się w pracy. Interfejs komunikatora można zaobserwować poniżej.



Graf. 11.9: chat w wersji beta

12. Konteneryzacja i wdrożenie

Aby ułatwić wdrożenie aplikacji, zespół postanowił zastosować konteneryzację, co umożliwia modularny rozwój każdego z elementów aplikacji, jak i pozwala na wygodne uruchamianie aplikacji we wielu środowiskach. Na potrzeby deweloperskie, skonfigurowano serwer, na którym rozwijano aplikację.

12.1. Docker (Kamil Czepiel)

Konteneryzacja to proces wdrażania aplikacji, gdzie wszystkie komponenty programu niezbędne do jego uruchomienia zawarte są w kontenerach. Są to wyizolowane środowiska korzystające z jądra systemu gościa w celu uruchomienia kodu źródłowego. Kontenery są niewielkie i samowystarczalne, potrzebują niewielkiego nakładu konfiguracji, aby zestawić je na maszynie. Umożliwia to uruchomienie wielu instancji różnych serwisów aplikacji bez wysycenia zasobów³⁶. W projekcie zastosowano system Docker, który jest otwarto-źródłową platformą do tworzenia, zarządzania i uruchamiania kontenerów³⁷. Każdy poszczególny mikroservis, jak i aplikacja przeglądarkowa otrzymała swój kontener. Do tworzenia wielu kontenerów z różnymi zależnościami używa się Docker Compose³⁸. Konfiguracja grupy kontenerów odbywa się poprzez plik yaml. W obrębie konfiguracji można skonfigurować:

- przyporządkowanie portów hosta do kontenera
- sieć, w jakiej działają kontenery, tak aby była możliwa komunikacja pomiędzy kontenerami
- wolumeny pozwalające utrzymywać dane niezależnie od żywotności kontenera
- przekazywać zmienne środowiskowe

```
messenger:
  container_name: messenger
  build:
    context: .
    dockerfile: ./messenger/Dockerfile
  environment:
    SPRING_PROFILES_ACTIVE: local
  ports:
    - "8091:8091"
    - "5091:5091"
  depends_on:
    - db
    - kafka
    - eureka
  networks:
    - my-network
```

Listing 12.1: definicja uruchomienia kontenera dla serwisu *messenger*

³⁶ Źródło dokumentacji: <https://www.docker.com/resources/what-container/>

³⁷ Źródło dokumentacji: <https://docs.docker.com/guides/get-started/>

³⁸ Źródło dokumentacji: <https://docs.docker.com/compose/>

Poniżej przedstawiono efekt wykonania polecenia **docker ps**, które wyświetla wszystkie aktywne kontenery uruchomione na maszynie.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
a13ea4afd888	onceaday-file_storage	"/bin/sh -c 'java -a..."	36 hours ago	Up 36 hours	0.0.0.0:5088->5088/tcp, ::5088->5088/tcp, 0.0.0.0:8088->8088/tcp, :
0323c648fe6a	eselfff/silence-splitter:v7	"python3 -m flask ru..."	39 hours ago	Up 37 hours	0.0.0.0:8092->5000/tcp, ::8092->5000/tcp
c2d528fe5f7f	bitnami/kafka:3.5.1	"/opt/bitnami/script..."	2 days ago	Up 2 days	9092/tcp, 0.0.0.0:29092->29092/tcp, ::29092->29092/tcp
0a91d04c12f1	onceaday-messenger	"/bin/sh -c 'java -a..."	2 days ago	Up 2 days	0.0.0.0:5091->5091/tcp, ::5091->5091/tcp, 0.0.0.0:8091->8091/tcp, :
1f69ee72c5f	onceaday-api_gateway	"java -jar gateway.j..."	7 days ago	Up 2 minutes	8080/tcp, 0.0.0.0:8081->8081/tcp, ::8081->8081/tcp
5cb747649427	onceaday-core	"/bin/sh -c 'java -a..."	10 days ago	Up 7 days	0.0.0.0:8086->8086/tcp, ::8086->8086/tcp, 0.0.0.0:5087->5086/tcp, :
70cb11ebc603	onceaday-sso	"/bin/sh -c 'java -a..."	10 days ago	Up 7 days	0.0.0.0:5083->5083/tcp, ::5083->5083/tcp, 0.0.0.0:8083->8083/tcp, :
b3a8963f0a8d	oad_frontend-oad_frontend	"docker-entrypoint.s..."	2 weeks ago	Up 7 days	0.0.0.0:3000->3000/tcp, ::3000->3000/tcp
bbf3d5f5453a	onceaday-engine	"python ./main.py"	2 weeks ago	Up 7 days	0.0.0.0:8079->8079/tcp, ::8079->8079/tcp
c7b1045aed71	onceaday-keycloak	"/opt/bitnami/script..."	2 weeks ago	Up 7 days	0.0.0.0:8080->8080/tcp, ::8080->8080/tcp
ae8f2b5d4b19	onceaday-keycloak-sidecar	"/bin/sh -c 'java -j..."	2 weeks ago	Up 7 days	5085/tcp, 0.0.0.0:8084->8084/tcp, ::8084->8084/tcp
05d87d7ab1b4	onceaday-eureka	"java -jar eureka.jar"	2 weeks ago	Up 7 days	0.0.0.0:8761->8761/tcp, ::8761->8761/tcp
89aeef72f6e2	zookeeper:3.5.2	"/docker-entrypoint..."	4 weeks ago	Up 7 days	2888/tcp, 3888/tcp, 0.0.0.0:2181->2181/tcp, ::2181->2181/tcp, 8080/
2bbd55ba2385	postgres:15.4	"docker-entrypoint.s..."	7 weeks ago	Up 7 days	0.0.0.0:5432->5432/tcp, ::5432->5432/tcp

Listing 12.2: efekt polecenia **docker ps** na serwerze deweloperskim

12.1.1. Dockerfile

Kontenery Docker to uruchomione instancje obrazów. Obraz to plik, zawierający wszystkie komponenty niezbędne do uruchomienia kontenera, takie jak kod, zależności oraz środowisko uruchomieniowe. Obrazy budowane są według instrukcji zawartych w pliku Dockerfile. W przypadku częstej przebudowy obrazów konieczna jest optymalizacja tych plików, aby etap budowy przebiegał jak najszybciej.

Layers

Każda instrukcja w pliku Dockerfile to warstwa (po ang. *layer*). Warstwy po wykonaniu zapisywane są w pamięci podręcznej. Dzięki temu kolejne przebudowy wznawiane są od instrukcji, w której coś się zmieniło. Dlatego bardzo ważna jest przemyślana kolejność wykonywanych instrukcji. Te najcięższe, najrzadziej podlegające zmianom powinny znajdować się na samym początku. Przykładem takich instrukcji są zależności, ich pobieranie jest czasochłonne, ale relatywnie rzadko istnieje potrzeba dodania nowych.

Stages

Etapy wykonania pliku Dockerfile, zaczynające się od instrukcji **FROM**. Sygnalizują pobranie nowego obrazu, na którym będą wykonywane następne instrukcje. W obrębie pliku Dockerfile możliwe jest kopiowanie plików pomiędzy różnymi etapami. Technikę tą wykorzystano, aby zminimalizować rozmiar wynikowego obrazu.

W poniższym pliku Dockerfile wyróżnić można trzy etapy. Pierwszy odpowiada za kopiowanie wyłącznie plików **pom.xml**, gdzie zawarta jest informacja, z jakich zależności korzystamy oraz instalacja tych właśnie zależności. Drugi etap kopiuje plik źródłowy, pobrane zależności, a następnie buduje wynikowy plik **.jar**. Trzeci etap kopiuje jedynie wynikowy plik **.jar**, dzięki czemu końcowy obraz jest lekki. W pliku zastosowano optymalną kolejność instrukcji. Jeśli od ostatniej przebudowy zmienił się tylko plik źródłowy **.java**, a nie została dodana nowa zależność do plików **pom.xml**, proces budowy rozpocznie się od warstwy **COPY . \$HOME/**.

```

FROM maven:3.8.7-openjdk-18-slim AS DEPS
ENV HOME=/usr/src/app
ENV SERVICE=messenger

WORKDIR $HOME

COPY pom.xml $HOME/pom.xml
COPY ./eureka_server/pom.xml $HOME/eureka_server/pom.xml
COPY ./sso/pom.xml $HOME/sso/pom.xml
COPY ./sso/sso_api/pom.xml $HOME/sso/sso_api/pom.xml
COPY ./sso/sso_server/pom.xml $HOME/sso/sso_server/pom.xml
COPY ./gateway/pom.xml $HOME/gateway/pom.xml
COPY ./database/pom.xml $HOME/database/pom.xml
COPY ./file_storage/pom.xml $HOME/file_storage/pom.xml
COPY ./file_storage/file_storage_api/pom.xml $HOME/file_storage/file_storage_api/pom.xml
COPY ./file_storage/file_storage_app/pom.xml $HOME/file_storage/file_storage_app/pom.xml
COPY ./commons/pom.xml $HOME/commons/pom.xml
COPY ./core/pom.xml $HOME/core/pom.xml
COPY ./core/core_api/pom.xml $HOME/core/core_api/pom.xml
COPY ./core/core_app/pom.xml $HOME/core/core_app/pom.xml
COPY ./messenger/pom.xml $HOME/messenger/pom.xml
COPY ./messenger/messenger_api/pom.xml $HOME/messenger/messenger_api/pom.xml
COPY ./messenger/messenger_app/pom.xml $HOME/messenger/messenger_app/pom.xml
COPY ./messenger/messenger_common/pom.xml $HOME/messenger/messenger_common/pom.xml
COPY ./messenger/messenger_common/pom.xml $HOME/messenger/messenger_common/pom.xml

RUN mvn dependency:go-offline -pl :${SERVICE}

FROM maven:3.8.7-openjdk-18-slim AS BUILDER
ENV HOME=/usr/src/app
ENV APP=messenger_app

WORKDIR $HOME

COPY --from=deps /root/.m2/ /root/.m2/
COPY --from=deps $home/ $home/

COPY . $HOME/

RUN mvn -Dmaven.test.skip package -T 1.5C -am -q -pl :${APP}

FROM openjdk:18 AS BUILD
ENV HOME=/usr/src/app
ENV SERVICE=messenger
COPY --from=builder $HOME/${SERVICE}/${SERVICE}_app/target/${SERVICE}.jar /app/${SERVICE}.jar
WORKDIR /app
ENTRYPOINT java -agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=:5091 -Djava.security.egd=file:/dev/./urandom -jar ${SERVICE}.jar

EXPOSE 5091
EXPOSE 8091

```

Listing 12.3: plik Dockerfile dla serwisu *messenger*

Aby ustalić zależności i ustawienia dla danego kontenera w systemie Docker korzysta się z plików Dockerfile. Zawierają one sekwencję pewnych poleceń, które składają się na budowę całego obrazu, z którego można uruchomić kontener. W tych plikach znajdują się instrukcje dotyczące instalacji zależności, kopiowania plików, konfiguracji środowiska i innych czynności potrzebnych do skonfigurowania środowiska uruchomieniowego aplikacji.

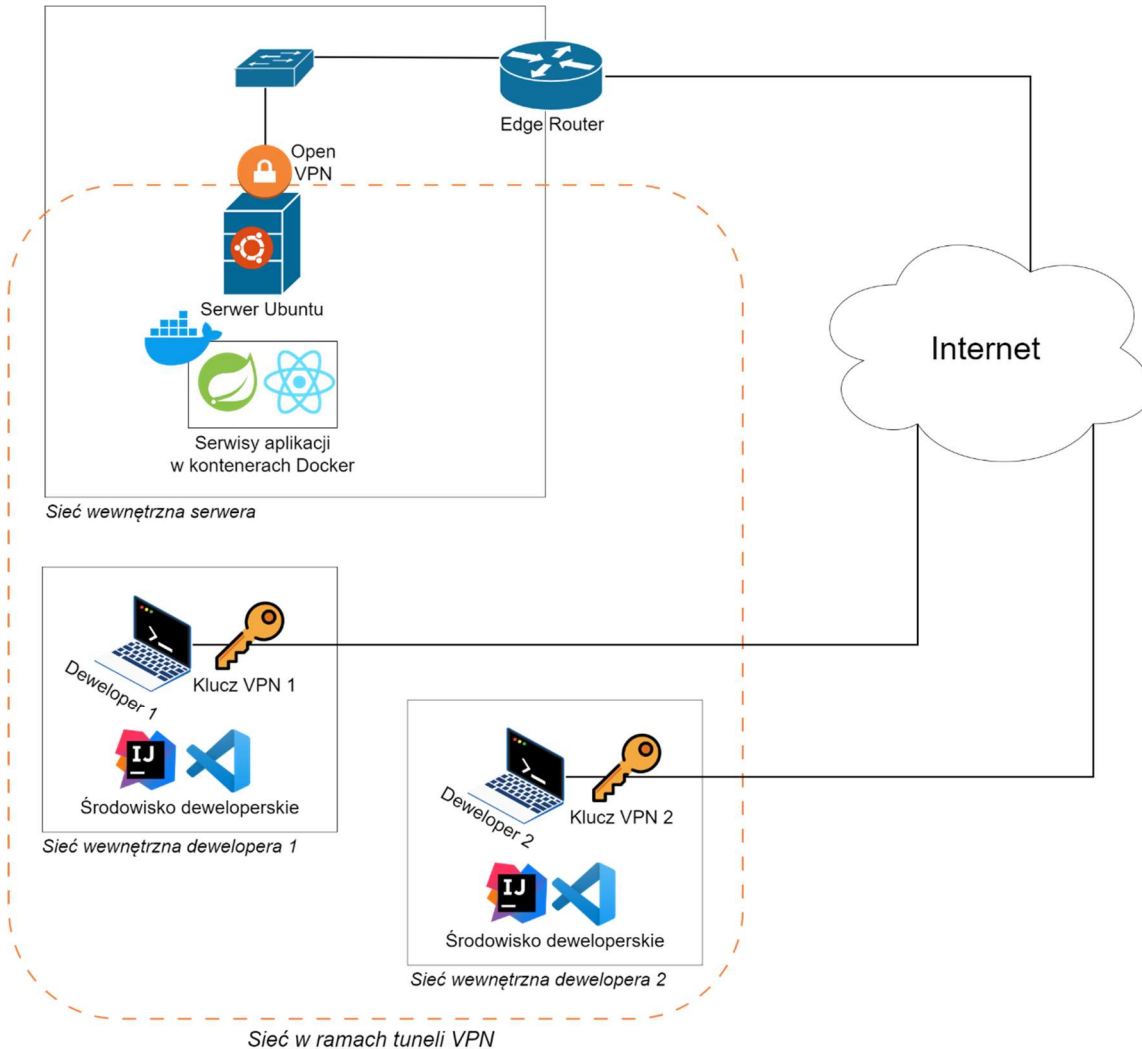
12.2. Serwer deweloperski aplikacji (Jakub Grzybowski)

Uruchomienie środowiska deweloperskiego na niezależnej maszynie, do której cały zespół miałby dostęp, było kluczowe w obliczu realizacji projektu. Zrezygnowano z oddelegowania tej usługi do zewnętrznych dostawców (takich jak Amazon ze swoimi usługami AWS czy Google z GCP) na rzecz zdalnego serwera, którzy autorzy pracy własnoręcznie skonfigurowali. Zalety i przyczyny takiego rozwiązania były następujące:

- Mniejszy nakład pieniężny, komercyjne usługi komercyjne mogły być drogie w utrzymaniu.
- Zespół inżynierski dysponował maszyną, którą mógł skonfigurować jako nieustannie działający serwer.

- Bezpośredni i odizolowany dostęp do serwera, zastosowano protokół OpenVPN do zestawienia szyfrowanego połączenia. Ograniczyło to niepożądany ruch z zewnątrz.
- Większe możliwości konfiguracji dla własnych potrzeb.

12.2.1. Topologia środowiska deweloperskiego



Graf. 12.1: topologia i sposób połączenia z serwerem deweloperskim

Do zestawienia połączenia z serwerem deweloperskim posłużyła technologia OpenVPN³⁹. Jest to otwarto-źródłowa platforma, która dostarcza narzędzi do zestawiania tuneli VPN. Poprzez protokół SSL/TLS (Secure Socket Layer/Transport Layer Security) klient uzgadnia bezpieczne połączenie poprzez tzw. *handshake* i wtedy następuje w pełni szyfrowana wymiana danych⁴⁰. Należy nadmienić, że samo połączenie jest nawiązywane przez sieć publiczną, ale dzięki asymetrycznej negocjacji klucza, ustalony jest tzw. klucz sesji, który zapewnia szyfrowanie po obu stronach (*end-to-end*). Zaprezentowana topologia na diagramie powyżej jest typowym przykładem sieci *client-to-site*, w której klient łączy się do sieci, w której znajdują się wszystkie zasoby potrzebne do realizacji projektu, w tym przypadku wytwarzania oprogramowania.

³⁹ źródło dokumentacji: <https://openvpn.net/community-resources/reference-manual-for-openvpn-2-4/>

⁴⁰ źródło dokumentacji: <https://www.ssl.com/article/ssl-tls-handshake-ensuring-secure-online-interactions/>

Zdecydowano się na wykorzystanie serwerowej dystrybucji Linux Ubuntu, który oferuje usługi do zestawienia połączenia VPN, jak i daje możliwość doinstalowania pakietów na potrzeby deweloperskie, w tym serwer Docker, poprzez którego wdrażano aplikację.

13. Podsumowanie

Podczas prac inżynierskich osiągnięto wymierny efekt w postaci wytworzenia unikatowej aplikacji. Opracowany w niej algorytm łączenia użytkowników wyróżnia ten projekt i stanowi o jego przyszłym rozwoju. Istotnym faktem jest możliwość rozbudowania aplikacji w przyszłości dzięki jej modularności, którą zapewniła wprowadzona architektura mikroserwisów. Autorzy planują dalsze usprawnienia oraz pragną spełniać kolejne założenia, aby osiągnąć w rezultacie innowacyjny produkt. Jego charakterystyka opierałaby się o główną ideę opisywaną w tej pracy – aby dać użytkownikom platformę do poznawania nowych ludzi, w przystępnej formie, dzięki skutecznemu mechanizmowi. Zachowując tym samym równowagę psychiczną w dobie wielu rozpraszaczy oraz aby unikać uzależnień od technologii. Celem na przyszłość jest zapoczątkowanie ery rozwiązań, które przyczyniłyby się na zwiększenie świadomości ludzi o cyfrowych zagrożeniach dla ich umysłu. W ciągu ostatnich lat pojawiło się wiele publikacji, które zwracają uwagę na te problemy, podpowiadają, jakie podjąć czyny, aby uniezależnić się od ekranu smartfona [16] czy komputera. Jest to misja, która zapewne będzie trwać, ale która potrzebuje szerszego rozgłosu i w efekcie skuteczności. Trzeba pamiętać, żeby się nie zatracić w wirtualnym świecie, warto dbać o ten prawdziwy i przede wszystkim mieć na uwadze drugą osobę, która jest ogromną wartością. Takie pomysły jak „Once a Day” starają się pokazać, że sensem jest człowiek i wszystko, co go określa.

13.1. Wnioski z projektu inżynierskiego

Praca nad projektem przyniosła również pewne przemyślenia oraz wskazówki na przyszłość. Poniżej wypisano najważniejsze z nich:

- Czasami celem pracy nie musi być wynik, który zakładano na początku. Podczas rozwoju aplikacji pojawiało się wiele nowych propozycji, sugestii i pomysłów. Charakterystyka projektu sprawiła, że niektóre z nich nie miały swojego pokrycia w ostatecznej wersji na zakończenie prac, ale mimo tego można uznać, że spełniono pewien cel. Mimo wielu funkcjonalności, które zaproponowano, udało się osiągnąć tą najważniejszą, czyli algorytm łączenia użytkowników. Dzięki temu aplikacja nie była tylko zwykłą koncepcją, a faktyczną podwaliną pod dalszy rozwój na bazie tego mechanizmu.
- Najprostszy algorytm nie zawsze musi dawać słabe rezultaty. Autorski algorytm opiera się na losowości. Zawiera on co prawda pewne cechy algorytmu stabilnych połączeń Gale-Shapleya, ale nie stanowi algorytmu genetycznego lub dynamicznego. Mimo tego okazało się, że testowe połączenia nie były wcale takie złe pod kątem ludzkiej percepcji. Toteż nie zawsze warto przedwcześnie optymalizować, ale najpierw przeanalizować wyniki „gorszego” algorytmu, ponieważ rezultat może być pozytywny.
- Próg wejścia w projekt informatyczny może być wysoki. Zespół pracując nad aplikacją, najwięcej czasu spędził na poprawianiu defektów i błędów. Tworzenie czegoś od zera jest ścieżką o tyle trudną, że jeszcze nigdy się nią nie szło. Kiedy przełamano pewną barierę niedoskonałości projektu, pracę zaczęły postępować o wiele szybciej. Wypracowane moduły nie potrzebowały już dużej ilości zmian i można było się szybciej skupić na

kolejnych fragmentach aplikacji. Trzeba sprawić sobie początkowo trud, aby następnie mieć pełną kontrolę nad wytwarzanym rozwiązaniem.

- Praca w dwójkę ma swoje zalety i wady. Przede wszystkim, w takim zespole nie ma wielu problemów z komunikacją. Jeśli obie strony znają swoje zadania, wiedzą co muszą poprawić albo za co wziąć odpowiedzialność to takie partnerstwo staje się wygodną opcją. Jednakże wiele razy brakowało kolejnego ogniwa, aby w tak obszernym projekcie rozłożyć pracę na więcej mniejszych części. Zespół zastanawia się nad włączeniem kolejnych osób w projekt, planuje również zmiany podejścia pracownia na zwinne (po ang. *agile*). Wtedy, sformalizowanie procesów i ról staje się trudniejsze, lecz obecnie istnieje wiele opisanych metodologii oraz *frameworków*, które jasno określają, jak rozwijać projekt w takim zespole.
- Zbieranie doświadczenia jest kluczowe. Kiedy pracuje się nad nowym rozwiązaniem, zyskuje się coś więcej niż pewien produkt. Jest to obycie w konkretnej technologii, metodologii czy sposobie wytwarzania oprogramowania. Jest to rzecz o tyle ważna, że nie jest do wyuczenia poprzez studiowanie podręczników czy czytanie dokumentacji. Faktyczna praca i podjęty wysiłek kształtują inżyniera w jego dziedzinie.

Wykaz literatury

- [1] Manfred S.: *Epidemia smartfonów*, Dobra Literatura, 2021.
- [2] Buchholz K.: *How Couples Met*, [online], 2020, odnośnik:
<https://www.statista.com/chart/20822/way-of-meeting-partner-heterosexual-us-couples/> [dostęp 2023-10-27]
- [3] Dixon S.J.: *U.S. smartphone dating app users 2019-2023*, [online], 2022, odnośnik:
<https://www.statista.com/statistics/274144/smartphone-dating-app-users-usa/> [dostęp 2023-04-17]
- [4] Hansom J.: *Gen Z's dating story: pre and post pandemic trends*, [online], 2022, odnośnik:
<https://savanta.com/eu/knowledge-centre/view/gen-zs-dating-story-pre-and-post-pandemic-trends/> [dostęp 2023-04-17]
- [5] Kuszner A.: *CYFROWY DOBROSTAN – JAK OSIĄGNĄĆ RÓWNOWAGĘ I KSZTAŁTOWAĆ ZDROWE NAWYKI*, [online], 2022, odnośnik:
https://cyberprofilaktyka.pl/blog/cyfrowy-dobrostan--jak-osiagnac-rownowage-i-ksztaltowac-zdrowe-nawyki_i30.html [dostęp 2023-10-23]
- [6] Kożewniakow M.: *Wpływ mediów społecznościowych na zdrowie psychiczne*, [online], 2022, odnośnik:
<https://wsparciepsychologiczne.psrp.org.pl/2022/11/wpływ-mediów-społecznościowych-na-zdrowie-psychiczne/> [dostęp 2023-11-06]
- [7] Czernski W., Gonciarz G.: *Ryzyko uzależnienia studentów od mediów społecznościowych na przykładzie Facebooka*, Lubelski Rocznik Pedagogiczny T. XXXVI, z. 4 – 2017, 2017
- [8] Ulatowska P.: *Aplikacja BeReal – jak działa Instagram bez filtrów i sztucznego pozowania?*, [online], 2023, odnośnik:
<https://nano.komputronik.pl/n/aplikacja-bereal-co-to-jak-dziala/>, [dostęp 2023-04-17]
- [9] Paul K.: *Meta sued by 33 states over claims youth mental health endangered by Instagram*, [online], 2023, odnośnik:
<https://www.theguardian.com/technology/2023/oct/24/instagram-lawsuit-meta-sued-teen-mental-health-us/> [dostęp 2023-12-02]
- [10] Osipenko A.: *Gale–Shapley algorithm simply explained*, [online], 2019, odnośnik:
<https://towardsdatascience.com/gale-shapley-algorithm-simply-explained-caa344e643c2/> [dostęp 2023-12-07]
- [11] Malik R.: *BeReal*, [online], 2023, odnośnik:
<https://research.contrary.com/reports/bereal> [dostęp 2023-11-04]
- [12] Sklencar A.: *20 BEREAL STATS YOU NEED TO KNOW*, [online], 2023, odnośnik:
<https://www.onlineoptimism.com/blog/bereal-stats-app-figures-data-be-real-numbers-to-know/> [dostęp 2023-04-17]
- [13] Knuth D.: *Structured Programming with go to Statements*, Stanford University, 1974
- [14] Martin R.C.: *Czysty kod. Podręcznik dobrego programisty*, Helion, 2010
- [15] Ajitsaria A.: *Build a Recommendation Engine With Collaborative Filtering*, [online], 2020, odnośnik:
<https://realpython.com/build-recommendation-engine-collaborative-filtering> [dostęp 2023-11-14]
- [16] Hansen A.: *Wyloguj swój mózg. Jak zadbać o swój mózg w dobie nowych technologii*, Znak, 2020

Spis grafik

Każda grafika z opisem, lecz bez odnośnika pochodzi z opracowania własnego.

Grafika 4.1: dedykowane filtry dla zdjęć we wczesnych wersjach Instagrama, odnośnik:
<https://www.wired.com/2011/09/instagram-update/>

Grafika 4.2: przykład interfejsu Instagram „Stories”, odnośnik:
<https://optinsta.com/using-instagram-stories-to-boost-your-brand/>

Grafika 4.3: przykład interfejsu Instagram „Reels”, odnośnik:
<https://socialpress.pl/2022/06/rolki-z-nowa-aktualizacja-co-sie-zmieni>

Grafika 4.4: materiał reklamowy Hinge, prezentujący jego funkcje i zalety, odnośnik:
<https://fortune.com/2016/10/11/hinge-new-dating-app/>

Grafika 4.5: przykładowy interfejs profilu w aplikacji Hinge, odnośnik:
<https://www.insider.com/dating-app-profile-expert-advice-replace-all-photos-except-dog-2021-9>

Grafika 4.6: przykładowy interfejs z funkcją „Your Turn”, odnośnik:
<https://eddie-hernandez.com/hinge-invited-you-to-start-the-chat/>

Grafika 4.7: komunikat o Najlepszym Dopasowaniu aplikacji Hinge, odnośnik:
<https://www.engadget.com/2018-07-12-hinge-most-compatible-suggestions.html>

Grafika 4.8: powiadomienie BeReal, odnośnik:
<https://www.businessinsider.com/guides/tech/how-to-post-on-bereal?IR=T>

Grafika 4.9: przykładowe publikacje zdjęć w aplikacji BeReal, odnośnik:
<https://petapixel.com/2022/09/07/what-makes-a-bereal-post-popular/>

Grafika 4.10: reakcje „RealMoj”, odnośnik:
<https://www.baldwinand.com/and-more/whats-be-real>

Grafika 4.11 oraz 4.12: interfejsy „Candid Story” oraz Tik Tok „Now”, odnośnik:
<https://research.contrary.com/reports/bereal>

Grafika 6.1: zrzut ekranu z tablicy Miro prezentujący mapę myśli pomysłów na funkcjonalności

Grafika 6.2: zrzut ekranu z tablicy Miro prezentujący mapę myśli pomysłów na mechanizmy łączenia

Grafika 7.1: zrzut ekranu z tablicy Miro prezentujący mapę myśli wyboru technologii

Grafika 7.2: struktura folderów w repozytorium oad_backend z logiką aplikacji

Grafika 7.3: struktura folderów aplikacji w projekcie Maven w technologii Java Spring

Grafika 7.4: struktura folderów interfejsu graficznego aplikacji w technologii React JS

Grafika 7.5: struktura folderów w repozytorium oad_automation

Grafika 8.1: wizualizacja architektury mikroserwisów w aplikacji Once a Day

Grafika 8.2: cykl autoryzacji użytkownika

Grafika 8.3: ER diagram wygenerowany przez narzędzie Dbeaver

Grafika 8.4: specyfikacja URI zapytania typu REST

Grafika 8.5: cykl wymiany wiadomości użytkowników za pomocą Websocket i protokołu STOMP

Grafika 8.6: przykładowa struktura jednego z mikroserwisów aplikacji

Grafika 9.1: wykres odległości cosinusowej w zadanej przestrzeni

Grafika 10.1: koncepcja perfekcyjnego przyjaciela na grafie

Grafika 10.2: wykres prezentujący wyniki badania algorytmów

Grafika 11.1: struktura folderów w projekcie aplikacji w React JS

Grafika 11.2: strona domowa aplikacji

Grafika 11.3: ekran logowania aplikacji

Grafika 11.4: widok profilu użytkownika utworzonego na potrzeby badań algorytmów

Grafika 11.5: pozostałe aktywności użytkownika

Grafika 11.6: lista połączonych użytkowników

Grafika 11.7: profil połączonego użytkownika

Grafika 11.8: strona deweloperska z widokiem na wszystkich użytkownikach

Grafika 11.9: chat w wersji beta

Grafika 12.1: topologia i sposób połączenia z serwerem deweloperskim

Spis listingów

Listing 8.1: skrypt w formacie XML tworzący tabelę `label_weights`, ograniczenia klucza obcego oraz indeksy na kluczowych pod względem zapytań tabel.

Listing 8.2: ograniczenie w tabeli `matches` poprzez natywne zapytanie SQL

Listing 8.3: kod kontrolera zajmującego się wyświetlaniem wiadomości

Listing 8.4: żądanie wyświetlenia profilu użytkownika w aplikacji przeglądarkowej

Listing 8.5: odpowiedź serwera dla aplikacji przeglądarkowej (w formie JSON)

Listing 8.6: żądanie wyświetlenia profilu użytkownika w aplikacji mobilnej

Listing 8.7: odpowiedź serwera dla aplikacji mobilnej (w formie JSON)

Listing 8.8: przykład zastosowania aspektu (przed podjęciem czynności)

Listing 8.9: przykład zastosowania aspektu (po podjęciu czynności)

Listing 8.10: interfejs wspólny dla każdej wiadomości `ProcessMessageStrategy.java`

Listing 8.11: implementacja metody `process` dla wiadomości AUDIO `ProcessAudioMessageImpl.java`

Listing 8.12: implementacja metody `process` dla wiadomości CHAT `ProcessChatMessageImpl.java`

Listing 8.13: kontekst rozwiązujący strategię na podstawie typu wiadomości `ProcessMessageContextImpl.java`

Listing 8.14: klasa `MessageService.java`

Listing 10.1: zapytanie zwracające połączenia według chronologii

Listing 11.1: pole `name` specyfikuje nazwę projektu.

Listing 11.2: pole `version` określa aktualną wersję aplikacji.

Listing 11.3: pole `dependencies` przechowuje obecne zależności projektu, a więc konkretne biblioteki i pakiety wgrane do projektu. Dodatkowo jest widoczna wersja każdej z zależności.

Listing 11.4: pole `scripts` definiuje polecenia, które można wykonać z poziomu terminala przy pomocy menadżera `npm`. Poszczególne polecenia zajmują się uruchomieniem nowego serwera deweloperskiego, zbudowaniem aplikacji do wersji produkcyjnej oraz przetestowaniem całej aplikacji.

Listing 11.5: komponent funkcyjny `Main`, generuje dynamiczną zawartość strony, w zależności od stanu zmiennych dynamicznych (`data`, `error` oraz `loading`).

Listing 11.6: komponent funkcyjny `UserList`, w jego nawiasach podane zostaje obiekt `users`, który został zdestrukturyzowany z `props`. Następnie pola obiektu są wykorzystywane w szablonie tego komponentu do wyświetlania informacji o użytkownikach.

Listing 11.7: hook `useFetch`, zawiera logikę pobierania zdjęć z serwera aplikacji.

Listing 11.8: fragment komponentu `OadApp`, w którym umieszczono instancję `React Router`.

Listing 11.9: stworzony hook na potrzeby pobrania danych z interfejsu `GraphQL`.

Listing 11.10: fragment komponentu `MeProfile`, w którym zawarto dane uwierzytelniające z serwisu `Keycloak`, aby odpowiednio wyświetlać nazwę użytkownika. Ponownie samą logikę zajmuje się zewnętrzny pakiet, a z poziomu aplikacji jest dostęp do odpowiednich pól i metod, które `Keycloak` umożliwia.

Listing 11.11: komponent `LoginButton`, w którym wykorzystana jest metoda `keycloak.login()`. Jej zadaniem jest przekierowanie użytkownika do serwisu logowania, a następnie przekierowania go z powrotem do aplikacji po udanym uwierzytelnieniu.

Listing 12.1: definicja uruchomienia kontenera dla serwisu `messenger`

Listing 12.2: efekt polecenia `docker ps` na serwerze deweloperskim

Listing 12.3: plik `Dockerfile` dla serwisu `messenger`

Spis tabel

Tabela 9.1: Zestawienie wykrytych etykiet z aktywności użytkowników

Tabela 9.2: Macierz cech użytkowników

Tabela 10.1: Rezultat wyświetlenia pięciu połączeń

Tabela 10.2: Odległość cosinusowa dla poszczególnych połączeń

Tabela 10.3: Połączenie nr 1, użytkownik nr 1

Tabela 10.4: Połączenie nr 1, użytkownik nr 2

Tabela 10.5: Połączenie nr 2, użytkownik nr 1

Tabela 10.6: Połączenie nr 2, użytkownik nr 2

Tabela 10.7: Połączenie nr 4, użytkownik nr 1

Tabela 10.8: Połączenie nr 4, użytkownik nr 2

Tabela 10.9: Połączenie nr 179, użytkownik nr 1

Tabela 10.9: Połączenie nr 179, użytkownik nr 2

Załączniki / dodatki

Załącznik 1: repozytorium oad_backend (plik zal1_oad_backend.zip)

Załącznik 2: repozytorium oad_frontend (plik zal2_oad_frontend.zip)

Załącznik 3: repozytorium oad_automation (plik zal3_oad_automation.zip)

Załącznik 4: wykres przedstawiający wynik badania algorytmu łączenia (plik zal4_wynik_badania_aglorytmow.png)

Załącznik 5: zrzuty ekranu z aplikacji przedstawiające przykładowe profile użytkowników i zdjęcia, na podstawie których zostali połączeni (plik zal5_zdjecia_polaczonych_uzytkownikow.zip)

Załącznik 6: topologia środowiska deweloperskiego (plik zal6_topologia.png)